VOLUME 8

Real Project, Leadership & Revision

RefinePoint case study, senior/lead questions, coding round, rapid-fire revision

Java 21 · Spring Boot 3 · SQL · Microservices · AI integration
Interview preparation for 12+ years experience · real scenarios from the RefinePoint platform

Contents

34. Real Project Case Study: RefinePoint

34.1 How to present the project
34.2 The business problem: financial reconciliation
34.3 Architecture: services and data ownership
34.4 Request flow: React → gateway → service → PostgreSQL
34.5 A reconciliation run, end to end
34.6 Multi-tenancy design
34.7 Security design
34.8 The shared platform library and auto-configuration
34.9 Transactional outbox and idempotent consumers
34.10 The rule language
34.11 Plugins with ServiceLoader
34.12 Billing: Paddle and the monday.com marketplace
34.13 AI features
34.14 Scaling strategy
34.15 Observability
34.16 Key design decisions and trade-offs
34.17 Challenges, told as STAR stories
34.18 What I would do differently
34.19 Interview questions and answers

35. Senior, Lead & Architect Questions

35.1 What interviewers assess at 12 years
35.2 The STAR method and templates
35.3 Technical leadership practices
35.4 Production incidents and postmortems
35.5 Performance improvement stories
35.6 Migration stories
35.7 Agile and Scrum in practice
35.8 Stakeholder communication
35.9 The solo-founder years, framed for employers
35.10 Questions to ask the interviewer
35.11 Interview questions and answers

36. Coding Round: Java Programs & DSA Patterns

36.1 How to run the round
36.2 Big-O cheat sheet
36.3 Java idioms for interviews
36.4 String problems
36.5 Arrays, hashing and two pointers
36.6 Binary search variants
36.7 Linked lists
36.8 Stacks and queues
36.9 Trees

36.10 Graphs
36.11 Dynamic programming
36.12 Heaps
36.13 Concurrency coding
36.14 Low-level design
36.15 Java Streams coding questions

37. Rapid-Fire Revision: 600 One-Line Answers

37.1 OOP and SOLID
37.2 Core Java language
37.3 Strings
37.4 Collections
37.5 Generics
37.6 Java 8 to 21 features
37.7 Concurrency
37.8 JVM and garbage collection
37.9 Design patterns
37.10 Spring core
37.11 Spring AOP and proxies
37.12 Spring Boot
37.13 Spring MVC and REST
37.14 WebFlux and reactive
37.15 JPA and Hibernate
37.16 Transactions
37.17 Spring Security, OAuth2 and JWT
37.18 SQL
37.19 Database internals and PostgreSQL
37.20 Caching and Redis
37.21 Microservices
37.22 Kafka and messaging
37.23 System design
37.24 DevOps, cloud and observability
37.25 Testing
37.26 AI and LLM integration
37.27 RAG, vectors and MCP

34. Real Project Case Study: RefinePoint

Every senior interview has a section that starts with "Walk me through a system you built." Most candidates lose points here, not because the system was weak, but because they tell it badly: they list technologies, skip the business problem, and cannot defend a single decision when the interviewer pushes. This chapter is the complete case study of RefinePoint, written so you can present it at three depths (two minutes, ten minutes, and a full hour of follow-up questions) and defend every decision with the alternatives you rejected.

Everything here matches the real code: class names, table names, defaults and trade-offs. When an interviewer asks "how exactly does that work?", the answer is in this chapter.

34.1 How to present the project

Interviewers are not checking whether you can recite an architecture. They are checking four things: did you understand the **business problem**, did you make **deliberate trade-offs**, do you know the **failure modes**, and can you **communicate** at the right level of detail for the listener.

The 2-minute version (memorise the shape, not the words)

> "RefinePoint is a multi-tenant SaaS for financial reconciliation. Finance teams upload bank statements, ledgers, payment-gateway settlements or tax returns, and the system matches them, shows the breaks, and drives the approval and month-end close workflow.
>
> The backend is thirteen Spring Boot 3.4 services on Java 21 behind a Spring Cloud Gateway, each owning its own PostgreSQL schema. Authentication is Keycloak with JWTs, plus workspace API keys for automation. Services call each other over HTTP with an internal token, and use Kafka only where an event has several consumers or must survive a consumer outage, always through a transactional outbox with idempotent consumers.
>
> The interesting parts are the run engine and the rule language. Runs are queued in Postgres and claimed with `SELECT … FOR UPDATE SKIP LOCKED`, so we scale by adding instances with no leader election. Each run prepares and filters its data in an in-process DuckDB that spills to disk, so that work doesn't need every row in the Java heap. Matching rules are written in a plain-English rule language I built by hand: lexer, parser, AST, a Java evaluator and a compiler to DuckDB SQL.
>
> On top there's an AI service with a provider-agnostic LLM client that drafts reconciliations from uploaded files, writes rules from a sentence and explains breaks, plus an MCP server so AI assistants can drive the product. Billing is Paddle, and we also sell through the monday.com marketplace. I built it solo, end to end, and we launch in February 2027."

That is about 250 words: problem, architecture, two deep technical points, AI, ownership. Stop there and let the interviewer pick a thread.

The 10-minute version (whiteboard order)

Minute	Topic	What to draw or say
0–1	Business problem	Two files that should agree, but don't. Breaks, approvals, close.
1–3	Architecture	Gateway, the 13 services, one Postgres with a schema each, Kafka, Keycloak
3–5	A run end to end	Ingest → queue → claim → DuckDB → results → outbox → Kafka → anomaly alert
5–6	Multi-tenancy and security	`org_id` everywhere, org-scoped paths, three security filter chains
6–7	Reliability	Outbox, idempotent consumers, heartbeats, "what happens when X is down"
7–8	Rule language	Lexer → parser → sealed `Node` AST → evaluator and SQL compiler
8–9	AI	`LlmClient`, structured JSON output, agentic rule builder, statistics for anomalies
9–10	Trade-offs and what I'd change	Postgres queues over Kafka/Redis, shared DB server, per-service DB users

> **Interview tip:** end the 10-minute version with a weakness you know about ("all services still share one database user; per-service users are next"). It shows judgement and gives the interviewer a safe place to dig instead of a place to catch you out.

Framing a solo project for an employer

Interviewers sometimes discount solo projects ("no team, no scale"). Pre-empt it: you made every decision a tech lead makes (service boundaries, data ownership, security model, deployment), you wrote the architecture document and the runbook ("when something is down" table), and you ran a real migration from a monolith to services with the public API unchanged. Say "we" for product decisions and "I" for decisions you personally made and defend.

34.2 The business problem: financial reconciliation

Reconciliation is proving that two (or more) records of the same money agree. Typical pairs:

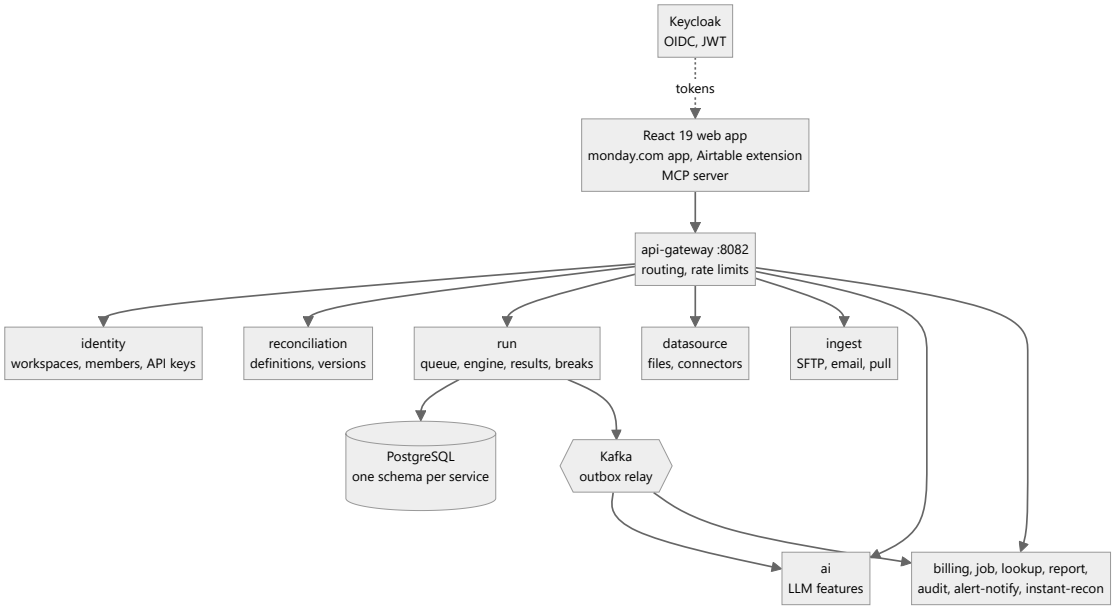
Reconciliation	Side A	Side B	Typical difficulty
Bank	Bank statement	Cash book / GL	Different references, timing, bank charges
Payment gateway	Stripe/Razorpay settlement	Orders	One payout = many orders (1:n), fees deducted
GST (India)	GSTR-2B from the tax portal	Purchase register	GSTIN keys, tax components, tolerance
Intercompany	Entity A's ledger	Entity B's ledger	Currencies, FX rates, reversals
Three/four-way	PO, goods receipt, invoice, payment		Multi-sided grouping

The output is a set of **matched** groups, **partial** matches (agree on keys but differ in amount beyond tolerance), and **unmatched** items called **breaks**. Breaks get assigned, commented on, resolved or approved, sometimes produce journals, and at month end an account is **certified** as reconciled. The engine ships with templates for these (bank, credit card, GST 2A/2B, GSTR-1 vs e-invoice, TDS 26AS, Stripe payouts, Razorpay settlements, intercompany, payroll, inventory, three-way and others) as JSON definitions.

Why it is a good interview story: it combines **data-intensive batch processing** (millions of rows, fuzzy matching), **workflow** (states, approvals, four-eyes), **strict correctness** (money), **multi-tenancy** and **audit**.

34.3 Architecture: services and data ownership

RefinePoint's backend is thirteen services and a gateway, each in its own Git repository, each owning its own data. They replaced an earlier single application (now removed) while keeping the same public API: the same paths on the same gateway port, with the same responses.



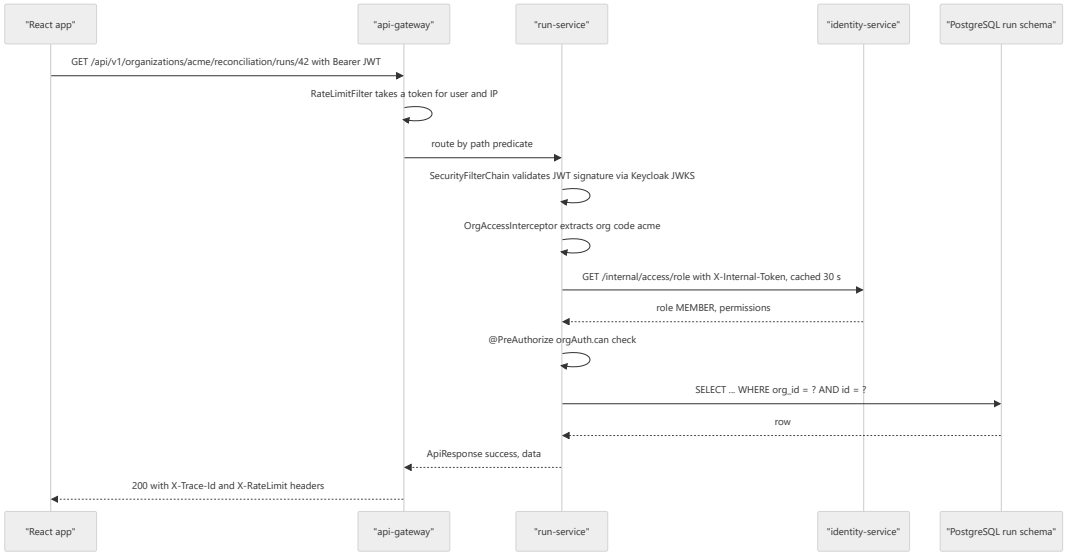
Service	Port	Owns
api-gateway	8082	Routing, CORS, rate limiting. Never routes `/internal/**`
identity-service	8101	Workspaces, members, roles, invitations, groups, API keys
billing-service	8102	Plans, subscriptions, Paddle, plan-limit checks, invoices, licence keys
datasource-service	8103	Data sources, connectors, file storage, previews, refreshes
reconciliation-service	8104	Reconciliation definitions, versions, visibility, data-quality rules
run-service	8105	Run queue, engine execution, results, breaks, workflow, open items, journals, exports
alert-notify-service	8106	Alerts, alert rules, email, webhooks, in-app notifications (SSE)
audit-service	8107	The audit log
ai-service	8108	AI assistance and per-workspace AI settings
instant-recon-service	8109	Airtable extension, monday.com app, instant reconciliations
ingest-service	8110	File routing: uploads, API, hosted SFTP inbox, inbound email, pulls
report-service	8111	Report builder and dashboards (read-only, asks owners as the caller)
lookup-service	8112	Lookup tables, workspace functions, expression catalog and tester
job-service	8113	Batch jobs and schedules with history

Four shared libraries: **ryfn-platform** (shared kernel: envelope, security, internal calls, events), **ryfn-engine** (match engine, DuckDB processing, DTOs), **ryfn-expression** (the rule language), **ryfn-plugin-api** (what plugins are written against, plain Java).

Data ownership rules. One PostgreSQL database, one schema per service (`spring.datasource.url` carries `currentSchema=${rp.schema}`). A service reads and writes only its own schema; references to other services' data are plain UUIDs with no cross-service foreign keys. Each service migrates its own schema with Flyway from `V1__baseline` , and Hibernate runs with `ddl-auto=validate` so a mismatch fails at startup instead of silently altering tables.

> **Interview tip:** when asked "why one database server and not one per service?", the answer is: logical isolation (schema per service, no cross-schema joins, no cross-service FKs) gives the ownership benefits at a fraction of the operational cost for a pre-launch product. Physical separation is a deployment change later because no code crosses schemas.

34.4 Request flow: React → gateway → service → PostgreSQL

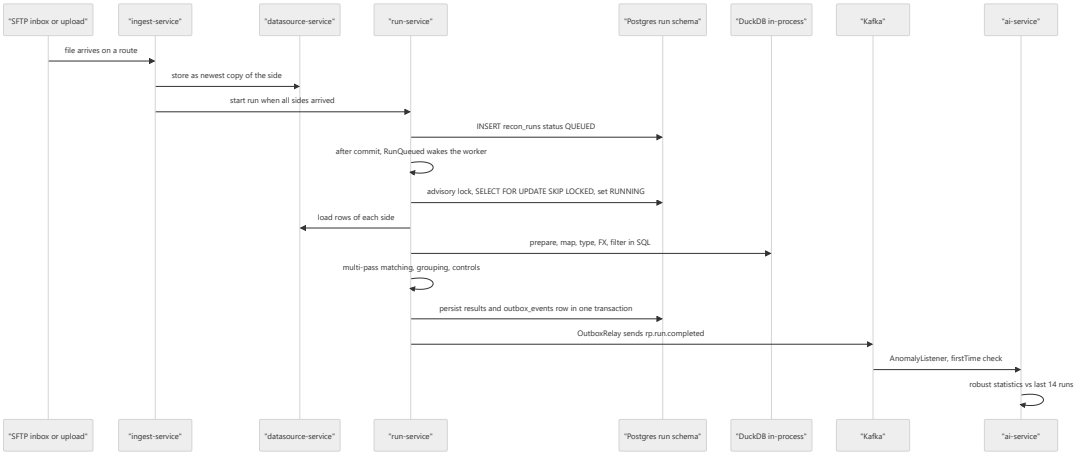


Key points to narrate:

1. **The gateway does not validate JWTs.** It reads the `sub` claim without checking the signature, purely to pick a rate-limit bucket. Every service validates the token itself (`spring.security.oauth2.resourceserver.jwt.issuer-uri`). That is defence in depth: a misrouted request or a call from inside the network is still authenticated.
2. **Org-scoped paths.** Every tenant resource lives under `/api/v1/organizations/{code}/...`. The `OrgAccessInterceptor` enforces membership on every such route in one place: reads need any role (VIEWER), writes need at least MEMBER. Finer rules are `@PreAuthorize("@orgAuth.can('breaks.resolve')")` on the controller.
3. **The `X-Org-Code` header** carries the workspace the user has open for endpoints that are not under an org path; `OrgContextService.resolveOrgId()` resolves it and checks membership and workspace locks exactly as for an org route.
4. **The `ApiResponse` envelope** (`success`, `data`, `message`, `errors`) is produced everywhere, and `GlobalExceptionHandler` maps exceptions to statuses: `EntityNotFoundException` → 404, `AccessDeniedException` → 403, `PlanLimitExceededException` → 402, `ConflictException` → 409, `RateLimitExceededException` → 429.

34.5 A reconciliation run, end to end

This is the story to tell when asked "walk me through the most complex flow".



Step 1: ingest

Every file (upload, API, SFTP inbox, pulled from a remote folder, mailed to a route's address) becomes a **submission**. The first active route by priority whose channel and file-name pattern fit decides which process side it belongs to. The route's trigger decides what happens next: `WHEN_COMPLETE` waits until every routed side has a new file (optionally within `waitHours`, else the waiting files are `TIMED_OUT` and an alert is raised), `EACH_FILE` starts a run per file, `NONE` only stores it. Files are capped at 50 MB per submission. Inbound email works through the mail provider's inbound-parse webhook posting multipart messages to `/api/v1/public/inbound-email/{secret}`.

Step 2: queue and claim

A run is created as `QUEUED` in `recon_runs`. Whoever asked (a person, a schedule from job-service, a file route, the API) gets an answer immediately; nothing runs on the instance that was asked. The worker is woken by a `@TransactionalEventListener` for `RunQueued`, which fires **after commit** (fixing a real defect where a run could start before its row was committed), and also polls every second as a fallback.

```
-- inside one transaction, simplified from RunWorker.claim()
SELECT pg_advisory_xact_lock(hashtext('run-queue'));     -- one claim at a time, cluster-wide
SELECT r.id, q.queue FROM recon_runs r
  LEFT JOIN run_queue_terms t ON t.org_id = r.org_id
  CROSS JOIN LATERAL (SELECT COALESCE(t.cap, :perWorkspace) AS cap,
                             COALESCE(t.priority, 0)       AS priority,
                             (SELECT count(*) FROM recon_runs x
                               WHERE x.org_id = r.org_id AND x.status = 'RUNNING') AS running) q
 WHERE r.status = 'QUEUED' AND q.running < q.cap
 ORDER BY q.priority DESC, q.running, r.queued_at NULLS FIRST
 LIMIT 1 FOR UPDATE OF r SKIP LOCKED;
UPDATE recon_runs SET status = 'RUNNING', instance = ?, heartbeat_at = now(), started_at = now()
WHERE id = ?;
```

Why both an advisory lock **and** `SKIP LOCKED`? `SKIP LOCKED` alone guarantees no two instances take the same row, but two instances could each count three running runs for a workspace capped at four and both claim, giving five. Serialising the claim (it takes milliseconds) makes the per-workspace cap exact. Ordering gives **fairness**: higher priority first, then the workspace with fewer runs in flight, then the oldest.

Each instance runs `runs.workers` runs at once (default 2) on a fixed thread pool and takes another only while at least `runs.min-free-heap-percent` (default 25%) of the heap is free; an idle instance always takes one, so a

huge run never starves.

Step 3: execute

`ReconciliationExecutor` takes a **snapshot** of the definition from reconciliation-service (so edits during a run do not affect it) and runs the pipeline `DataLoadStage → DataQualityStage → FilterStage → MatchStage → PersistStage` (multi-sided definitions use `MultiSidePipeline`). Inside `ReconEngine`:

1. Each side is loaded into an **in-memory DuckDB per run**, then prepared, mapped onto fields, typed, converted to one currency (`rp_fx`) and filtered **in SQL** compiled from the rule language.
2. Record controls: required fields, duplicates, GSTIN, thresholds.
3. Aggregating sides are rolled up.
4. **Passes** run in order (`PassMatcher`): 1:1 pairs are scored (amount difference, date distance, failed comparisons) and taken best first, so the result does not depend on row order; 1:n finds the smallest set of up to `groupLimit` records whose total agrees (bounded search); m:n compares totals of records sharing key values. Exact strategies (`EXACT` , `CASE_INSENSITIVE` , `IGNORE_CHARS` , `REFERENCE` , `GSTIN` , `DATE_ONLY`) narrow candidates through an index; tolerant ones are checked pair by pair.
5. Linked records are grouped, statuses decided, and run controls computed (running balance, bank reconciliation statement, ageing).

Step 4: finish and announce

`RunFinisher` writes results, records an audit event, sends a notification to the requester and publishes `RunCompletedEvent` to `rp.run.completed` through the outbox, all in the same database transaction as the status change. Two independent consumers react: reconciliation-service updates run counters ("total runs", "last run", which used to be a column update in the monolith), and ai-service checks for anomalies.

Step 5: failure paths

Failure	What happens
Any exception, including `OutOfMemoryError` in the run	Run ends `FAILED` with its reason; a person retries it (attempt counter increments)
Instance stopped cleanly	`@PreDestroy` puts its `RUNNING` runs back to `QUEUED` immediately
Instance killed	Heartbeat (every 15 s) stops; after 2 minutes `reapStale()` releases the run with `UPDATE … WHERE id IN (… FOR UPDATE SKIP LOCKED) RETURNING id` (so only one instance fails it) and marks it failed with "Retry it"
datasource-service down	Run fails with "service not available"
Kafka down	Nothing visible: the event waits in `outbox_events`

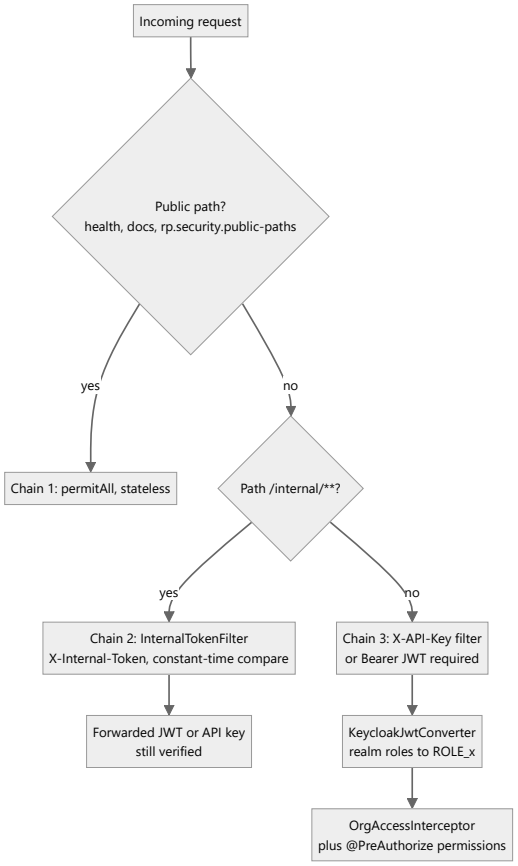
34.6 Multi-tenancy design

A tenant is a **workspace** (organisation). The model is **shared database, shared schema, discriminator column**:

- Every tenant table has `org_id`, through `BaseOrgEntity extends BaseEntity` (`@MappedSuperclass` with UUID id, timestamps, `org_id`).
- Every tenant path carries the org code; `OrgAccessInterceptor` resolves code → org id and checks role; repositories always filter by `org_id`.
- API keys are bound to exactly one workspace and role; `OrgContextService` uses the key's own workspace and ignores headers.
- Workspace roles live in identity-service; other services ask through the `WorkspaceAccess` port, implemented by `RemoteWorkspaceAccess` with answers cached for 30 seconds (`rp.cache-ttl`).
- **Workspace locks**: when a licence is suspended or a workspace archived, `WorkspaceLocks.check` allows only what a locked workspace may do (read, export, pay).
- **Noisy neighbours**: per-workspace run caps and dedicated queues (`RUN_QUEUES=acme` for an instance that serves only one big customer), per-caller rate limits at the gateway, DuckDB memory shared between concurrent runs.
- **Deletion**: deleting a workspace calls `POST /internal/workspace-purge` on every service, recorded in `workspace_purges` and retried until each has confirmed, because there is no cross-schema cascade any more.

Model	Isolation	Cost	Chosen?
Database per tenant	Strongest	Highest (connections, migrations × tenants)	No
Schema per tenant	Strong	Migrations × tenants, catalog bloat	No
Shared schema + `org_id`	Logical, enforced in code	Lowest, simplest	**Yes**
Shared schema + Postgres RLS	Logical, enforced by DB	Medium, needs session variable per transaction	Candidate for later

34.7 Security design



`PlatformSecurityAutoConfiguration` gives every service the same three ordered `SecurityFilterChain`s:

1. **Public** (`@Order(1)`): health, Swagger and whatever the service lists in `rp.security.public-paths` (supports `"POST /api/x/**"` to match one method). No sign-in.
2. **Internal** (`@Order(2)` , `/internal/**`): the `InternalTokenFilter` requires the shared `X-Internal-Token` , compared in constant time. The gateway has a `block-internal` route, so the outside world cannot reach these paths at all; the token stops anything else on the host network. A forwarded user JWT or API key is still verified so internal endpoints can act for the original caller.
3. **Everything else** (`@Order(3)`): a Keycloak bearer token or an `X-API-Key` . Stateless, CSRF disabled (no cookies), CORS handled by the gateway.

Keycloak. OIDC with PKCE for the web app; services are resource servers. `KeycloakJwtConverter` maps `realm_access.roles` to `ROLE_` authorities, used for platform roles like `super_admin` . Workspace roles are *not* in the token: they live in identity-service because a person has different roles in different workspaces and roles change without re-login.

API keys. `ApiKeyAuthenticationFilter` turns a valid key into a synthetic `JwtAuthenticationToken` for the person who created it, carrying the key's workspace and role as claims, with **no realm roles on purpose** (a key can never be admin of the platform), and a 5-minute lifetime. Rules: sending both a key and a bearer token is a 400; keys are only allowed on workspace endpoints, never for members, API keys or billing (`ApiKeyPaths`). `RemoteApiKeyVerifier` posts the key to identity-service (`/internal/api-keys/verify`) and caches the answer under the key's SHA-256 digest for at most 5 seconds, so a revoked key stops working everywhere within seconds and the plain secret is not kept in memory as a cache key. Unknown keys are cached for 5 seconds too, so repeated bad keys do not hammer identity.

Permissions. About 100 fine-grained permissions (`Permissions`), with dependencies (attaching a file to a break requires commenting, which requires viewing). Owners and admins always have all; Member and Viewer defaults are configurable; workspaces define custom roles; access can be granted per process to groups or members. Every public endpoint carries `@PreAuthorize("@orgAuth.can('…')")` . The UI hides disallowed buttons, but the services decide.

Service-to-service. `InternalClient` offers `system(service)` (acts as itself: scheduled jobs, event handlers; sends only the internal token) and `asCaller(service)` (forwards the user's bearer token or API key and workspace header). This is why the AI service, reading results for a user, can never see more than that user could.

Other controls: `OutboundUrlValidator` against SSRF on user-supplied URLs, DuckDB locked down (`enable_external_access` off, then `lock_configuration`) so a bug in generated SQL cannot read files, load extensions or reach the network; log masking (`LogMasks`); Paddle and monday webhooks verified by signature.

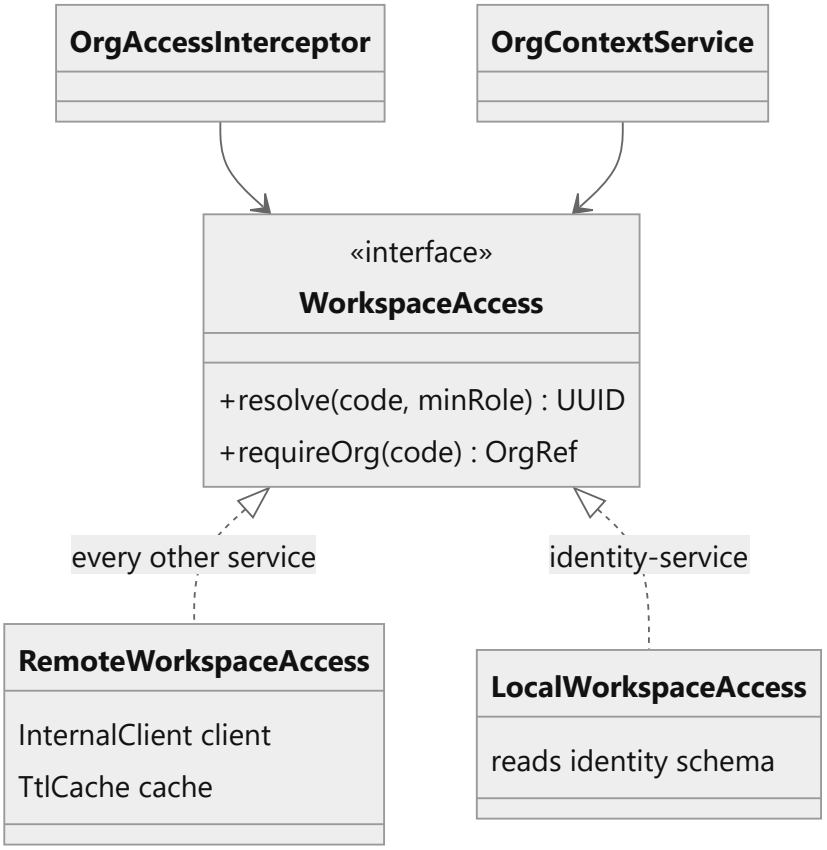
34.8 The shared platform library and auto-configuration

`ryfn-platform` is the shared kernel every service depends on. Adding the jar gives a service security, events, the envelope and observability.

```
ryfn-platform/src/main/resources/META-INF/spring/
  org.springframework.boot.autoconfigure.AutoConfiguration.imports
    PlatformSecurityAutoConfiguration      three filter chains, API keys, internal token
    PlatformRemoteAutoConfiguration        InternalClient + remote ports (WorkspaceAccess,
PlanLimits, ...)
    PlatformEventsAutoConfiguration        EventStore, EventPublisher, OutboxRelay,
ConsumedEvents, topics
    PlatformWebAutoConfiguration           registers OrgAccessInterceptor on org-scoped routes
    PlatformOpenApiAutoConfiguration       springdoc with Keycloak PKCE
    PlatformObservabilityAutoConfiguration metrics token, X-Trace-Id
META-INF/spring.factories
  EnvironmentPostProcessor = PlatformDefaults   shared defaults at LOWEST priority
```

Three techniques worth explaining in an interview:

1. **`@ConditionalOnMissingBean` ports and adapters.** The platform declares ports (`WorkspaceAccess` , `ApiKeyVerifier` , `PlanLimits` , `Alerts` , `Mailer` , `LookupSource`) and registers remote adapters (`RemoteWorkspaceAccess` , `RemoteApiKeyVerifier` , `RemotePlanLimits` …) only if the service has not defined its own. Identity-service, which owns workspaces, provides `LocalWorkspaceAccess` and automatically wins; every other service gets the remote one. Same callers, different wiring.
2. **`EnvironmentPostProcessor` defaults.** `PlatformDefaults` adds a property source *last* (lowest priority), so a service's own `application.yml` or environment always wins. It sets the Kafka connection from `KAFKA_*` variables (SASL_SSL with SCRAM-SHA-256, `acks=all` , idempotent producer), the datasource URL with the service's schema, `ddl-auto=validate` , `open-in-view=false` , Hikari pool size 5, Flyway per schema, Keycloak issuer, actuator exposure, percentile histograms, tracing sampling 10%, and where the other services live (`RP_<SERVICE>_URL`). Services set environment variables instead of repeating 80 lines of YAML.
3. **Ordering.** `PlatformEventsAutoConfiguration` is `@AutoConfiguration(after = {DataSource…, HibernateJpa…, Kafka…, Flyway…})` and the `EventStore` bean is `@DependsOnDatabaseInitialization` , so event tables are added after the service's own migrations.



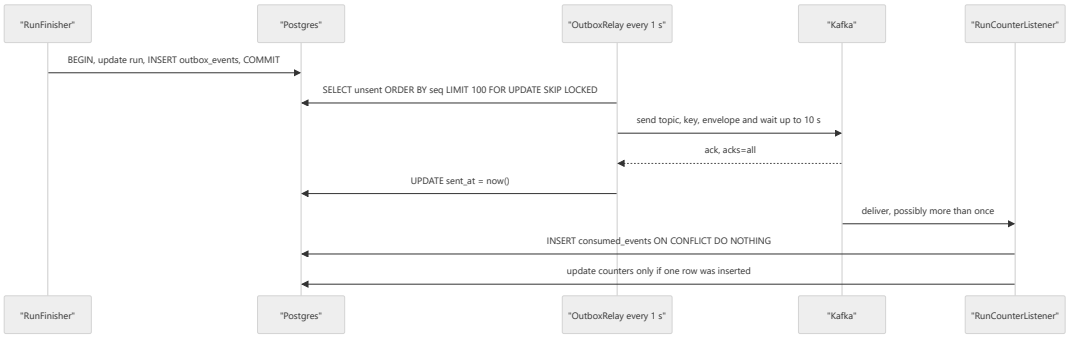
Other platform pieces: `ApiResponse` and `PagedResponse`, `BaseEntity` / `BaseOrgEntity`, `TtlCache` (a bounded `ConcurrentHashMap` with expiry, cleared when full), `SlidingWindowLimiter` (per-key `Deque<Long>` of timestamps in a rolling window, per instance), `AuditTrail`, `Notifier`, workspace and user purge hooks, and `TestDatabase` (a real embedded PostgreSQL for service tests, no Docker).

> **Trap:** not everything comes through auto-configuration. Platform classes marked `@Component` (`KeycloakJwtConverter`, `OrgAccessInterceptor`, `OrgContextService`) are found by component scanning, because every service's main class sits in the `com.ryfnflow.crm` package that also contains `com.ryfnflow.crm.common`. It works, but it is an implicit contract: a service whose main class moved to another package would lose those beans. A pure starter would declare all of them as `@Bean`s. Say this before the interviewer finds it.

> **Trap:** a shared library becomes a distributed monolith if it holds *business* logic. The rule here: the platform holds cross-cutting mechanisms (security, events, envelope), never a domain concept like "reconciliation". The cost is real, though: after changing the platform, every service needs `mvn clean package`, or unchanged services keep the old jar.

34.9 Transactional outbox and idempotent consumers

The dual-write problem: if a service commits to Postgres and then sends to Kafka, a crash between the two loses the event; if it sends first and the transaction rolls back, consumers see an event for something that never happened. The outbox fixes it by making the event part of the business transaction.




```
// EventPublisher.publish(...) - runs inside the caller's transaction
String id = UUID.randomUUID().toString();
var envelope = new EventEnvelope(id, type, source, Instant.now(),
objectMapper.valueToTree(payload));
store.jdbc().update("INSERT INTO outbox_events (event_id, topic, event_key, envelope) VALUES (?,
?, ?, ?)",
        id, topic, key, objectMapper.writeValueAsString(envelope));

// ConsumedEvents.firstTime(...) - call inside the handler's transaction
return store.jdbc().update("INSERT INTO consumed_events (consumer, event_id) VALUES (?, ?) ON
CONFLICT DO NOTHING",
        consumer, eventId) == 1;
```

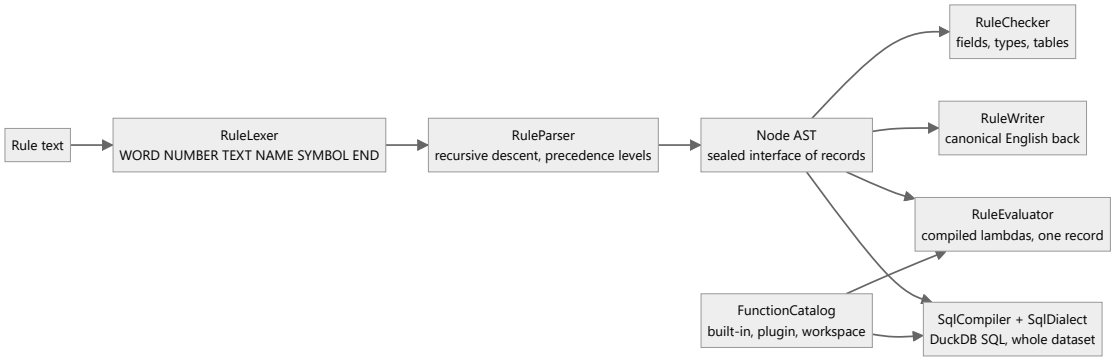
Design details interviewers probe:

- **Ordering.** Rows are read `ORDER BY seq`, and a failed send **stops the batch** (`break`) so a later event is not sent before an earlier one. The relay loops while it gets full batches of 100.
- **Multiple instances.** `FOR UPDATE SKIP LOCKED` lets every instance run the relay with no leader election. Strict global order across instances is not promised; consumers cope because each event is self-contained and deduplicated.
- **Delivery semantics.** At least once: the relay can crash after Kafka acked but before `sent_at` was committed. Consumers dedupe on the envelope's UUID per consumer name.
- **Transactional versus best-effort consumers.** `RunCounterListener` and `AuditEventListener` are `@Transactional`, so the `consumed_events` marker commits with the work and a failed handler is redelivered. `AnomalyListener` is deliberately not: "an alert is a bonus", so it catches exceptions and logs them, and a failed check is not retried.
- **Poison messages.** `ConsumedEvents.parse` returns `null` for an unreadable envelope, so one bad message cannot block a partition. `DefaultErrorHandler(new FixedBackOff(1000, 3))` retries a failing handler three times one second apart, then logs and skips. There is no dead-letter topic today.
- **Housekeeping.** Sent events are purged after 7 days, consumed ids after 30 days.
- **Partition key.** `rp.run.completed` is keyed by process id, so events for one process stay in order on one partition. Topics have 2 partitions and 2 replicas by default, because the small managed Kafka plan allows no more; both are configuration.
- **Kafka only where needed.** Three topics: `rp.audit.events` (every service → audit), `rp.run.completed` (run → reconciliation counters, AI anomaly), `rp.notifications` (any service → alert-notify). Everything else is a direct HTTP call. The rule: an event when there are several independent consumers or the consumer's outage must not block the producer.
- **Two consumer groups on one topic.** alert-notify reads `rp.notifications` twice: once in the service's shared group to *store* each notification once, and once in a per-instance group to *push* it over Server-Sent Events to whichever instance holds the user's open page.

34.10 The rule language

Filters, field mappings, calculated fields, reason codes, journal rules, workflow and certification rules are all written in one plain-English language modelled on the Natural Rule Language (NRL 1.4):

```
amount is greater than 500 and narration contains 'NEFT'
[round] amount to 2
"net" represents amount - tds, net > 0
the group of party in VENDORS default 'OTHER'
if status is one of 'OPEN', 'HELD' then 'REVIEW' else 'OK'
```



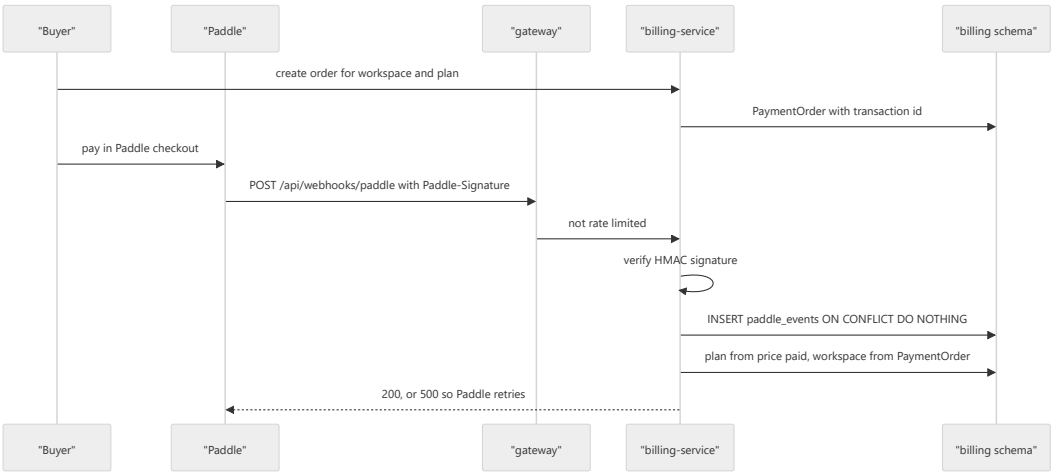
- **Lexer** turns text into tokens of six kinds. Words are case-insensitive; single-quoted text uses `''` as the escape; double-quoted names allow fields with spaces.
- **Parser** is hand-written recursive descent. Precedence lowest first: `if/then/else`, `or`, `and`, `not`, comparisons, `+ -`, `* / mod`, unary minus. Comparisons accept words or symbols (`is greater than`, `>`), `is one of`, `is between`, `is present`. Function calls read like English: `[days between] a and b`, with joiner words (`to`, `and`, `with`, `from` ...).
- **AST** is a Java 21 sealed interface `Node` with records: `Field`, `Column`, `Text`, `Num`, `Bool`, `Call`, `Arith`, `Negate`, `Compare`, `And`, `Or`, `Not`, `If`, `Let`. `RuleEvaluator.compile` is a pattern-matching `switch` over `Node`, so a new node type is a compile error where it is not handled.
- **Evaluator** compiles the tree once into nested lambdas (`RuleEvaluator.Compiled`), and `ExpressionEngine` caches parsed rules in an access-ordered `LinkedHashMap` LRU of 2,000 entries, keyed per version of the workspace's own functions.
- **SQL compiler** produces a typed `SqlExpr` with the *same semantics* as the Java evaluator: text compares ignoring case and surrounding spaces, blank text is missing (NRL has no null), `+` adds numbers and joins text. Functions with a native SQL template compile natively; others call their Java code inside DuckDB through the registered `rp_fn` function. `SqlParityTest` asserts Java and SQL agree.
- **FunctionCatalog** is the single place a function is defined: English form, typed parameters, Java implementation, optional SQL template. Infix functions (`a sounds like b`) and predicates (`date is a weekend`) are catalog entries too, so new kinds of condition need no parser change. Workspaces can define **their own functions** without a release (`[net of] amount and fee` = `amount - fee`), stored in lookup-service and checked when saved.
- The web app uses a **TypeScript port** of the parser and writer, so the rule builder can show any rule as English sentences and edit it visually.

34.11 Plugins with ServiceLoader

```
public interface DataConnector {          // ryfn-plugin-api, plain Java
    ConnectorDescriptor descriptor();
    ConnectorResult fetch(ConnectorContext ctx);
}
// a plugin jar lists its class in META-INF/services/com.ryfnflow.plugin.connector.DataConnector
List<Plugins.Found<DataConnector>> found = Plugins.load(DataConnector.class);
```

`Plugins.load` uses `ServiceLoader.load(extensionPoint, loader())`, where the loader is a `URLClassLoader` named `ryfn-plugins` over every jar in `RYFN_PLUGINS_DIR`, whose parent is the service's own class loader, so a plugin sees the API and the JDK and brings everything else itself. The loader is created lazily with double-checked locking on a `volatile` field. Iteration catches `ServiceConfigurationError`, `LinkageError` and runtime exceptions per provider, so one broken plugin is logged and skipped and the service still starts. Two extension points: `FunctionPlugin` (functions as `FunctionSpec`s, which then appear in the parser, checker, SQL compiler, the screens' function reference and the AI rule builder's guide) and `DataConnector` (shown in the connector catalog with a settings form, encrypted secrets, test and preview). Name clashes with built-ins are left out and logged. Plugins are **trusted code** installed only by whoever runs the servers.

34.12 Billing: Paddle and the monday.com marketplace



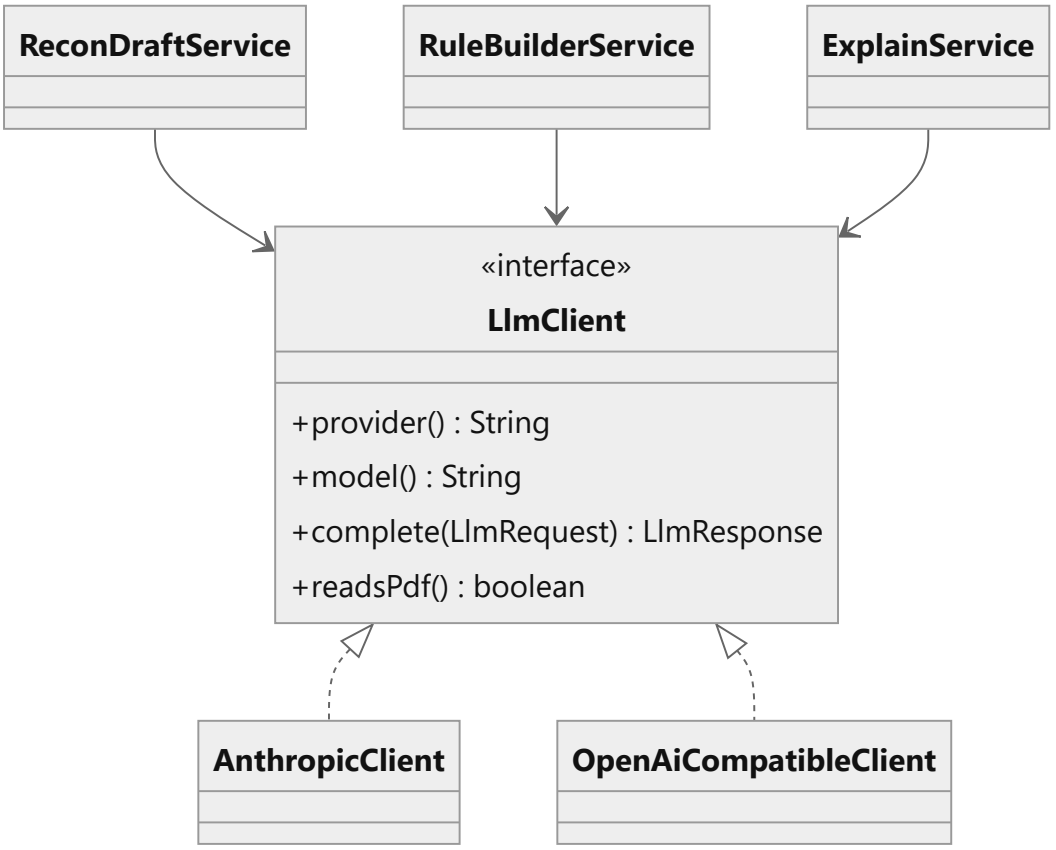
Principles:

- **Verify first, process in the same request.** The signature is checked first; processing runs in one transaction while Paddle waits; any failure answers 500 and Paddle sends the event again.
- **Idempotent by event id**: the `paddle_events` insert with `ON CONFLICT DO NOTHING` is part of the same transaction, so a failed attempt rolls back its marker too and the retry is processed.
- **Trust nothing the browser could set.** The plan comes from the price paid (tagged in Paddle's catalog), and the workspace from the `PaymentOrder` billing created or the subscription already linked, never from checkout custom data.
- **Duplicate purchases.** A workspace already paying is never moved to a second subscription; the extra one is cancelled and refunded (`DuplicatePurchases`). A purchase made on the website before signing up waits for the buyer's email and is claimed when a workspace is created.

- **Plan limits** are enforced by `PlanLimits` (remote to billing). Exceeding one throws `PlanLimitExceededException` → HTTP 402. If billing is down, checks are **allowed and logged**: a billing outage must not stop customers reconciling.
- **Licence keys**: `RPL1.<terms>.<signature>` with an Ed25519 signature. Only RefinePoint's private key can make one; the public key can verify one, so a dedicated installation could check keys without calling us.

monday.com marketplace. The monday app is billed through monday with the same plans. monday's app lifecycle webhooks (`install` , `app_subscription_created` , `app_subscription_changed` , `app_subscription_renewed` , `app_subscription_cancelled` , `uninstall` ...) arrive at `/api/webhooks/monday` in instant-recon-service, verified against the app's client secret. That service finds the account's RefinePoint workspaces and has billing apply the plan through `MarketplaceSubscriptions.apply` , which answers `APPLIED` , `ENDED` , `UNCHANGED` or `KEPT_PADDLE` : a workspace that pays through Paddle keeps that subscription, because the marketplace must never override a direct customer. When the marketplace subscription ends, the workspace returns to the free plan.

34.13 AI features



`LlmClient` is provider-agnostic, with records `LlmRequest(system, messages, temperature, maxTokens, jsonMode)` , `Message(role, content, attachments)` , `Attachment(mediaType, data)` and `LlmResponse(content, model, provider)` . `AiConfig` picks `AnthropicClient` or `OpenAiCompatibleClient` (a local GGUF model server or any OpenAI-compatible API) from `ai.provider` , with its own longer read timeout because local models are slow.

Feature	How it works
Draft a reconciliation from uploaded files	`ReconDraftService` : `ReconPromptBuilder` builds the prompt from the files' columns, the model returns JSON, `JsonExtractor` pulls the first balanced `{...}` (models wrap JSON in fences or prose), and the result becomes a draft definition
Rule builder in natural language	An agent loop: each turn the model checks a draft, tests it on sample records, inspects a lookup table, or answers. Accepted only when it checks clean, at most 8 steps; the steps are shown to the user
Mapping assist	Pairs columns of two sources, picks keys and match strategies (`MappingAssistService` , `NameMatcher` , `StrategyAdvisor`)
Break explanation and triage	`ExplainService` , `TriageService` : why a row did not match, closest counterpart, exceptions grouped by cause and ranked by value
Anomaly detection	Not an LLM: median and median absolute deviation over recent completed runs (up to 14), robust z-score threshold 4, never before 4 runs of history. Cheap, deterministic, explainable
Digests, statement extraction from PDF and images, help assistant, audit summaries	`NarrativeService` , `StatementExtractionService` , `HelpAssistant` , `AuditSummaryService` , each over `LlmClient`

Safety rules:

- AI reads results through `asCaller` , so it sees only what the user can.
- Two per-workspace `AiSettings` are **off until an admin turns them on**: `shareRowValues` (cell values, not only field names and statistics, may be sent to the model) and `bulkApprove` (suggested low-risk approvals may be applied in one step). Changing them is audited.
- AI reads and proposes; people approve. The **MCP server** (TypeScript, 22 tools such as `list_reconciliations` , `run_reconciliation` , `explain_result_row` , `triage_exceptions`) deliberately has no tool for approving rows.
- MCP is a thin layer over the REST API using a workspace API key or a Keycloak token, so the backend's normal authorisation applies, and replies are capped at 20,000 characters so a large result set cannot flood the assistant's context.

34.14 Scaling strategy

- **Stateless services.** No HTTP session, JWT on every request, files in blob storage, caches are short-TTL accelerators only. Any instance can serve any request.
- **Work in database queues.** Runs (`recon_runs`), jobs (`JobScheduler`), exports (`run_exports` , `ExportWorker`) and data-source refreshes (`data_source_refreshes` , `RefreshWorker`) are rows claimed with `FOR UPDATE SKIP LOCKED` . More throughput = more instances; no gateway change because workers pull from the database, not from requests.
- **Separation of request and work capacity.** `RUN_WORKERS=0` makes an instance answer requests only; a bigger machine can run more slots.
- **Memory-aware admission.** A run is taken only while 25% of the heap is free; DuckDB memory is a percentage of the machine split between runs in flight, spilling to disk beyond it.
- **Dedicated queues** for large tenants (`RUN_QUEUES=acme` , or `default,acme` to share).

- **Single-instance scheduled jobs** (reminders, digests, escalations) use a lease row in `scheduled_locks` (`UPDATE … WHERE locked_until <= now()`), not a distributed lock service.
- **The heap still matters.** DuckDB handles preparing, mapping, typing and filtering. Matching, grouping and controls run in Java on the records that come out, so `-Xmx` still decides how large and how many runs an instance can hold. Never say "DuckDB removed the memory limit".
- **Known limits.** The gateway rate limiter and `SlidingWindowLimiter` are per instance, so N gateways allow N times the limit. Postgres is the shared bottleneck.
- **Deployment today**: one Ubuntu server, a systemd unit template (`refinepoint@<service>`), configuration in `/etc/refinepoint/*.env`. The jars are identical everywhere; dev and production differ only in host and env files. Each repository has GitHub Actions CI; a chosen set of services can be deployed with `deploy-pick.sh` or a workflow. Kubernetes manifests are not done yet, but the services are ready for it: stateless, health probes enabled, configuration from the environment.

What has actually been measured

These are real load-test results from `ryfn-deploy/tests` on the pre-launch system, not production traffic.

Test	Result
Request capacity, dev server with 12 GB RAM, load from far away (235 ms round trip)	About 54 requests/s healthy at 20 concurrent users (p95 1,020 ms, 0% errors). Over the limit at 40 users (p95 2,076 ms)
Slowest endpoints under load	Run records and results pages; at 160 users `run: records` reached p95 25 s while reconciliation endpoints stayed under 600 ms. The bottleneck is run-service reads, not the whole system
Runs, 20,000 rows a side, simple definition, local machine	One run 17.8 s (2,186 records/s); four at once 42.5 s (3,652 records/s); exact counts every time
Runs, 50,000 rows a side, 50 check rules per record plus a 5,000-row lookup, dev server	**Failed**: timed out after about 21 minutes at 77 records/s, with `OutOfMemoryError: Java heap space` in run-service

> **Interview tip:** the failed test is your best story, not something to hide. Numbers plus one honest failure are more credible than "it scales". Always add "pre-launch load tests", and re-run the heavy test before quoting any improvement on it.

> **Trap:** the local regression baseline records 458.8 requests/s at 5 users, but 89.59% of those responses were 429s from our own gateway rate limiter, because the load test uses one API key limited to 300 per minute. Do not quote that as capacity. It is a good lesson: check the error column before believing a throughput number.

34.15 Observability

Signal	Implementation
Metrics	Micrometer → `/actuator/prometheus`, protected by `RP_METRICS_TOKEN`; every metric tagged `application` and `environment`; percentile histograms for `http.server.requests` and `http.client.requests`
Traces	Gateway starts the trace; W3C `traceparent` flows through `InternalClient` (RestClient with the observation registry) and Kafka (`observation-enabled` on template and listener); OTLP export when `OTEL_EXPORTER_OTLP_ENDPOINT` is set; sampling 10% by default
Logs	Trace and span id in every line; `LOG_FORMAT=json` switches to Elastic Common Schema structured logs
User-facing	Every response carries `X-Trace-Id`; an unexpected error shows it as a reference the customer can quote
Business	Run-queue admin page from `run_workers` (heap, slots, queues) and queue depth; status page from the gateway's public status endpoint; scrape config and alert rules in the deploy repo
Audit	Every audited action → `rp.audit.events` → audit-service

34.16 Key design decisions and trade-offs

The "cost accepted" column is what makes this table senior: every decision has one.

Decision	Alternatives considered	Why this one	Cost accepted
Work queues in PostgreSQL with `SKIP LOCKED`	Kafka, RabbitMQ, Redis lists or streams, Quartz cluster	Claim and state change are one ACID transaction with the run row; per-workspace caps and fairness are one SQL query; no extra infrastructure; the queue is queryable for the admin page	Claims are serialised, so throughput is limited by claim time; polling every second
DuckDB in-process for run data	Pure Java collections, Postgres temp tables, Spark	Columnar, vectorised, spills to disk, zero operations; rules compile to SQL	Native library in the JVM; per-run memory budgeting; lock-down needed; `DataEngine` keeps Spark open
Hand-written rule language	SpEL, MVEL, JEXL, Drools, JavaScript engine, SQL fragments	Business users read and write it; one meaning in Java *and* SQL; safe (no method calls, no reflection); precise error messages; AI can be taught it from one guide	Maintaining a parser, checker, compiler and TypeScript port; parity tests
Provider-agnostic `LlmClient`	Spring AI, LangChain4j, vendor SDK directly	Tiny surface, local model possible for privacy-sensitive customers, switch by property, trivial stub in tests	Re-implementing what frameworks give (retries, tool-calling helpers)
Shared platform library	Copy-paste per service, service mesh or sidecar	One implementation of security and events; overridable with `@ConditionalOnMissingBean`	Rebuild every service on change; version discipline
Outbox + idempotent consumers	Direct Kafka send, CDC (Debezium)	Correct under crashes with only Postgres and a scheduler	Up to about 1 s latency; outbox tables per service
HTTP by default, Kafka only for fan-out	Event-driven everything	Simpler reasoning, errors surface to the user	Temporal coupling, mitigated by the "when X is down" design
One Postgres, schema per service	Database per service	Ownership without operational cost pre-launch	Shared failure domain, one shared DB user today
Shared schema multi-tenancy with `org_id`	Schema or DB per tenant	Simple, cheap, one migration	Isolation relies on code; RLS is a candidate

34.17 Challenges, told as STAR stories

These are real, and they happened before launch. Tell them that way. Never imply a customer outage or production traffic that did not exist; an interviewer who asks "how many customers?" will find the gap in one question.

1. Splitting the monolith without breaking clients. *Situation:* one Spring Boot application held everything; the web app, Airtable extension, landing page and MCP server all depended on its API. *Task:* move to services with no client changes. *Action:* drew boundaries by data ownership; kept the same paths and port by putting the gateway on the old port 8082 with path-predicate routes; replaced each thing the single database used to do across modules explicitly (cascading deletes became purge calls recorded and retried until each service

confirms; "total runs" columns became a Kafka consumer; run numbers moved under a per-process advisory lock); wrote an end-to-end suite through the gateway, now about 100 checks. *Result:* the monolith was deleted and clients did not change. The split also exposed three defects that had been in the monolith all along: an amount with a thousands separator failed the whole run, the Airtable extension's first sign-in hit a `NullPointerException`, and a run could start before its row was committed. All three were fixed.

2. Runs that could start before they existed. *Situation:* a run was started asynchronously from inside the transaction that created it. *Task:* make the hand-off safe. *Action:* moved the trigger to a `@TransactionalEventListener` that fires after commit, with a one-second polling fallback. *Result:* the worker reads only committed rows, and because workers pull from the database, the same design allows any number of instances. *Lesson:* never give another thread an id before the transaction that created it has committed.

3. The heavy-rules load test that failed. *Situation:* a load test on the dev server ran one reconciliation of 50,000 rows a side with 50 check rules per record. *Task:* find out what ran out first. *Action:* the harness samples CPU and memory over SSH and reads the logs. CPU averaged only 15%, but run-service logged `OutOfMemoryError: Java heap space`, and the run timed out after about 21 minutes at 77 records/s. Heap, not CPU, was the limit. Runs at the time shared Spring's default thread pool, and the HTTP threads died with them. I built the run queue that same day: dedicated worker threads, admission only while 25% of the heap is free, and a run that fails cleanly with a `RESOURCES` category instead of harming the rest of the service. *Result:* run capacity became a setting (`RUN_WORKERS`, `RUN_MIN_FREE_HEAP_PERCENT`) and can be raised by adding instances. *Be honest:* this changed the failure mode and the scaling model; check the latest test results before claiming that specific workload is now fast.

4. Moving run processing into DuckDB. *Situation:* evaluating rules row by row in Java does not scale with data size. *Action:* compiled rules to SQL, moved preparing, mapping, typing, FX conversion and filtering into an in-process DuckDB, and kept a `DataEngine` interface as the seam. The next day I made memory and threads a percentage of the machine shared by the runs in flight, because a fixed amount did not suit both small and large servers. A few days later I hardened it (`enable_external_access` off, `lock_configuration`), because generated SQL should never be able to reach files or the network. *Result:* the same setting works on any machine, and the parity test guarantees the semantics did not change.

5. A fair queue across tenants. *Situation:* with a shared queue, one tenant's month-end batch could delay everyone. There were no customers yet, so this was designed for and then tested rather than suffered. *Action:* per-workspace caps, ordering by "fewest runs in flight", priority and dedicated queues; serialised claims with a transaction-scoped advisory lock so the cap is exact. *Result:* the cap holds by construction. Load-test scenario Q asks for 30 runs of 50,000 rows a side at once and checks every run completes with the right counts.

34.18 What I would do differently

1. **Per-service database users from day one.** Today schemas separate data logically, but one user could technically read every schema. It is on the "not done" list.
2. **Row-level security as a second tenant guard.** `org_id` filtering is enforced by code; Postgres RLS with a per-transaction `SET LOCAL` would make a missed `WHERE` harmless.
3. **Distributed rate limiting.** In-memory buckets are per gateway; a Redis token bucket is needed once there are several gateways.
4. **Contract tests between services.** Internal endpoints are verified by the end-to-end suite; consumer-driven contracts would catch breaking changes earlier.

5. **Fewer services at first.** Thirteen services for one developer is a lot of deployment surface. Some (lookup, report, audit) could have stayed modules of a modular monolith until load or team size demanded more.
6. **Load test earlier.** The heap limit was found after the architecture was in place; a heavy-rules test in the first month would have shaped the run design sooner.

34.19 Interview questions and answers

Q1. Tell me about your project in two minutes.

Use the script in 34.1: business problem (financial reconciliation), architecture (13 Spring Boot 3.4 services on Java 21 behind a gateway, schema per service, Keycloak, Kafka through an outbox), two deep technical points (Postgres run queue with `SKIP LOCKED` and DuckDB processing; a hand-written rule language compiled to SQL), AI (provider-agnostic client, MCP server), billing (Paddle and monday.com), and ownership (solo, launching February 2027). Stop and let them choose.

Q2. Why microservices at all, for a solo developer?

The workloads differ sharply (memory-heavy runs, light CRUD, slow LLM calls), so independent scaling and failure isolation have real value, and clear data ownership kept a growing codebase understandable. The cost is operational surface, reduced with a shared platform library, one systemd template, identical jars per environment and one database server. With hindsight, I would start some services as modules (see 34.18). Admitting that scores higher than defending thirteen services as obviously right.

Q3. How did you decide service boundaries?

By data ownership and change rate, not by layers. Each service owns tables nobody else writes: identity owns membership, reconciliation owns definitions, run owns results. Things that must change together in one transaction stayed together: runs, breaks, workflow, journals and open items are all in run-service.

Q4. How does a request reach the database?

React sends the Keycloak JWT to the gateway; `RateLimitFilter` charges per-user and per-IP buckets; the path predicate routes it; the service validates the JWT, `OrgAccessInterceptor` checks membership for the org in the path (asking identity, cached 30 s), `@PreAuthorize` checks the permission, and the repository queries by `org_id`. The response returns in the `ApiResponse` envelope with `X-Trace-Id`.

Q5. Why doesn't the gateway validate the JWT?

Because every service does, and services must not assume a request came through the gateway. The gateway only decodes `sub` without verifying it, to choose a rate-limit bucket. A forged token is counted against the address bucket too and is rejected by the service.

Q6. Explain the run queue.

Runs are rows in `recon_runs` with status `QUEUED`. Each run-service instance has a worker that, while it has free slots and free heap, claims the next run in one transaction: an advisory lock serialises claims cluster-wide, then `SELECT … FOR UPDATE OF r SKIP LOCKED` picks the best candidate by priority, the workspace's in-flight count and age, respecting the per-workspace cap, and marks it `RUNNING` with the instance name. A heartbeat every 15 s keeps it alive.

Q7. What does SKIP LOCKED do exactly?

`FOR UPDATE` locks the selected rows; `SKIP LOCKED` makes the query skip rows already locked by another transaction instead of waiting. Concurrent workers each get a different row without blocking. It exists for queue tables (Postgres 9.5+). The view is intentionally not consistent, which is fine for "give me any available job".

Q8. If SKIP LOCKED prevents double claims, why the advisory lock?

For the per-workspace cap. Two instances could both count three running runs for a workspace capped at four and both claim, giving five. `pg_advisory_xact_lock(hashtext('run-queue'))` serialises the claim so the count is exact, and it is released automatically at commit.

Q9. What happens if an instance dies mid-run?

Its heartbeat stops. Every instance's heartbeat task looks for `RUNNING` runs whose heartbeat is older than 2 minutes and releases them with `UPDATE … WHERE id IN (SELECT … FOR UPDATE SKIP LOCKED) RETURNING id`, so exactly one instance handles each; the run is failed with a "retry it" reason. On a clean shutdown, `@PreDestroy` puts the instance's runs straight back to `QUEUED`.

Q10. Why not automatically retry failed runs?

A run that exhausted an instance's memory would probably do it again, and retrying set-up failures only repeats the same error. Failing visibly with a reason and a category (`RESOURCES`, `TIMEOUT`, `VALIDATION`, `UPSTREAM`, `INTERNAL`), and letting a person retry, is safer for financial workflows. A retry increments `attempt` and counts against plan limits again. Only one run per process may be queued or running at a time.

Q11. Why Postgres for queues instead of Kafka or Redis?

Because the queue's state *is* business state: claiming a run and marking it `RUNNING` must be atomic with the run row, caps need a count over other rows, and the admin page must query the queue. Postgres gives all of that in one transaction with no extra infrastructure. Kafka is a log, not a work queue: per-message acknowledgement, priorities, per-tenant caps and "your place in line" are awkward, and partitions cap consumer parallelism. Redis would add a second source of truth to keep consistent. Runs last seconds to minutes, so claim rate is far below what Postgres handles.

Q12. When would you move off the Postgres queue?

When claim throughput or polling load becomes measurable on the database: thousands of short tasks per second, or hundreds of polling workers. Then a dedicated broker, with Postgres remaining the state of record.

Q13. Why DuckDB?

Runs need joins, aggregation, type conversion and filtering over large files. DuckDB is an embedded columnar, vectorised engine: it runs inside the JVM with no server, uses all cores, and spills to disk past its memory limit. The rule language compiles to its SQL. Postgres temp tables would load the shared database with scratch work; Spark is operationally heavy at our sizes, but `DataEngine` is the seam if we need it. Matching still runs in Java on what comes out, so the heap remains a limit.

Q14. How do you stop DuckDB using all memory when several runs execute?

`memory_limit` is a share of a machine-wide budget (default 40% of machine or container memory) divided by the number of engines open when the run starts (an `AtomicInteger`); threads are shared the same way. A run's share is fixed at start, and later runs get smaller shares. Runs beyond their limit spill to `temp_directory`. The run worker's heap check guards the Java side.

Q15. Isn't generated SQL a security risk?

It could be: SQL is built from a workspace's rules and column names. The compiler emits known constructs with dialect-quoted identifiers, and DuckDB is locked down per connection: external access disabled (no files, no `httpfs`, no extensions, no attach), then `lock_configuration` so nothing later can turn it back on. Even an injection could only touch the run's own in-memory data.

Q16. Walk me through the rule language implementation.

The lexer produces tokens (WORD, NUMBER, TEXT, NAME, SYMBOL, END); a recursive-descent parser with explicit precedence levels builds a sealed `Node` AST of records; `RuleChecker` validates fields, types and tables, suggesting the nearest name; `RuleEvaluator` compiles the AST into lambdas for per-record evaluation (cached in an LRU of 2,000); `SqlCompiler` turns the same AST into typed DuckDB SQL for whole datasets; `RuleWriter` prints the canonical English. `FunctionCatalog` defines every function once for all of them.

Q17. Why not SpEL, MVEL or Drools?

Our users are accountants. They need to read `amount is greater than 500 and narration contains 'NEFT'`, not `#amount > 500 && #narration.contains('NEFT')`. SpEL and MVEL can call arbitrary methods, a sandboxing risk for tenant-written rules. None of them compile to SQL with identical semantics. Drools is a full rule engine, heavyweight for expressions.

Q18. How do you guarantee Java and SQL evaluation agree?

One AST, one catalog, explicit semantic rules (case-insensitive trimmed text comparison, blank means missing, missing equals only missing), and `SqlParityTest`, which checks that both give the same meaning. Checks in

core-ui keep the TypeScript parser in agreement with the Java one.

Q19. Why a sealed interface for the AST?

Exhaustiveness. A `switch` with type patterns over `Node` has no default, so adding a record is a compile error wherever it is not handled, and no stage silently ignores a node type. Records also give immutability and structural equality, useful for tests.

Q20. How does the plugin system work?

Plugins are jars implementing `FunctionPlugin` or `DataConnector` from `ryfn-plugin-api`, listed in `META-INF/services`. `Plugins.load` runs `ServiceLoader` over a `URLClassLoader` built from `RYFN_PLUGINS_DIR`, parented by the service's loader. Failures per provider are caught and logged. Drop a jar, restart, and functions appear in rules, SQL and the UI.

Q21. Why ServiceLoader rather than Spring component scanning for plugins?

Plugins must be plain Java, independent of Spring and of internal classes, so the API stays small and stable. ServiceLoader is the JDK standard for this, and it works in any service with the plugin classes in their own class loader.

Q22. Are plugins sandboxed?

No: they are trusted code installed by the operator, with the service's permissions. Tenant-written logic goes through the rule language and workspace functions instead, which are declarative and cannot call arbitrary code.

Q23. Explain the outbox.

The service writes the event into `outbox_events` in the same transaction as the business change; `OutboxRelay` (every second on every instance) locks unsent rows with `SKIP LOCKED` in `seq` order, sends each to Kafka waiting for the ack, marks `sent_at`, and stops the batch on the first failure. A rolled-back change never emits an event; a Kafka outage only delays them.

Q24. How are consumers idempotent?

Each event has a UUID in its envelope. A consumer calls `ConsumedEvents.firstTime(consumerName, eventId)`: `INSERT … ON CONFLICT DO NOTHING` into `consumed_events`; if no row was inserted, the event is skipped. In `@Transactional` listeners (run counters, audit) the marker commits with the work, so a failed handler rolls back and the redelivery is processed. The anomaly listener is deliberately best-effort: its alert is a bonus, so a failure is logged and not retried.

Q25. Exactly-once?

No. At-least-once delivery plus idempotent processing gives effectively-once *effects* for database changes. Side effects outside the database, such as an email, can still repeat after a rare crash.

Q26. Why not Kafka transactions or Debezium?

Kafka transactions cover Kafka-to-Kafka exactly-once; they do not make a Postgres commit and a Kafka send atomic. Debezium (CDC on the WAL) is the scalable form of the outbox but adds Kafka Connect to operate. Polling with `SKIP LOCKED` is simple and correct at our volume; CDC is the upgrade path.

Q27. How did you handle multi-tenancy?

Shared schema with `org_id` on every tenant table via `BaseOrgEntity`; org code in every tenant path; `OrgAccessInterceptor` checks membership and role on every org route in one place; `OrgContextService` resolves the active workspace from `X-Org-Code` or an API key's own workspace; per-tenant caps, queues and rate limits for noisy neighbours; purge hooks on every service for deletion.

Q28. How would you prevent a developer forgetting `org_id` in a query?

Today the guards are the base entity, the interceptor and repository conventions. That is enforcement by code, and I would not call it sufficient at team scale. Next: Postgres row-level security with `SET LOCAL app.org_id` per transaction, or a Hibernate filter enabled per request, so a missed `WHERE` is harmless.

Q29. How do API keys work?

An admin creates a key for one workspace and role. Requests send `X-API-Key`; the filter verifies it with identity (answer cached under its SHA-256 for at most 5 s, so revocation is near-immediate) and creates a synthetic JWT authentication for the key's creator with the key's workspace and role and *no* realm roles. Keys cannot touch members, API keys or billing, and cannot be combined with a bearer token. The gateway gives keys their own bucket (300/min).

Q30. How do services authenticate to each other?

A shared internal token in `X-Internal-Token` on `/internal/**` paths, compared in constant time, and the gateway blocks `/internal/**`. When acting for a user, `InternalClient.asCaller` also forwards the user's credentials so the callee applies that user's permissions. One shared secret is the weak point; the next step would be mTLS or short-lived signed service tokens.

Q31. How do errors propagate across service calls?

`InternalClient` reads the callee's `ApiResponse` error and rethrows the matching local exception: 400 → `IllegalArgumentException`, 402 → `PlanLimitExceededException`, 403 → `AccessDeniedException`, 404 → `EntityNotFoundException`, 409 → `ConflictException`, 429 → `RateLimitExceededException`; a 5xx becomes a 502, and an unreachable service a 503 with a friendly message. The caller's `GlobalExceptionHandler` keeps working unchanged.

Q32. What happens when identity-service is down?

Nothing new can be authorised beyond cached answers (30 s), because every service asks identity about workspace roles. It is the most critical service and the system's clearest single point of failure; running several

instances is the mitigation. Others degrade more gracefully: billing down → plan checks allowed and logged; audit or Kafka down → events wait in the outbox; AI down → only AI features fail.

Q33. How does the gateway rate limiter work?

A token bucket per caller in a `ConcurrentHashMap`: API key (by SHA-256 digest) 300/min, user (JWT `sub`) 900/min, plus every IP 1,800/min, refilling evenly. Over the limit → 429 with `Retry-After`; responses carry `X-RateLimit-Limit` and `X-RateLimit-Remaining`. Idle buckets are evicted every 10,000 calls. Health checks, CORS preflights and webhooks are exempt.

Q34. What is its weakness?

It is per gateway instance, so N gateways allow N times the limit, and state is lost on restart. Fine as a protective floor; a shared limiter (Redis with a Lua token bucket) is needed with several gateways. It also skewed one of our own load tests, which reported 90% 429s until we read the error column.

Q35. How does billing stay correct with webhooks?

Verify the signature, process in one transaction, record the event id in `paddle_events` with `ON CONFLICT DO NOTHING` in that transaction so a failure rolls it back and Paddle's retry is processed, and answer 500 on any error. The plan comes from the price paid and the workspace from our own order record, never from data the browser could tamper with. Duplicate subscriptions are cancelled and refunded.

Q36. How does monday.com billing integrate?

monday's lifecycle webhooks, signed with the app secret, say which plan a monday account pays for. instant-recon-service finds the account's workspaces, and billing's `MarketplaceSubscriptions.apply` sets the plan, ends it on cancellation, or keeps an existing Paddle subscription.

Q37. What does a plan limit look like at runtime?

Services call `PlanLimits` (remote to billing). Over the limit throws `PlanLimitExceededException`, which the handler turns into HTTP 402. If billing is unreachable, the check passes and is logged: billing availability must not block work.

Q38. How is the AI integrated without vendor lock-in?

Everything depends on the small `LlmClient` interface; `AnthropicClient` and `OpenAiCompatibleClient` implement it and `AiConfig` picks one from `ai.provider`. Business services never see a vendor SDK. Switching to a local model for a privacy-sensitive customer is configuration.

Q39. Why not Spring AI?

The need was small: one completion call, attachments for PDF and image extraction, an OpenAI-compatible local server, and control over timeouts and JSON extraction. A tiny interface was simpler than adopting a

framework. Spring AI could sit *behind* `LlmClient` later without touching callers. The cost is writing retries and tool helpers ourselves.

Q40. How do you get reliable structured output from an LLM?

JSON mode where the provider supports it, a prompt with the expected shape, then `JsonExtractor` to take the first balanced `{...}` (string- and escape-aware) because models wrap JSON in prose or fences, and then deserialise and validate. A truncated answer gives a clear "raise AI_MAX_TOKENS" error rather than a parse exception.

Q41. Explain the agentic rule builder.

The model gets the rule-language guide, where the rule is used, whether it must be a condition or a value, the fields and types, source columns and lookup tables. Each turn it chooses one step: check a draft, test it on samples, look at a table, or answer. The service executes the step and reports back. An answer is accepted only if it checks cleanly, within eight steps, and the steps are shown to the user.

Q42. Is anomaly detection done by the LLM?

No. It is robust statistics: compare the latest run's match rate, variance, exceptions and volume with the median of up to 14 earlier completed runs using the median absolute deviation (threshold 4), only with at least 4 runs of history. It is cheap, deterministic and explainable. Using an LLM for arithmetic would be slower, costlier and less reliable. This is a good answer to "where would you *not* use AI?".

Q43. How do you stop the AI leaking another tenant's data?

AI reads data through `InternalClient.asCaller`, so the callee applies the user's own permissions. Cell values are not sent to the model at all unless a workspace admin turns on `shareRowValues`; by default the model sees only field names and statistics. AI proposes but never approves.

Q44. What is the MCP server?

A TypeScript Model Context Protocol server exposing RefinePoint as 22 tools to assistants like Claude Desktop: list reconciliations, get results, draft or create a reconciliation, run it, explain a row, triage exceptions. It calls the REST API with an API key or Keycloak token, so authorisation is the backend's. Replies are capped at 20,000 characters; approving rows is deliberately not a tool.

Q45. How do you scale the system?

Add instances. Services are stateless; background work is pulled from Postgres queues, so a new run-service instance starts taking runs with no routing change. Request and run capacity are configured separately (`RUN_WORKERS`). Large tenants can get dedicated queues and machines. Today everything runs on one server, so this is designed and partly load-tested, not proven at production scale. The next bottleneck is Postgres.

Q46. How do scheduled jobs avoid running twice with several instances?

job-service claims due jobs with `FOR UPDATE SKIP LOCKED` and marks them running (`running_execution_id`), so a job never runs twice at once; missed times are made up once or skipped per job; a watchdog enforces time limits and fails executions of dead instances; retries use doubling backoff. Small notification jobs in run-service use a lease row in `scheduled_locks`.

Q47. How do you trace a problem across services?

The gateway starts a trace; W3C `traceparent` propagates through `InternalClient` and Kafka; every log line has trace and span ids; the response has `X-Trace-Id`, and an error message shows it to the user. Given the id from a ticket, search all services' logs, or open the trace in Tempo or Jaeger when OTLP export is on. Sampling is 10% by default, so metrics are the first signal and traces the drill-down.

Q48. How do you deploy?

Each service is a jar run by a systemd template on an Ubuntu server, with env files under `/etc/refinepoint`. Each repository has GitHub Actions CI; I can deploy one service, a chosen set, or a whole new server by script. Services migrate their own schema on start. Dev and production run identical jars. Kubernetes is future work.

Q49. How do you handle schema changes?

Flyway per service, in the service's own schema. Release 1 is a single `V1__baseline` per service; every change since is a new `V2+` migration, never an edit. Hibernate validates the mapping at startup. For zero downtime: expand (add a nullable column, dual-write), migrate data, then contract in a later release.

Q50. What was the hardest problem?

Pick the failed heavy-rules test (34.17, story 3): CPU was only 15% busy, but the run timed out at 77 records/s with a heap `OutOfMemoryError`. Reading the numbers showed heap was the limit, which led to the run queue with heap-aware admission. If they want a pure bug, use the run that could start before its row was committed.

Q51. What would you change if starting again?

Per-service DB users and RLS from day one, a distributed rate limiter, consumer-driven contract tests, earlier heavy load tests, and fewer services initially, splitting only when load or team size requires it.

Q52. What are the consistency guarantees between services?

Within a service, ACID. Across services, eventual consistency through the outbox, plus compensating steps for multi-service operations, and purge requests recorded and retried until every service confirms. There are no distributed transactions.

Q53. How do you test all of this as one developer?

Unit tests on the engine and rule language (including SQL parity); service tests on a real embedded PostgreSQL (`TestDatabase`); about 100 end-to-end checks through the gateway on a local stack; `tests/regression.py` running everything into one report with a performance baseline; load tests for request capacity and parallel runs; and a read-only smoke test on the deployed server.

Q54. Why keep `JdbcTemplate` alongside JPA?

JPA for aggregates with behaviour (runs, subscriptions); `JdbcTemplate` for paths where the SQL itself is the design: queue claims with `SKIP LOCKED` , outbox inserts, `ON CONFLICT` upserts, advisory locks and `UPDATE … RETURNING` . JPA would hide or prevent those.

Q55. How do you protect against a big tenant starving others?

Four layers: gateway rate limits per caller; per-workspace run caps with fairness ordering; dedicated queues and instances; and per-run DuckDB memory shares plus heap-aware admission.

Q56. How much load has this handled?

Be exact: it is pre-launch, so these are load tests, not production. On a 12 GB dev server, about 54 requests/s at 20 concurrent users with p95 near 1 s including a 235 ms round trip, saturating by 40 users, with run-service reads as the bottleneck. Locally, 20,000 rows a side took 17.8 s, and four runs at once took 42.5 s with exact counts. A much heavier rule set at 50,000 rows a side failed on the dev server. Then say what you would do next: tune the records query and add run-service instances.

Q57. How many customers do you have?

None in production yet: launch is planned for February 2027, and the Paddle checkout is live. Say it plainly and move to what the project demonstrates: architecture, failure design and measured behaviour. Hedging here damages everything else you have said.

35. Senior, Lead & Architect Questions

At 12 years of experience, the technical round only gets you to this round. Plenty of candidates can explain `HashMap`. Far fewer can explain how they got two teams to agree on an API, why they said no to a VP, or what they changed after an outage they caused. This chapter covers how those rounds are scored, how to structure your answers, and model answers for the questions you will almost certainly get.

> **Trap:** every model answer in this chapter that mentions "a previous company" is a *template*. Square brackets mark what you must replace with your own facts: `[team size]`, `[latency number]`. Never tell a story that did not happen to you. Behavioural interviewers are trained to probe three levels down ("Who disagreed? What exactly did they say? What would you do differently?"), and an invented story collapses by the second follow-up. The stories marked **RefinePoint** are real and can be told as they are.

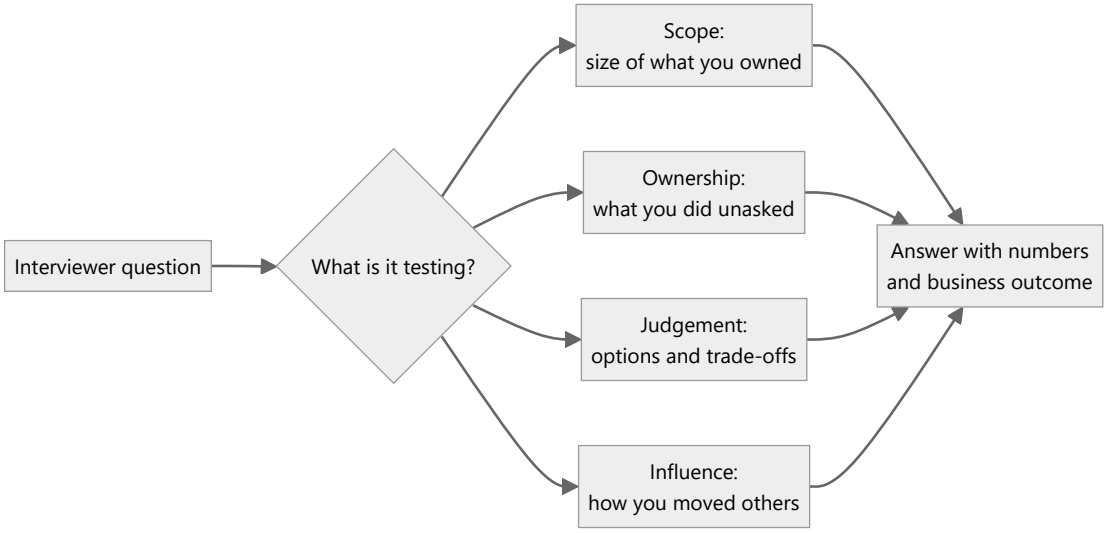
35.1 What interviewers assess at 12 years

The bar moves from "can you build it?" to "can you be trusted with something ambiguous and important without supervision?" Four things get scored:

Dimension	Question behind the question	Strong signal	Weak signal
Scope	How big a problem can you own?	Cross-team, multi-quarter, business outcome	"My module", "my ticket"
Ownership	What do you do when it isn't your job but it's broken?	Stayed on it through production, fixed the process	"That team was responsible"
Judgement	Do you make good calls with incomplete information?	Named alternatives, costs, and what would change your mind	One option, no downside, "best practice"
Influence	Can you move people you don't manage?	Data, prototypes, listening, compromise	"I escalated" or "I told them"

Levels differ mainly in where the leverage comes from:

Level	Typical scope	Main leverage	Failure mode they screen for
Senior engineer	A service or feature area	Own code and design	Great coder, no view beyond the ticket
Tech lead	A team's delivery	Decomposition, unblocking, reviews	Hero who does all the hard work alone
Staff / principal	Several teams	Technical direction, standards	Abstract advice, no contact with code
Architect	A domain or the organisation	Trade-offs, roadmaps, governance	Ivory tower diagrams nobody follows
Engineering manager	People and outcomes	Hiring, growth, prioritisation	Avoiding hard conversations

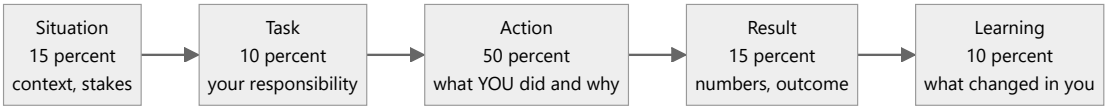


Say "I", then "we". Interviewers need to know what *you* did. "We migrated the platform" gives them nothing to score. "I proposed the sequence, wrote the first two services as the template, and the team did the other nine" does. Credit people by role, and be exact about your own part.

Show the technical substance. A leadership answer with no technical content sounds like a project manager. Name the actual choice: "The disagreement was about synchronous calls versus an outbox."

35.2 The STAR method and templates

STAR means **Situation, Task, Action, Result**. For senior roles, add **L, Learning**.



Most candidates spend two minutes on the Situation and rush the Action. Interviewers score the Action. Aim for 2–3 minutes in total, then stop and let them ask.

Generic template:

```
S: At [company/project], [system] served [scale], and [problem] was costing [impact].
T: I was [role] and responsible for [specific thing]. The hard part was [constraint].
A: First I [diagnosed/measured]. I considered [option A] and [option B].
   I chose [B] because [reason], accepting [cost]. I got [person] on board by [how].
   Then I [executed], and when [setback] happened, I [response].
R: [Metric] went from [x] to [y]. [Business effect].
L: Now I always [changed behaviour].
```

Conflict template: state the disagreement fairly (make the other side sound reasonable), say how you found the real concern behind it, what evidence settled it, the outcome, and what happened to the relationship.

Failure template: a real failure that was yours, not a disguised strength. Own it in the first sentence, explain the impact, how you fixed it, and the process change that prevents a repeat.

> **Interview tip:** a good answer has a moment of tension: something went wrong or someone disagreed. If your story has none, the interviewer learns nothing about your judgement.

Build your story bank before the interview

Prepare 8–10 real stories. Each can answer several questions. Fill in the right-hand column yourself.

Story type	Answers questions about	RefinePoint story you can use	Your employer story
Large migration	Ownership, planning, risk	Monolith to 13 services, API unchanged	`[fill in]`
Performance win	Technical depth, measurement	Heavy-rules test → heap-aware run queue	`[fill in]`
Production incident	Pressure, postmortems	Not a production story (pre-launch)	`[fill in: essential]`
Technical disagreement	Influence	Not applicable (solo)	`[fill in: essential]`
Failure	Humility, learning	Load testing too late; too many services	`[fill in]`
Mentoring	Leadership	Not applicable (solo)	`[fill in: essential]`
Ambiguity	Self-direction	Whole product: choosing what to build	`[fill in]`
Saying no / scope	Judgement	Cut Kubernetes, per-service DB users until later	`[fill in]`
Cross-team dependency	Influence, delivery	Paddle and monday.com integrations (external parties)	`[fill in]`
Hiring or team building	Leadership	Not applicable (solo)	`[fill in if true]`

> **Trap:** RefinePoint is solo-built and pre-launch, so it cannot answer questions about conflict, mentoring, on-call or hiring. Those need stories from your employed years. An interview built only on the solo project will look like you have avoided working with people.

35.3 Technical leadership practices

Design reviews

A design review exists to find problems while they are cheap, not to approve a document. To run one well:

1. Circulate a short written design 2–3 days ahead, so people arrive with comments.
2. Open with the problem and constraints, not the solution.
3. Show at least two alternatives and why they were rejected.
4. Focus on failure modes, data ownership, migrations, operability and cost.
5. End with decisions, owners and open questions recorded in writing.

As a reviewer: ask "what happens when this is down?", "how do we roll back?" and "what is the data migration?". Separate blocking concerns from preferences, and say which is which.

ADRs: Architecture Decision Records

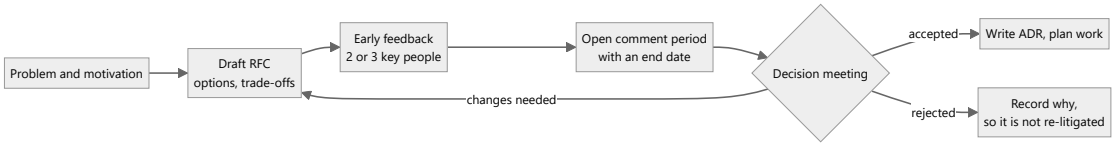
An ADR is a short, permanent note explaining one decision, so that people two years later know *why*. It is one page, stored in the repository next to the code. When a decision changes, write a new ADR that supersedes the old one; do not edit history.

```
# ADR-012: Work queues in PostgreSQL, not a message broker
Status: Accepted (2026-10-27)
Context: Runs must be claimed by any number of workers. Claiming must be atomic with
  the run's status. Per-workspace caps need a count over other rows.
Decision: Rows in recon_runs, claimed with FOR UPDATE SKIP LOCKED under an advisory lock.
Alternatives: Kafka (no per-message ack, priorities or caps); RabbitMQ (new infrastructure,
  second source of truth); Redis (same, and durability concerns).
Consequences: No new infrastructure. Polling each second. Claims are serialised, so this
  must be revisited at thousands of claims per second.
```

(This decision is real; the ADR format is shown as an example of how to record it.)

RFCs: Request for Comments

An RFC is a longer proposal that seeks input *before* a decision, usually when many teams are affected. The ADR records the outcome.



Two habits make this work: talk to the people most likely to object *before* the document goes wide, and set an end date, because RFCs with no deadline never finish.

Estimation

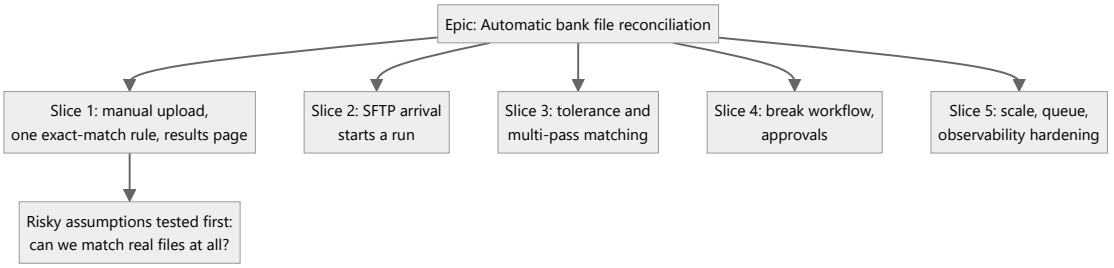
Estimates are forecasts with uncertainty, not promises. Practical techniques:

- **Decompose** until tasks are 1–3 days. Uncertainty hides in big items.
- **Three-point estimate:** optimistic (O), likely (M), pessimistic (P); expected = (O + 4M + P) / 6. A wide range tells you to run a spike.
- **Reference class:** "the last integration of this kind took six weeks" beats reasoning from first principles.
- **Spikes:** time-box the unknown, then estimate again.
- **Give ranges with confidence:** "6–9 weeks, 80% confident; the main risk is the bank's API".
- **Include the invisible work:** tests, reviews, migration, observability, rollout, documentation, holidays, on-call.

Pressure to reduce an estimate is a negotiation about **scope**, not about arithmetic: "We can hit that date with X and Y. Z would move to the next release. Which do you prefer?"

Breaking down an epic

Slice **vertically** (each piece delivers working behaviour end to end), not horizontally ("database layer", then "API layer"). Horizontal slicing delays all learning and all risk discovery until the end.



Order slices by **risk and learning first**, then value. A good first slice is thin but real, and it proves the riskiest assumption.

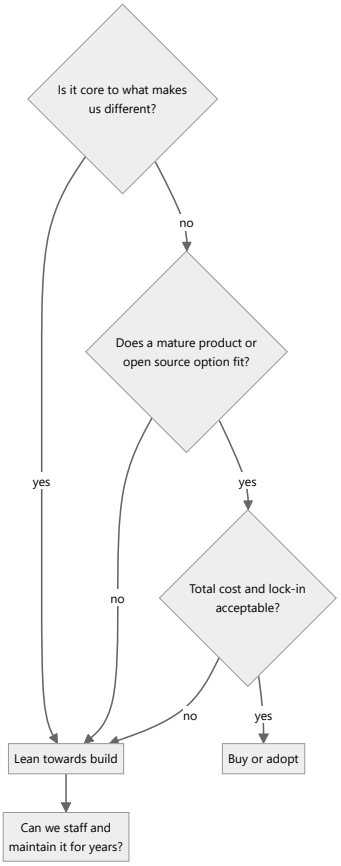
Risk management

Keep a short risk list, reviewed weekly. For each: likelihood, impact, mitigation, an owner and a trigger that tells you it is happening.

Risk	Likelihood / impact	Mitigation	Trigger
External API not ready	High / High	Stub contract, parallel design	No sandbox by week 3
Data migration slower than planned	Medium / High	Rehearse on a production-sized copy	Rehearsal over the window
Key person leaves	Low / High	Pairing, docs, shared ownership	Single reviewer on a component
Performance unknown	Medium / Medium	Early load test on the riskiest flow	p95 over budget

The most useful sentence in a status report is "this risk is becoming real, and here is what we need". Raise it early, with options.

Build versus buy

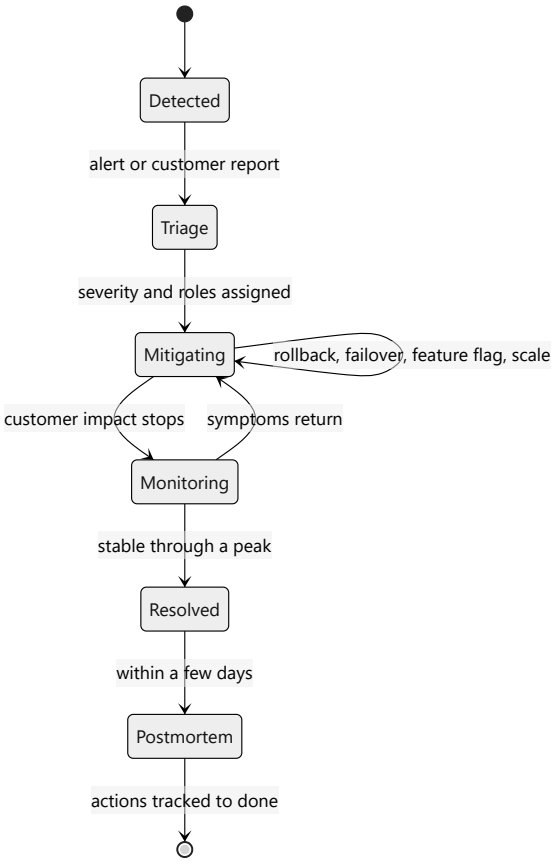


Count total cost over 3 years, not licence versus salary: integration, upgrades, on-call, compliance, exit cost. Ask what the exit looks like before you enter.

> **RefinePoint:** built: the rule language and match engine (these are the product, and no library gives both plain-English rules and identical SQL semantics). Bought or adopted: Keycloak for identity, Paddle for payments and tax, DuckDB for analytics, a managed Kafka. Authentication and tax handling are not where a reconciliation product wins.

35.4 Production incidents and postmortems

The order during an incident is **mitigate first, understand later**. Roll back or fail over before finding the root cause.



Roles in a serious incident: an **incident commander** coordinates and does not debug, a **tech lead** investigates, and a **communicator** updates stakeholders on a fixed schedule ("next update in 30 minutes", even if there is no news). One person doing all three is the most common failure.

First questions: What changed? (Deploy, config, traffic, a dependency.) Who is affected: everyone or one tenant? When did it start, according to the metrics? Is rollback safe?

Blameless postmortem. Blameless does not mean nobody is accountable. It means asking why the system allowed a reasonable person to make that mistake, because people hide information when they are blamed.

```
Title, date, severity, duration, customers affected
Summary: two sentences a non-engineer understands
Timeline: UTC, facts only (detected, escalated, mitigated, resolved)
Root cause and contributing factors: 5 whys, stop at a process or system
What went well / what went badly / where we got lucky
Action items: each with an owner and a date; prevent, detect, mitigate
```

"Where we got lucky" is the section people skip, and it is often the most useful. "Be more careful" is not an action item. "Add a pre-deploy check that fails when a migration lacks an index on a large table" is.

Metrics to know: MTTD (time to detect), MTTR (time to restore), SLI (the measure, such as successful requests divided by total), SLO (the target, such as 99.9%), error budget (the allowed failure: 99.9% is about 43 minutes a month).

> **RefinePoint:** we designed for incidents before having any: an architecture document with a "when something is down" table for all 13 services and Kafka, an `X-Trace-Id` on every response that a customer can quote, and an alert on every run failure. There has been no production incident to describe because we have not launched, so do not invent one. Use an employer incident in this round.

35.5 Performance improvement stories

A performance story needs a method, not just a result:

1. **Define the goal in numbers.** "p95 under 300 ms at 200 requests/s", not "faster".
2. **Measure before changing anything:** metrics, async-profiler or JFR, `EXPLAIN ANALYZE`, GC logs.
3. **Find the actual bottleneck.** Usually one thing: a missing index, an N+1 query, lock contention, an undersized pool, GC.
4. **Change one thing at a time** and measure again under the same load.
5. **Protect it:** add a regression check and an alert.

The most common causes in Spring and JPA systems, in rough order: N+1 queries; missing or unused indexes; no pagination; connection pool waits; synchronous remote calls inside a transaction; huge payloads; lock contention; GC pressure from allocation.

Template:

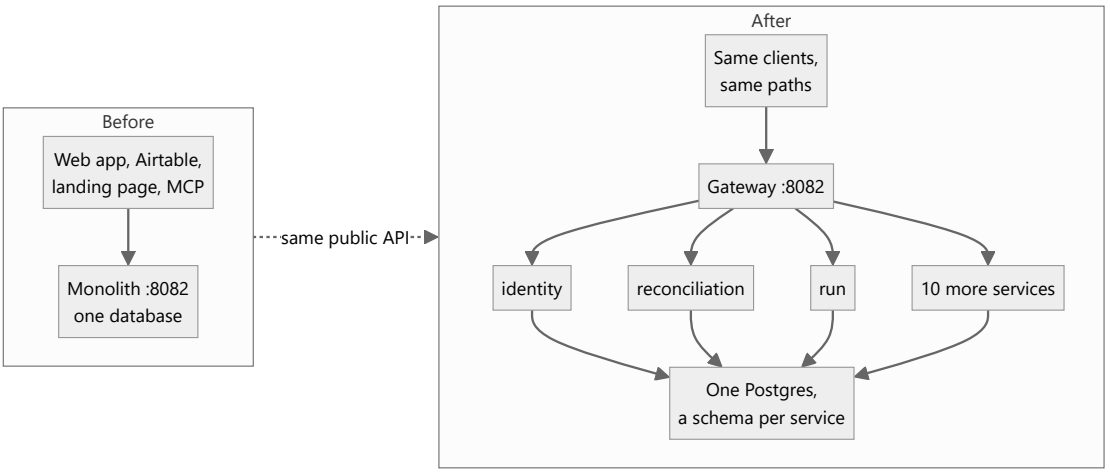
> "Our `[endpoint]` p95 was `[x]` under `[load]`, which broke `[business consequence]`. I profiled with `[tool]` and found `[cause]`, not the `[assumed cause]` the team suspected. I changed `[fix]`, considered `[alternative]` but rejected it because `[reason]`. p95 fell to `[y]` and `[cost or capacity effect]`. I added `[regression guard]`."

> **RefinePoint:** a real story with real numbers. A load test on the dev server ran 50,000 rows a side with 50 check rules per record. It timed out after about 21 minutes at 77 records/s, with `OutOfMemoryError: Java heap space` in run-service while CPU averaged only 15%. The instinct is "the rules are slow"; the measurements said heap was the limit. I replaced Spring's shared thread pool with a run queue, dedicated worker slots and admission only while 25% of the heap is free, so a run that needs too much fails cleanly with a reason. A separate test, 20,000 rows a side, runs in 17.8 s, and four at once in 42.5 s with exact counts. Those are different workloads, so do not present them as a before and after. Re-run the heavy test for a true comparison.

35.6 Migration stories

Monolith to microservices (RefinePoint, real)

RefinePoint began as a single Spring Boot application. It was split into 13 services and a gateway, and the monolith was deleted.



What makes this a strong story is the specifics:

- **Boundaries by data ownership.** Who writes this table? Runs, breaks and journals stayed together because they change in one transaction.
- **The public contract was frozen.** Same paths, port and responses, so no client changed. The gateway took the monolith's port.
- **Every cross-module database feature was replaced on purpose:**

In one database	After the split
Deleting a reconciliation cascaded to its runs	Reconciliation asks run to delete first; on failure it keeps the reconciliation
Run numbers from a counter on the process	From the runs themselves, under a per-process advisory lock
"Total runs" updated by the run	Reconciliation updates them from `rp.run.completed`
Moving a process between workspaces in one transaction	Compensating steps, moved back on failure
Deleting a workspace deleted its rows	Identity asks every service, recorded and retried until each confirms

- **An end-to-end suite through the gateway** (now about 100 checks) verified that behaviour held.
- **Three defects surfaced** that had existed in the monolith: a thousands separator failing a run, an Airtable first sign-in crash, and a run starting before commit.

Be honest about the limits of this story. There was no production traffic, so this was not a live cutover. It did not need a long strangler migration, dual writes or a traffic ramp. If asked how you would do it with live traffic, answer from the textbook and say that you are doing so: strangler routing at the gateway, one bounded context at a time, the data moved via dual-write or CDC with a reconciliation check, a feature flag per route, and the old path kept until the new one has soaked.

Java 8 to 21

RefinePoint started on modern Java, so this is general knowledge. If you have done this migration at a job, use your own story. Go in steps (8 → 11 → 17 → 21) and take each to production.

Step	Main problems	What to do
8 → 11	Java EE modules removed (JAXB, JAX-WS, `javax.annotation`); module system warnings; old bytecode libraries; `-XX:+PrintGC` flags replaced by `-Xlog`	Add explicit dependencies; upgrade Lombok, Mockito, ByteBuddy, ASM, Spring; run `jdeps` and `jdeprscan`
11 → 17	Strong encapsulation of JDK internals by default (JDK 16–17); CMS removed (14); Nashorn removed (15)	Fix libraries first; use `--add-opens` only as a temporary measure; move to G1 or ZGC
17 → 21	Fewer breaks; virtual threads available; generational ZGC; deprecations such as finalization	Adopt records, sealed types and pattern matching gradually; test virtual threads for pinning

Risks that often catch people: default locale data changed to CLDR in Java 9, altering date and number formats; default charset became UTF-8 in Java 18; heap and GC behaviour differ, so re-baseline and load test; and container-aware memory defaults change how much heap you actually get.

Spring Boot 2 to 3

Same advice: if this happened at a job, tell your own version. RefinePoint is on Boot 3.4 throughout.

1. Upgrade to the latest **Boot 2.7** first and clear its deprecations.
2. Move to **Java 17 or later** (the Boot 3 baseline).
3. Add `spring-boot-properties-migrator` temporarily to report renamed properties, such as `spring.redis.*` → `spring.data.redis.*`.
4. Run OpenRewrite for the mechanical `javax.*` → `jakarta.*` rename. Every library that uses those packages must also have a Jakarta version; that is the main blocker.
5. **Spring Security 6:** `WebSecurityConfigurerAdapter` is gone, `authorizeRequests` becomes `authorizeHttpRequests`, and method security is via `@EnableMethodSecurity`.
6. **Hibernate 6:** changes to HQL and ID generation, and `@Type` changes. Test your queries.
7. **Spring Framework 6:** trailing-slash matching is off by default, so `/api/x/` no longer matches `/api/x`.
8. **Sleuth** → **Micrometer Tracing**; auto-configurations must be listed in `AutoConfiguration.imports`, not `spring.factories`.
9. Roll out service by service, starting with the least critical, with shared libraries handled first.

35.7 Agile and Scrum in practice

Ceremony	Purpose	What a senior person does
Sprint planning	Commit to a goal	Challenges vague stories, flags dependencies
Daily stand-up	Find blockers	Keeps it short; takes technical debates offline
Backlog refinement	Make work ready	Slices stories, surfaces risk, gets acceptance criteria
Review / demo	Show working software	Gets real feedback, not status
Retrospective	Improve the process	Brings data; ensures one action actually gets done

Definition of Ready and **Definition of Done** prevent most arguments: Done should include tests, review, documentation, observability and deployment. **Velocity** is a planning tool for one team, not a performance

measure and not comparable between teams. Once it is used as a target, it gets inflated.

Kanban suits flow work (support, platform teams): limit work in progress and measure cycle time. A useful senior opinion is that the process should serve delivery: "We dropped estimation poker because stories had become small enough that counting them predicted delivery just as well." Only say this if it is true of your team.

35.8 Stakeholder communication

Adapt to your listener. Executives want impact, risk, decision and date; product wants options and trade-offs; engineers want the mechanism.

Instead of	Say
"The DB connection pool is exhausted because of N+1 queries"	"Checkout is failing for about 8% of users. We have a fix in test, live within the hour. Next update at 3 pm"
"That's impossible"	"We can do it by that date without X, or with X by this date. Which matters more?"
"We need to refactor"	"Each change here costs about a week and caused 3 of last quarter's incidents. Two weeks now removes that"
"It's nearly done"	"Two of five parts are live. The risk is the bank's API, and I'll know more on Thursday"

Bring bad news early, with options and a recommendation. Nobody forgives a surprise on the deadline; most people forgive a problem raised three weeks before it.

Tech debt gets funded when it is translated into business terms: delivery speed, incident rate, cost, risk. Three approaches work: a standing 15–20% allocation, folding the cleanup into features that touch that code, or a specific case with a measured payoff. "The code is ugly" funds nothing.

35.9 The solo-founder years, framed for employers

Employers worry about three things with a founder: will you take direction, can you work in a team, and will you leave as soon as your startup takes off? Address these before they ask.

The strengths are real: you owned every decision, from product to production; you have run things on a budget; you understand customers, pricing and billing; you know the cost of complexity because you paid it yourself. The evidence is concrete: architecture document, runbook, ~100 end-to-end checks, load tests, live payments.

Be equally honest about the limits: no team, no code review from others, no on-call under real traffic, not yet launched. Saying this first makes everything else more credible.

Worry	How to answer
"You'd be bored taking direction"	"I've seen what happens when nobody challenges your design; I want that challenge"
"Is this a real product?"	"Live Paddle checkout, a monday.com marketplace app, launching February 2027. Here's the architecture. No customers yet"
"You'll leave when it takes off"	Your true plan, stated plainly: is it sold, paused, a side project? Never leave this vague
"Can you work in a team?"	Use stories from your employed years, not the founder period

> **Trap:** before you interview, decide what happens to RefinePoint if you take a job, and read the employment contract's clauses on outside work and IP assignment. Some contracts would claim what you build while employed. Raise it openly once there is an offer; a conflict discovered later is far worse. Take legal advice if the contract is unclear.

35.10 Questions to ask the interviewer

You are also assessing them, and your questions reveal your level. Ask two or three, chosen for who you are talking to.

To an engineer or tech lead:

- What is the most painful part of the codebase, and what is being done about it?
- How does a change get to production, and how long does that take?
- Tell me about your last serious incident. What changed afterwards?
- Who decides on architecture here, and how are disagreements settled?

To a manager or director:

- What would make this hire a clear success after a year?
- What problem do you need this person to solve first?
- How is technical debt balanced against features? What was the last thing you delayed for quality?
- How do people here grow to the next level?

To an architect or CTO:

- Which technical bet are you least sure about?
- What will break first if the business grows tenfold?
- Where does engineering disagree with the business right now?

Avoid: anything answered on the website; pay and leave in the first technical round; and "no, I have no questions".

35.11 Interview questions and answers

Each answer begins with the shape of a strong response. Replace every `[bracket]` with your own facts. Answers marked **RefinePoint** are true as written.

Q1. Tell me about yourself.

A 90-second arc, not a biography: "I'm a backend engineer with 12 years in Java and Spring, mostly `[domain]`. Most recently I `[biggest scope]`. For the last `[period]` I've built RefinePoint, a multi-tenant financial reconciliation platform with 13 Spring Boot services, on my own, which gave me full ownership from architecture to billing. I'm looking for `[what you want]`, and this role interests me because `[specific reason]`." Finish on why you are in this room.

Q2. Tell me about a conflict with a colleague.

Pick a real technical disagreement, and describe the other person's view fairly. "A senior colleague wanted `[approach A]`; I preferred `[approach B]`. We had gone back and forth in review comments, so I suggested we talk. His real concern was `[concern]`, which I had not understood. We agreed on `[small test or data]`, which showed `[finding]`. We went with `[outcome]`, partly his idea. We still work well together." The test is whether you find the concern behind the position. Avoid stories where the other person was simply wrong.

Q3. Tell me about a time you disagreed with your architect.

Show respect plus a spine: disagree with evidence, then commit. "The architect mandated `[decision]` for all services. I thought it would cost us `[impact]` because `[reason]`. I wrote a one-page comparison, then asked for 30 minutes rather than arguing in a meeting. They showed me a constraint I hadn't known, `[constraint]`, and I showed them `[data]`. We agreed `[compromise]`." If you lost: "I committed fully and we delivered it. Six months later `[what happened]`." Disagree and commit is the phrase.

Q4. Tell me about a time you disagreed with your manager.

Disagree privately, with facts and a recommendation; accept the decision once it's made, unless it is unethical or unsafe. "My manager wanted to commit to `[date]`. My estimate said `[later date]`. I brought the breakdown and three options with their risks. They chose `[option]` for `[business reason I hadn't known]`. I asked that we flag the risk to `[stakeholder]`, and we did."

Q5. What is your biggest failure?

Name a real failure with real cost, and own it in the first sentence. "I `[decision]`, and it caused `[impact]`. I'd skipped `[step]` because `[honest reason]`. I `[recovery]`, told `[people]` straight away, and then `[process change]`. It hasn't recurred." **RefinePoint** option, for a design failure rather than an outage: "I built 13 services alone. In hindsight, lookup, report and audit could have stayed as modules until load or team size justified splitting them, because I now carry the deployment surface for all of them. I also ran the first heavy load test after the architecture was in place, when it should have shaped the design." Pair this with a people or delivery failure from a job.

Q6. Describe a production outage you handled.

Use an employer story; RefinePoint has not launched. "At `[time]`, `[alert]` fired: `[symptom]` affecting `[scope]`. I took incident lead. We checked what had changed and found `[deploy]`. Rather than debug, we

rolled back and impact stopped within `[minutes]`. Root cause: `[cause]`. In the postmortem we added `[detection]` and `[prevention]`." Show the order: mitigate, communicate, understand, prevent.

Q7. Tell me about a time you caused a production issue.

Everyone senior has one, so "never" is a red flag. Say what you did, how quickly you raised it yourself, how you fixed it, and what you changed so that nobody could repeat it, such as a safer default or an automated check. Do not blame the reviewer.

Q8. Tell me about a missed deadline.

Focus on when you noticed and whom you told. "We missed `[date]` by `[time]`. I had underestimated `[item]`. By `[early point]` it was clear, so I told `[stakeholder]` and offered: reduce scope to X, or keep it and move two weeks. They chose `[option]`. Since then I time-box unknowns with a spike before committing."

Q9. How do you push back on scope?

Turn "no" into a choice. "I never say we can't. I say what it costs: 'We can add that. It adds about three weeks and pushes the launch, or we swap it for Y. What matters more?' Then the decision belongs to the person who owns the business priority, with the facts in front of them." **RefinePoint** example: Kubernetes manifests and per-service database users are recorded in the architecture document as "not done yet" rather than half-built, so the choice is visible and not forgotten.

Q10. How do you handle technical debt?

Make it visible, tie it to business cost, and pay it down in small steps. "I keep a debt register with the cost of each item: incidents, delay, risk. Small items go in with the feature that touches the code; large ones get a business case. For example, `[system]` caused `[x]` incidents a quarter, so I proposed `[work]`, and incidents fell to `[y]`." Also say that not all debt should be paid: code that rarely changes can stay ugly.

Q11. How have you mentored someone?

Specifics, with the person's growth as the result. "`[Junior or mid-level engineer]` was strong at coding but avoided design. I had them write the design for `[feature]`, reviewed it with questions rather than answers, paired on the hard part, and let them present it. Within `[period]` they led `[bigger thing]`." Mention how you changed your style for different people. Teaching by giving answers does not scale.

Q12. How do you give feedback to someone underperforming?

Early, privately, specific, and based on behaviour. "I describe what I saw and its effect, ask what's going on, because there's often a reason I don't know, agree on concrete expectations and support, and check in weekly. If there's no change, I involve the manager early rather than quietly doing their work." Doing their work for them is the common and wrong choice.

Q13. What does good code review culture look like?

Reviews are for shared understanding and catching defects, not for showing off. Small pull requests, under a few hundred lines; a response within a business day; automate style so nobody argues about it; authors explain why; reviewers separate blocking issues from suggestions ("nit:"); seniors' code gets reviewed too. A good sign is junior engineers commenting on senior code without fear.

Q14. How do you handle a pull request you strongly disagree with?

Find out why first. I ask questions in the comments rather than ruling. If the design is really wrong, I talk directly, as a long comment thread is a poor place for it, and ideally I catch this earlier through a design discussion. I distinguish "this will cause an incident" (blocking) from "I'd have done it differently" (approve and mention it).

Q15. How do you work with a dependency on another team?

Agree on the interface early, reduce the coupling, and create visibility. "We depended on `[team]`'s API, and they had other priorities. I met their lead to understand their constraints, we agreed the contract in a shared document, and I built against a stub so we weren't blocked. I made the dependency visible on both roadmaps and checked in weekly. When they slipped, the stub let us `[fallback]`." Escalate only with options, after trying directly, and tell them first.

Q16. Tell me about delivering under ambiguity.

"The brief was `[vague goal]`. No requirements and no owner. I spent the first week talking to `[users]`, wrote down what I believed the problem was along with my assumptions, and got `[sponsor]` to correct it. I built the thinnest version to test the riskiest assumption, which showed `[learning]`, then planned from there." **RefinePoint** fits this question well, since there was no brief and every product decision was mine, but add your own true detail of what you chose to build first and why.

Q17. How do you make a decision with incomplete information?

First ask how reversible it is. Two-way doors get decided quickly, by the person closest to the work; one-way doors (data model, public API, vendor lock-in) deserve more research. I state my assumptions, decide what evidence would change my mind, and record the decision. Waiting for certainty is itself a decision.

Q18. Tell me about a decision you got wrong.

"I chose `[option]` because `[reason]`, and `[what happened]` proved it wrong. I noticed when `[signal]`, and reversed it at a cost of `[cost]`. I now `[changed practice]`." **RefinePoint** example: I ran the first heavy load test after the run design existed, so the heap limit surfaced late, and how runs are taken up had to be redesigned.

Q19. How do you influence people without authority?

Understand what they care about, then show evidence and make it easy to agree. "I wanted teams to adopt `[standard]`. Mandating it wouldn't work. I built it for my own team first, measured `[benefit]`, helped one

other team migrate, and then let them speak for it. Adoption followed." Prototypes beat slides, and allies beat escalation.

Q20. When would you escalate?

When the decision is above the people in the room, and the cost of delay is real, and only after trying directly. I tell the other person first: "We don't agree, so let's take this to X together." I present both views fairly. Escalating is not failure; escalating as a surprise is.

Q21. Have you ever said no to a senior leader?

"A `[VP]` asked for `[request]` by `[date]`. I said what it would cost, not just no: `[risk or tradeoff]`, offered `[alternative]`, and explained what we'd give up. They chose `[outcome]`." Tone matters: you're helping them decide, not defending your turf.

Q22. What do you do when requirements keep changing?

First find out why: a real shift in the market, or an unclear decision-maker. Then shorten the loop: smaller slices, earlier demos, one named owner. Make the cost visible, and design the likely-to-change parts behind seams. Changing requirements are normal; unowned ones are the problem.

Q23. How do you prioritise when everything is urgent?

Ask what happens if each thing slips by a week, in terms of revenue, risk and customer commitments. Force-rank with the stakeholders in one room, because the conflict is usually between people rather than tasks. Production and security issues come first; then commitments; then highest value. Then say explicitly what is *not* happening.

Q24. How do you onboard yourself to an unfamiliar codebase?

Trace one real request end to end; run it locally; read the ADRs and postmortems; find who knows what. Ship a small fix in the first week to learn the delivery path. Hold back large opinions for a month, because some of what looks wrong has a reason.

Q25. How do you handle a team member who resists change?

Assume they have a reason, since they're often seeing a real risk. Ask what would go wrong. Often they're right about something, so fix that. Run a small pilot rather than a big switch, and let them help design it. If the decision stands after listening, be clear that it stands.

Q26. How have you improved engineering practices on a team?

Pick one measurable improvement: "Deploys took `[x]` and were feared. I automated `[steps]`, added `[tests or canary]`, and we went from `[monthly]` to `[weekly]`, with fewer rollbacks." **RefinePoint**: one command runs every library and service test, the web builds, about 100 end-to-end checks, a performance baseline and a server smoke test, into one report. It catches regressions that a solo developer has nobody else to catch.

Q27. What do you look for when hiring?

Evidence over credentials: how someone reasons through a problem they haven't seen, whether they name trade-offs, and how they respond to a hint or a challenge. Can they explain something they built, including what was wrong with it? I use a practical exercise close to the real job, a structured scorecard to reduce bias, and write my assessment before talking to the other interviewers.

Q28. You've interviewed a brilliant engineer who's rude. Hire?

No. A strong individual who drives others away reduces the team's total output, and they're hard to remove. I would check whether it's a pattern, or nerves on one occasion, but I'd not trade collaboration for raw skill.

Q29. How do you handle low morale after a failed project?

Acknowledge it honestly, without spin. Run a real retrospective that actually changes something, and give people credit for what they did well. Give the team a clear, winnable next goal. Protect them from blame coming from above.

Q30. A deadline is impossible. What do you do?

Show why with a breakdown, not an opinion. Find out what's behind the date: a contract, an event, or someone's hope. Offer options: reduced scope, phased delivery, more people only where that truly helps. Get a decision in writing. Never agree silently and hope; that's the only truly unprofessional choice.

Q31. How do you balance hands-on coding with leadership?

Stay close enough to have credible judgement: take smaller or non-blocking tasks rather than the critical path, since your time is interrupted; prototype risky ideas; review a lot; fix something in production occasionally. A lead who writes all the hard code creates a bottleneck, and one who writes none loses judgement within about two years.

Q32. Would you rather be a manager or stay technical?

Answer truthfully, with reasons. "I get the most energy from `[technical direction, hard problems]`, but I've found I also enjoy `[developing people]`. I want a role where `[…]`." A vague "either" reads as lack of self-knowledge.

Q33. How do you estimate a project you've never done before?

Decompose as far as possible, find a comparable project, time-box a spike on the biggest unknown, then give a range with confidence and say what would narrow it. Update as you learn. A single number at the start is a guess, so I don't present it as a commitment.

Q34. How do you run a design review?

Written design beforehand; start with the problem and constraints; show rejected alternatives; focus on failure modes, data, migration, operations and cost; separate blockers from preferences; end with decisions and

owners recorded in an ADR. The aim is to find problems cheaply, and a review that finds nothing was probably held too late.

Q35. How do you decide between a monolith and microservices?

Start from team size, deployment needs and domain clarity, not fashion. A small team with an unclear domain does best with a modular monolith, with clean module boundaries that can be split later. Split when teams block each other, when workloads need different scaling, or for failure isolation. **RefinePoint** is my evidence in both directions: run processing truly needs separate scaling and isolation, but I'd keep the lighter services as modules if I started again.

Q36. Tell me about a time you improved performance.

Use section 35.5: goal in numbers, measurement, real bottleneck, rejected alternative, verified result, regression guard. **RefinePoint**: CPU at 15% with a heap `OutOfMemoryError` showed that heap, not rule speed, was the limit, and that led to a run queue with heap-aware admission. Be clear that it is a pre-launch load test.

Q37. Tell me about a migration you led.

RefinePoint: monolith to 13 services with a frozen public API, explicit replacement of each thing the single database did, end-to-end checks through the gateway, three old defects found, monolith deleted. State the limit: no live traffic to cut over. Use an employer migration if you have one with real traffic, because that's a stronger proof of risk management.

Q38. What's the most complex system you've designed?

Give scope, the hard constraint and a decision: "RefinePoint's run engine. The hard part is correct matching over large files with tenant-written rules, on shared infrastructure." Then offer depth: "I can go into the queue, the DuckDB processing, or the rule compiler. Which would you like?" Letting them choose shows you can navigate your own design.

Q39. How do you stay current?

Be specific and recent, and show that you apply what you learn. "I read JEP summaries and release notes, and I tried virtual threads on `[workload]` before forming an opinion. Recently I built LLM features with a provider-agnostic client and an MCP server, because I wanted first-hand knowledge rather than opinions." Name one thing you evaluated and rejected.

Q40. Why should we hire you?

Three true points tied to what they told you: "You need `[problem from the interview]`. I've done `[matching experience]` at `[scale]`. I also bring `[differentiator: end-to-end ownership]`." Refer to things said earlier in the interview; it shows that you listened.

Q41. What's your greatest strength? Your greatest weakness?

Strength: one claim and a proof. "I find the real constraint fast; for example `[story]`." Weakness: a real one with what you're doing about it, and not "I work too hard". For example: "Working solo for years has left me out of practice at code review and team process. I'm deliberately asking for that environment." Or: "I tend to take hard problems on myself rather than delegating; I now `[practice]`."

Q42. Why are you leaving your current role?

Stay positive, truthful and forward-looking: move *toward* something. Never criticise your employer. If you are a founder: "I've built RefinePoint to `[stage]` and learned what I wanted about owning a whole product. I miss working on larger systems with strong colleagues, and I want `[what this role offers]`." Prepare your honest answer about what happens to RefinePoint (35.9), because the follow-up is coming.

Q43. Why were you out of employment, or why did you start a company?

Don't apologise. "I wanted to own a whole product, from design to billing, which is hard to do inside a company. Here is what I built and what I learned." Then show evidence. If the company didn't succeed, say so plainly, and explain what you learned.

Q44. Where do you see yourself in five years?

Show direction without pretending to know, and show it could happen here: "Having wider technical influence, leading direction across several teams, still close enough to code to have good judgement. I'd want to get there by solving hard problems here, like `[something they mentioned]`." Avoid "running my own company", "your job" or "I don't plan that far".

Q45. What are your salary expectations?

Research the market first, using several sources, and know your walk-away number. Early on: "I'd like to understand the role first. Could you share the budgeted range?" If pressed, give a researched range whose bottom you'd truly accept: "For this scope in this market, I'm looking at `[range]`, depending on the full package." Consider the whole package: base, bonus, stock, notice buyout, leave. Do not inflate your current pay, because it can be verified, and lying can void an offer.

Q46. What's your notice period? Can you join earlier?

Give the exact contractual figure and don't overpromise. "It's `[x]` days. I can ask about an early release, but can't promise it." Do not accept an offer intending to use it only as leverage or to break it later; that reputation travels in a small industry.

Q47. Do you have other offers?

Be honest, and don't invent them. "I'm in conversations with `[number]` companies at `[stage]`. This role is my preference because `[reason]`." If you've no other offers, "I'm early in my search" is true and fine. A fabricated offer can be called out.

Q48. An employee on your team is underpaid compared with a new hire. What do you do?

Check the facts, then raise it with your manager or HR with data, because pay equity is a retention risk and a fairness issue. Be honest with the employee about what you can and cannot promise. Don't share others' salaries.

Q49. You discover a security issue in code a colleague shipped last month. What do you do?

Assess severity straight away, and tell the owner and the security process; don't sit on it or fix it quietly. If it may have been exploited, check logs and follow the incident process, including any legal duties to notify. Fix, add a check that catches the whole class of bug, and treat it as a system gap rather than as the colleague's fault.

Q50. Your manager asks you to ship something you believe is unsafe.

State the risk in writing, with specifics and a proposed alternative. Most of these are business trade-offs, and once the owner has understood and accepted the risk, you can proceed. If it's truly illegal or endangers users' data or money, refuse and use the formal escalation route. Knowing the difference is the judgement being tested.

Q51. How do you measure whether your team is doing well?

Outcomes and flow, not activity. The DORA metrics (deployment frequency, lead time, change failure rate, time to restore), reliability against SLOs, and whether customers got what they needed. Add attrition and a regular conversation about morale. Never lines of code or velocity comparisons.

Q52. How would you spend your first 90 days here?

First 30: listen, read, ship small things, meet stakeholders, and learn why things are as they are. Next 30: take ownership of something real, and identify the top two or three problems with evidence. Last 30: deliver one visible improvement and propose a plan with the team. No big reorganisation or rewrite in the first month.

Q53. A system you inherited is a mess. The team wants to rewrite it. Would you?

Usually not all at once. Big rewrites take longer than planned, lose hidden business rules, and freeze features. Find out what really hurts, add tests around it, and use the strangler approach to replace the worst part first. Rewrite a component when change is impossible and the behaviour is well understood. Note that **RefinePoint**'s split kept the same API and caught behaviour with end-to-end checks, rather than rewriting the logic.

Q54. What was your biggest technical mistake as a solo founder?

RefinePoint: "Over-splitting early. Thirteen services was more deployment surface than one person needed. Also, I load tested late. And all services still share one database user. The last one is a known gap, on the list to close." Three real items, with reasons, shows that you review your own work.

Q55. Having worked alone, how will you adapt to process and code review?

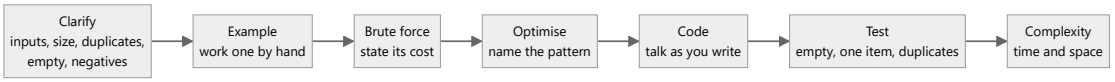
"I'm looking forward to it, because I know what I've missed: nobody has challenged my designs for a long time. I also know what process is for, having had to substitute for it with automated tests and written architecture notes. Before the founder years I worked in teams for `[years]`, and here's an example of how: `[story]`."

36. Coding Round: Java Programs & DSA Patterns

Senior candidates fail coding rounds for a surprising reason: they know the algorithms, but have not written one by hand in years, and they go silent under pressure. Interviewers at this level care less about the cleverest trick than about whether you clarify the problem, name a brute force and its cost, improve it, write clean code, and test it on an edge case without being asked.

Every solution below is compiled Java 21. Learn the **pattern** behind each problem, not the code: most interview questions are one of about fifteen patterns with a different story attached.

36.1 How to run the round



Questions worth asking first: How large is the input? Can it be empty or null? Are there duplicates or negatives? Is it sorted? Can I modify it? Is the answer unique? ASCII or Unicode? The size tells you the target complexity:

Input size n	Acceptable complexity	Likely technique
≤ 20	$O(2^n)$, $O(n!)$	Backtracking, bitmask
≤ 500	$O(n^3)$	Interval DP
≤ 5,000	$O(n^2)$	2-D DP, nested loops
≤ 10^6	O(n log n), O(n)	Sort, heap, hash, two pointers, sliding window
≥ 10^9	O(log n), O(1)	Binary search, math

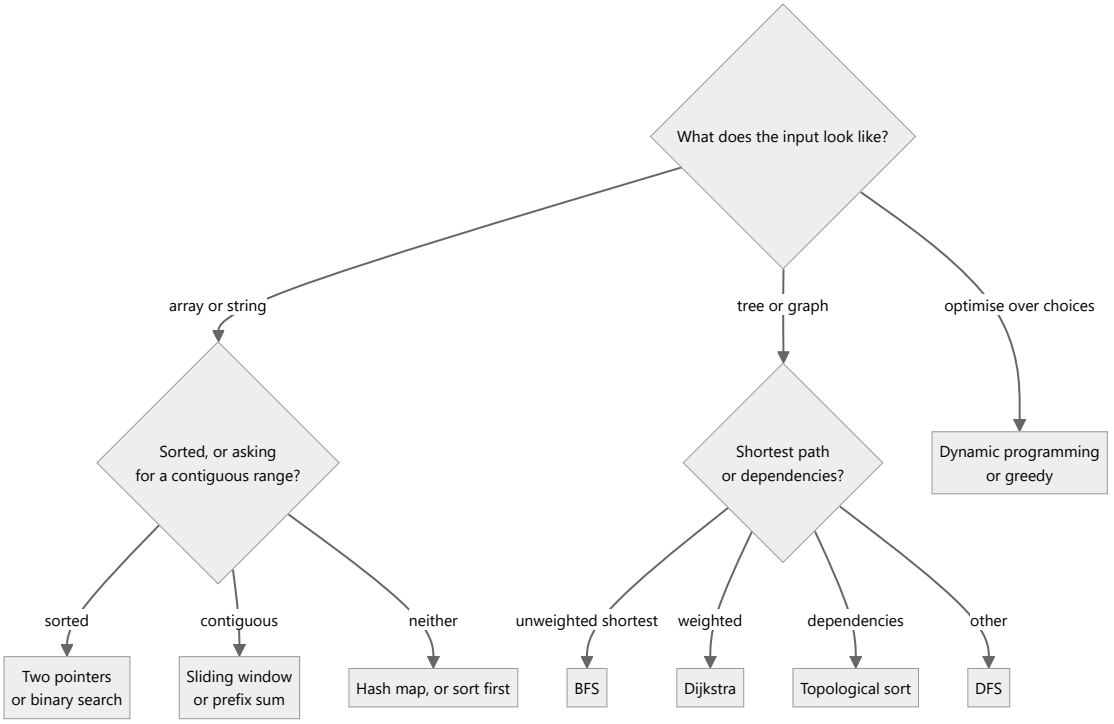
36.2 Big-O cheat sheet

Structure	Access	Search	Insert	Delete	Note
Array / `ArrayList`	O(1)	O(n)	O(1) amortised at end, O(n) middle	O(n)	Cache friendly
`LinkedList`	O(n)	O(n)	O(1) at ends	O(1) at ends	Rarely the right choice
`ArrayDeque`	–	O(n)	O(1) at ends	O(1) at ends	Use for stack and queue
`HashMap` / `HashSet`	–	O(1) average	O(1) average	O(1) average	Worst O(log n) with treeified bins
`TreeMap` / `TreeSet`	–	O(log n)	O(log n)	O(log n)	Sorted, floor and ceiling
`PriorityQueue`	O(1) peek	O(n)	O(log n)	O(log n) poll	Building from a collection is O(n)

Algorithm	Time	Space
`Arrays.sort(int[])` (dual-pivot quicksort)	O(n log n) average	O(log n)
`Collections.sort` / `Arrays.sort(Object[])` (TimSort, stable)	O(n log n)	O(n)
Binary search	O(log n)	O(1)
BFS / DFS	O(V + E)	O(V)
Dijkstra with a heap	O((V + E) log V)	O(V)
Topological sort	O(V + E)	O(V)

Pattern recognition table

Problem signal	Technique
Sorted array, pair or triple with a target	Two pointers
"Longest or shortest subarray/substring with property"	Sliding window
Subarray sum equals k, with negatives	Prefix sum + hash map
Sorted or monotonic answer, "minimum feasible value"	Binary search, or binary search on the answer
"Have I seen this?", counts, complement lookup	Hash map or set
Top K, K-th largest, merge K streams	Heap
Next greater or smaller, histogram	Monotonic stack
Sliding window maximum	Monotonic deque
Overlapping intervals, scheduling	Sort by start or end
Linked list middle or cycle	Fast and slow pointers
Tree path, depth, subtree property	DFS recursion
Shortest path with no weights, level by level	BFS
Dependencies, build order	Topological sort
Weighted shortest path, non-negative weights	Dijkstra
Connectivity, merging groups	Union-Find or DFS
"Count ways", "min/max cost", choices with overlap	Dynamic programming
All combinations or permutations	Backtracking



36.3 Java idioms for interviews

```
void idioms(int[] nums, String s, List<Integer> list) {
    int[] counts = new int[26];                      // faster than a map for lowercase letters
    counts[s.charAt(0) - 'a']++;
    Map<Character, Integer> freq = new HashMap<>();
    freq.merge('x', 1, Integer::sum);                // increment a count
    Map<String, List<Integer>> groups = new HashMap<>();
    groups.computeIfAbsent("key", k -> new ArrayList<>()).add(1);
    Deque<Integer> stack = new ArrayDeque<>();       // not java.util.Stack (synchronised,
legacy)
    stack.push(1); stack.pop(); stack.peek();
    Deque<Integer> queue = new ArrayDeque<>();
    queue.offer(1); queue.poll();
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
    PriorityQueue<int[]> byFirst = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
    Arrays.fill(nums, -1);
    int[] copy = Arrays.copyOf(nums, nums.length);
    int mid = 0 + (nums.length - 0) / 2;             // lo + (hi - lo) / 2 avoids int overflow
    List<Integer> boxed = Arrays.stream(nums).boxed().toList();
    int[] unboxed = list.stream().mapToInt(Integer::intValue).toArray();
    String joined = String.join(",", List.of("a", "b"));
    StringBuilder sb = new StringBuilder().append('a').reverse();
    char[] chars = s.toCharArray();
    String back = new String(chars);
}
```

Trap: `a[0] - b[0]` in a comparator can overflow; use `Integer.compare` or `Comparator.comparingInt`. Compare boxed `Integer` values with `equals`, not `==`, since `==` is only safe by accident for values in the cache range, -128 to 127. `list.remove(1)` removes by *index*; `list.remove(Integer.valueOf(1))` removes by value. `Arrays.asList(intArray)` gives a list of one `int[]`.

36.4 String problems

Q1. Reverse a string.

Two pointers swapping in a char array: O(n) time, O(n) for the copy. In practice, use `new StringBuilder(s).reverse()`, which also handles surrogate pairs correctly.

```
String reverse(String s) {
    char[] c = s.toCharArray();
    for (int i = 0, j = c.length - 1; i < j; i++, j--) {
        char t = c[i]; c[i] = c[j]; c[j] = t;
    }
    return new String(c);
}
```

Q2. Check a palindrome, ignoring case and non-alphanumerics.

Two pointers moving inwards. O(n) time, O(1) space.

```
boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (!Character.isLetterOrDigit(s.charAt(i))) i++;
        else if (!Character.isLetterOrDigit(s.charAt(j))) j--;
        else if (Character.toLowerCase(s.charAt(i++)) != Character.toLowerCase(s.charAt(j--)))
return false;
    }
    return true;
}
```

Q3. Are two strings anagrams?

Count letters up and down. O(n) time, O(1) space for a fixed alphabet. Sorting is O(n log n). Use a `HashMap` for full Unicode.

```
boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < a.length(); i++) { count[a.charAt(i) - 'a']++; count[b.charAt(i) - 'a']-
-; }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

Q4. First non-repeating character.

Count, then scan in the original order. O(n) time. `LinkedHashMap` also works and keeps insertion order for any alphabet.

```
int firstUnique(String s) {
    int[] count = new int[128];
    for (char c : s.toCharArray()) count[c]++;
    for (int i = 0; i < s.length(); i++) if (count[s.charAt(i)] == 1) return i;
    return -1;
}
```

Q5. Longest substring without repeating characters.

Sliding window storing the last index of each character; jump the left edge past a repeat. O(n) time, O(alphabet) space.

```
int longestUnique(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int best = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        Integer seen = last.put(s.charAt(right), right);
        if (seen != null && seen >= left) left = seen + 1;
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Q6. String compression ("aaabb" → "a3b2").

Count runs. Return the original if the result isn't shorter. O(n).

```
String compress(String s) {
    StringBuilder out = new StringBuilder();
    for (int i = 0; i < s.length(); ) {
        int j = i;
        while (j < s.length() && s.charAt(j) == s.charAt(i)) j++;
        out.append(s.charAt(i)).append(j - i);
        i = j;
    }
    return out.length() < s.length() ? out.toString() : s;
}
```

Q7. Reverse the words in a sentence.

Split on whitespace, join backwards. O(n).

```
String reverseWords(String s) {
    String[] w = s.trim().split("\\s+");
    Collections.reverse(Arrays.asList(w));
    return String.join(" ", w);
}
```

Q8. Group anagrams.

The sorted letters form the key. O(n · k log k) for n words of length k.

```
List<List<String>> groupAnagrams(String[] words) {
    Map<String, List<String>> groups = new HashMap<>();
    for (String w : words) {
        char[] c = w.toCharArray();
        Arrays.sort(c);
        groups.computeIfAbsent(new String(c), k -> new ArrayList<>()).add(w);
    }
    return new ArrayList<>(groups.values());
}
```

Q9. Longest palindromic substring.

Expand around each of the 2n−1 centres. O(n^2) time, O(1) space.

```
String longestPalindrome(String s) {
    int start = 0, len = 0;
    for (int c = 0; c < s.length(); c++) {
        for (int odd = 0; odd <= 1; odd++) {
            int l = c, r = c + odd;
            while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
            if (r - l - 1 > len) { len = r - l - 1; start = l + 1; }
        }
    }
    return s.substring(start, start + len);
}
```

36.5 Arrays, hashing and two pointers

Q10. Two sum.

For each value, look up its complement. O(n) time, O(n) space. If the array is sorted, two pointers give O(1) space.

```
int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> index = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        Integer j = index.get(target - nums[i]);
        if (j != null) return new int[] { j, i };
        index.put(nums[i], i);
    }
    return new int[0];
}
```

Q11. Three sum (unique triples adding to zero).

Sort, fix one value, two pointers on the rest, skip duplicates. $O(n^2)$ time.

```
List<List<Integer>> threeSum(int[] a) {
    Arrays.sort(a);
    List<List<Integer>> out = new ArrayList<>();
    for (int i = 0; i < a.length - 2 && a[i] <= 0; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;
        int l = i + 1, r = a.length - 1;
        while (l < r) {
            int sum = a[i] + a[l] + a[r];
            if (sum < 0) l++;
            else if (sum > 0) r--;
            else {
                out.add(List.of(a[i], a[l], a[r]));
                while (l < r && a[l] == a[l + 1]) l++;
                while (l < r && a[r] == a[r - 1]) r--;
                l++; r--;
            }
        }
    }
    return out;
}
```

Q12. Maximum subarray sum (Kadane).

At each index, either extend the current run or start again. O(n), O(1). Starting from `nums[0]` handles all-negative input.

```
int maxSubArray(int[] nums) {
    int best = nums[0], current = nums[0];
    for (int i = 1; i < nums.length; i++) {
        current = Math.max(nums[i], current + nums[i]);
        best = Math.max(best, current);
    }
```

```
    return best;
}
```

Q13. Move zeros to the end, keeping order.

Write non-zeros forward, then fill with zeros. O(n), O(1).

```
void moveZeros(int[] a) {
    int w = 0;
    for (int x : a) if (x != 0) a[w++] = x;
    while (w < a.length) a[w++] = 0;
}
```

Q14. Rotate an array right by k.

Reverse everything, then reverse each part. O(n), O(1). Remember `k %= n`.

```
void rotate(int[] a, int k) {
    if (a.length == 0) return;
    k %= a.length;
    reverse(a, 0, a.length - 1);
    reverse(a, 0, k - 1);
    reverse(a, k, a.length - 1);
}
void reverse(int[] a, int i, int j) {
    while (i < j) { int t = a[i]; a[i++] = a[j]; a[j--] = t; }
}
```

Q15. Merge overlapping intervals.

Sort by start, extend the last interval or add a new one. O(n log n).

```
int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, Comparator.comparingInt(x -> x[0]));
    List<int[]> out = new ArrayList<>();
    for (int[] cur : intervals) {
        if (out.isEmpty() || out.get(out.size() - 1)[1] < cur[0]) out.add(cur.clone());
        else out.get(out.size() - 1)[1] = Math.max(out.get(out.size() - 1)[1], cur[1]);
    }
    return out.toArray(new int[0][]);
}
```

Q16. Sliding window maximum.

A deque of indices with decreasing values: the front is the maximum. Each index enters and leaves once, so O(n).

```
int[] maxWindow(int[] a, int k) {
    int[] out = new int[a.length - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();
    for (int i = 0; i < a.length; i++) {
        if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast();
```

```
        dq.offerLast(i);
        if (i >= k - 1) out[i - k + 1] = a[dq.peekFirst()];
    }
    return out;
}
```

Q17. Product of array except self, without division.

Left products in one pass, right products in a second. O(n), O(1) extra.

```
int[] productExceptSelf(int[] a) {
    int[] out = new int[a.length];
    out[0] = 1;
    for (int i = 1; i < a.length; i++) out[i] = out[i - 1] * a[i - 1];
    int right = 1;
    for (int i = a.length - 1; i >= 0; i--) { out[i] *= right; right *= a[i]; }
    return out;
}
```

Q18. Best time to buy and sell a stock, one transaction.

Track the lowest price so far. O(n), O(1).

```
int maxProfit(int[] prices) {
    int min = Integer.MAX_VALUE, best = 0;
    for (int p : prices) { min = Math.min(min, p); best = Math.max(best, p - min); }
    return best;
}
```

Q19. Find the missing number in 0..n.

XOR indices and values: everything cancels except the missing number. O(n), O(1), no overflow risk.

```
int missing(int[] a) {
    int x = a.length;
    for (int i = 0; i < a.length; i++) x ^= i ^ a[i];
    return x;
}
```

Q20. Longest consecutive sequence.

Put everything in a set. Only start counting from a number with no predecessor, so each element is visited at most twice. O(n).

```
int longestConsecutive(int[] a) {
    Set<Integer> set = new HashSet<>();
    for (int x : a) set.add(x);
    int best = 0;
    for (int x : set) {
        if (set.contains(x - 1)) continue;
        int len = 1;
        while (set.contains(x + len)) len++;
        best = Math.max(best, len);
```

```
    }
    return best;
}
```

Q21. Count subarrays summing to k.

Prefix sums: a subarray ending here sums to k when `prefix - k` was seen earlier. O(n). A sliding window fails here because negatives break monotonicity.

```
int subarraySum(int[] a, int k) {
    Map<Integer, Integer> seen = new HashMap<>(Map.of(0, 1));
    int prefix = 0, count = 0;
    for (int x : a) {
        prefix += x;
        count += seen.getOrDefault(prefix - k, 0);
        seen.merge(prefix, 1, Integer::sum);
    }
    return count;
}
```

Q22. Container with most water.

Pointers at both ends; move the shorter side, as only that can improve the area. O(n).

```
int maxArea(int[] h) {
    int l = 0, r = h.length - 1, best = 0;
    while (l < r) {
        best = Math.max(best, Math.min(h[l], h[r]) * (r - l));
        if (h[l] < h[r]) l++; else r--;
    }
    return best;
}
```

Q23. Remove duplicates from a sorted array in place.

A slow write pointer. O(n), O(1). Returns the new length.

```
int removeDuplicates(int[] a) {
    int w = 0;
    for (int x : a) if (w == 0 || a[w - 1] != x) a[w++] = x;
    return w;
}
```

Q24. Maximum sum of a window of size k.

Add the entering value, subtract the leaving one. O(n), O(1).

```
long maxSumWindow(int[] a, int k) {
    long sum = 0, best;
    for (int i = 0; i < k; i++) sum += a[i];
    best = sum;
    for (int i = k; i < a.length; i++) { sum += a[i] - a[i - k]; best = Math.max(best, sum); }
```

```
    return best;
}
```

Q25. Minimum window substring.

Expand right until the window covers all needed characters, then shrink from the left. O(|s| + |t|).

```
String minWindow(String s, String t) {
    int[] need = new int[128];
    for (char c : t.toCharArray()) need[c]++;
    int missing = t.length(), start = 0, best = Integer.MAX_VALUE, l = 0;
    for (int r = 0; r < s.length(); r++) {
        if (need[s.charAt(r)]-- > 0) missing--;
        while (missing == 0) {
            if (r - l + 1 < best) { best = r - l + 1; start = l; }
            if (++need[s.charAt(l++)] > 0) missing++;
        }
    }
    return best == Integer.MAX_VALUE ? "" : s.substring(start, start + best);
}
```

36.6 Binary search variants

The bug is almost always in the boundary. Pick one template and keep the invariant in your head.

Q26. Classic binary search.

O(log n). `lo + (hi - lo) / 2` avoids overflow, and `Arrays.binarySearch` is the library version.

```
int search(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}
```

Q27. First and last position of a target.

The lower bound finds the first index with value ≥ x. The last position of x is `lowerBound(x + 1) - 1`.

```
int lowerBound(int[] a, int x) {
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < x) lo = mid + 1; else hi = mid;
    }
    return lo;
}
int[] searchRange(int[] a, int x) {
    int first = lowerBound(a, x);
    if (first == a.length || a[first] != x) return new int[] { -1, -1 };
```

```
    return new int[] { first, lowerBound(a, x + 1) - 1 };
}
```

Q28. Search a rotated sorted array.

One half is always sorted; check whether the target lies in it. O(log n).

```
int searchRotated(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid]) {
            if (a[lo] <= target && target < a[mid]) hi = mid - 1; else lo = mid + 1;
        } else {
            if (a[mid] < target && target <= a[hi]) lo = mid + 1; else hi = mid - 1;
        }
    }
    return -1;
}
```

Q29. Find the minimum in a rotated sorted array.

Compare with the right end to find which side holds the drop. O(log n).

```
int findMin(int[] a) {
    int lo = 0, hi = a.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] > a[hi]) lo = mid + 1; else hi = mid;
    }
    return a[lo];
}
```

Q30. Binary search on the answer: minimum eating speed (Koko).

When feasibility is monotonic in the answer, binary search the answer itself. O(n log max).

```
int minSpeed(int[] piles, int hours) {
    int lo = 1, hi = Arrays.stream(piles).max().orElse(1);
    while (lo < hi) {
        int speed = lo + (hi - lo) / 2;
        long need = 0;
        for (int p : piles) need += (p + speed - 1L) / speed;
        if (need <= hours) hi = speed; else lo = speed + 1;
    }
    return lo;
}
```

36.7 Linked lists

```
class ListNode { int val; ListNode next; ListNode(int v) { val = v; } }
```

Q31. Reverse a linked list.

Three pointers. O(n), O(1). Be ready to write it from memory without hesitation.

```
ListNode reverseList(ListNode head) {
    ListNode prev = null;
    while (head != null) { ListNode next = head.next; head.next = prev; prev = head; head = next;
}
    return prev;
}
```

Q32. Detect a cycle and find where it starts (Floyd).

Fast and slow meet inside any cycle. A pointer from the head and one from the meeting point then meet at the start. O(n), O(1).

```
ListNode cycleStart(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) {
            for (slow = head; slow != fast; fast = fast.next) slow = slow.next;
            return slow;
        }
    }
    return null;
}
```

Q33. Merge two sorted lists.

A dummy head avoids special-casing the first node. O(n + m), O(1).

```
ListNode mergeLists(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { tail.next = a; a = a.next; } else { tail.next = b; b = b.next; }
        tail = tail.next;
    }
    tail.next = a != null ? a : b;
    return dummy.next;
}
```

Q34. Middle of a linked list.

When fast reaches the end, slow is in the middle (the second middle for even lengths).

```
ListNode middle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
    return slow;
}
```

Q35. Remove the n-th node from the end.

Advance one pointer by n first, then move both. A dummy node handles removing the head.

```
ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0); dummy.next = head;
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) fast = fast.next;
    while (fast.next != null) { fast = fast.next; slow = slow.next; }
    slow.next = slow.next.next;
    return dummy.next;
}
```

Q36. LRU cache using `LinkedHashMap`.

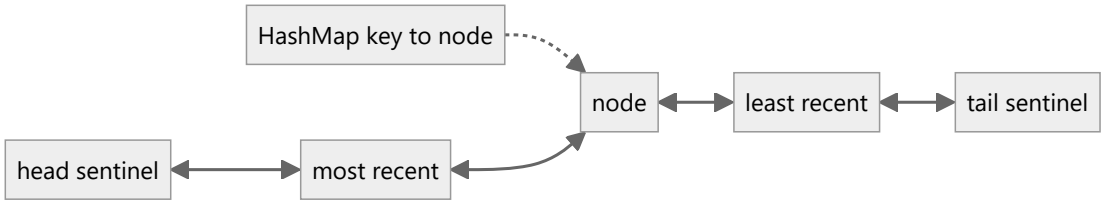
Access-order mode moves each read entry to the tail; `removeEldestEntry` evicts the head. O(1). Not thread-safe; wrap with `Collections.synchronizedMap` or use Caffeine in production.

```
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    LruCache(int capacity) { super(16, 0.75f, true); this.capacity = capacity; }
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> eldest) { return size() >
capacity; }
}
```

> **RefinePoint:** `ExpressionEngine` caches parsed rules exactly this way: a `Collections.synchronizedMap` around an access-ordered `LinkedHashMap` that keeps 2,000 entries.

Q37. LRU cache with a doubly linked list and `HashMap`.

This is what interviewers usually want. The map gives O(1) lookup, the list gives O(1) move and evict. Sentinel head and tail nodes remove all null checks.




```
class LruDll {
    private class Node { int key, value; Node prev, next; Node(int k, int v) { key = k; value =
v; } }
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0), tail = new Node(0, 0);
    private final int capacity;
    LruDll(int capacity) { this.capacity = capacity; head.next = tail; tail.prev = head; }

    int get(int key) {
        Node n = map.get(key);
        if (n == null) return -1;
        unlink(n); addFront(n);
        return n.value;
```

```
    }
    void put(int key, int value) {
        Node n = map.get(key);
        if (n != null) { n.value = value; unlink(n); addFront(n); return; }
        if (map.size() == capacity) { Node lru = tail.prev; unlink(lru); map.remove(lru.key); }
        n = new Node(key, value);
        map.put(key, n);
        addFront(n);
    }
    private void unlink(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
    private void addFront(Node n) { n.next = head.next; n.prev = head; head.next.prev = n;
head.next = n; }
}
```

Follow-up: "Make it thread-safe." A single lock is simple and correct, but `get` also mutates the list, so a read-write lock doesn't help. For high concurrency, use striped segments or Caffeine, which buffers reads and replays them.

36.8 Stacks and queues

Q38. Valid parentheses.

Push the expected closer; every closer must match the top. O(n).

```
boolean validParens(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        switch (c) {
            case '(' -> stack.push(')');
            case '[' -> stack.push(']');
            case '{' -> stack.push('}');
            default -> { if (stack.isEmpty() || stack.pop() != c) return false; }
        }
    }
    return stack.isEmpty();
}
```

Q39. Min stack with O(1) `getMin` .

Store each value together with the minimum at that depth.

```
class MinStack {
    private final Deque<int[]> s = new ArrayDeque<>();
    void push(int x) { s.push(new int[] { x, s.isEmpty() ? x : Math.min(x, s.peek()[1]) }); }
    void pop() { s.pop(); }
    int top() { return s.peek()[0]; }
    int getMin() { return s.peek()[1]; }
}
```

Q40. Next greater element.

A monotonic stack holding indices still waiting for an answer. Each index is pushed and popped once, so O(n).

```
int[] nextGreater(int[] a) {
    int[] out = new int[a.length];
    Arrays.fill(out, -1);
    Deque<Integer> waiting = new ArrayDeque<>();
    for (int i = 0; i < a.length; i++) {
        while (!waiting.isEmpty() && a[waiting.peek()] < a[i]) out[waiting.pop()] = a[i];
        waiting.push(i);
    }
    return out;
}
```

Q41. Implement a queue using two stacks.

Push to `in`; pop from `out`, refilling it only when empty. Amortised O(1), since each element moves once.

```
class QueueTwoStacks {
    private final Deque<Integer> in = new ArrayDeque<>(), out = new ArrayDeque<>();
    void add(int x) { in.push(x); }
    int poll() {
        if (out.isEmpty()) while (!in.isEmpty()) out.push(in.pop());
        return out.pop();
    }
}
```

36.9 Trees

```
class TreeNode { int val; TreeNode left, right; TreeNode(int v) { val = v; } }
```

Q42. Tree traversals: recursive and iterative.

Inorder gives sorted order for a BST. Preorder copies a tree. Postorder deletes one. Recursion is O(n) time and O(h) stack; on a skewed tree of a million nodes it can overflow, so know the iterative version.

```
void inorder(TreeNode n, List<Integer> out) {
    if (n == null) return;
    inorder(n.left, out); out.add(n.val); inorder(n.right, out);
}
List<Integer> inorderIterative(TreeNode root) {
    List<Integer> out = new ArrayList<>();
    Deque<TreeNode> stack = new ArrayDeque<>();
    for (TreeNode cur = root; cur != null || !stack.isEmpty(); cur = cur.right) {
        while (cur != null) { stack.push(cur); cur = cur.left; }
        cur = stack.pop();
        out.add(cur.val);
    }
    return out;
}
```

Q43. Level-order traversal (BFS).

Process the queue's current size as one level. O(n) time, O(width) space.

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> out = new ArrayList<>();
    if (root == null) return out;
    Deque<TreeNode> q = new ArrayDeque<>(List.of(root));
    while (!q.isEmpty()) {
        List<Integer> level = new ArrayList<>();
        for (int i = q.size(); i > 0; i--) {
            TreeNode n = q.poll();
            level.add(n.val);
            if (n.left != null) q.offer(n.left);
            if (n.right != null) q.offer(n.right);
        }
        out.add(level);
    }
    return out;
}
```

Q44. Height of a tree.

O(n) time, O(h) stack.

```
int height(TreeNode n) {
    return n == null ? 0 : 1 + Math.max(height(n.left), height(n.right));
}
```

Q45. Lowest common ancestor.

In a binary tree: if both sides find something, this node is the LCA. O(n). In a BST, walk down using ordering: O(h).

```
TreeNode lca(TreeNode n, TreeNode p, TreeNode q) {
    if (n == null || n == p || n == q) return n;
    TreeNode l = lca(n.left, p, q), r = lca(n.right, p, q);
    return l != null && r != null ? n : l != null ? l : r;
}
TreeNode lcaBst(TreeNode n, int p, int q) {
    while (n != null) {
        if (p < n.val && q < n.val) n = n.left;
        else if (p > n.val && q > n.val) n = n.right;
        else return n;
    }
    return null;
}
```

Q46. Validate a binary search tree.

Pass allowed bounds down. Checking only a node's immediate children is the classic wrong answer. `long` bounds handle nodes equal to `Integer.MAX_VALUE`.

```
boolean isBst(TreeNode n, long min, long max) {
    if (n == null) return true;
    return n.val > min && n.val < max
        && isBst(n.left, min, n.val) && isBst(n.right, n.val, max);
}
```

Q47. Diameter of a binary tree.

Compute the height while tracking the best left + right. O(n).

```
int best = 0;
int depth(TreeNode n) {
    if (n == null) return 0;
    int l = depth(n.left), r = depth(n.right);
    best = Math.max(best, l + r);
    return 1 + Math.max(l, r);
}
```

36.10 Graphs

Represent a graph as an adjacency list, `List<List<Integer>>`. A grid is an implicit graph with up to four neighbours per cell.

Q48. BFS and DFS.

Both O(V + E). BFS finds the shortest path by number of edges; DFS suits connectivity, cycles and ordering. Mark nodes as visited when you *enqueue* them, or they get added several times.

```
List<Integer> bfs(List<List<Integer>> g, int start) {
    boolean[] seen = new boolean[g.size()];
    List<Integer> order = new ArrayList<>();
    Deque<Integer> q = new ArrayDeque<>(List.of(start));
    seen[start] = true;
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : g.get(u)) if (!seen[v]) { seen[v] = true; q.offer(v); }
    }
    return order;
}
void dfs(List<List<Integer>> g, int u, boolean[] seen, List<Integer> order) {
    seen[u] = true;
    order.add(u);
    for (int v : g.get(u)) if (!seen[v]) dfs(g, v, seen, order);
}
```

Q49. Number of islands.

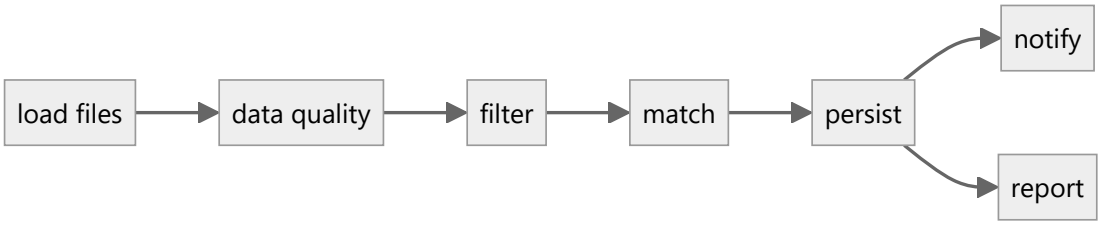
Flood-fill each unseen land cell. O(rows × cols). This modifies the grid; ask first whether that's allowed.

```
int islands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++)
        for (int c = 0; c < grid[0].length; c++)
            if (grid[r][c] == '1') { count++; sink(grid, r, c); }
    return count;
}
void sink(char[][] g, int r, int c) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != '1') return;
    g[r][c] = '0';
```

```
    sink(g, r + 1, c); sink(g, r - 1, c); sink(g, r, c + 1); sink(g, r, c - 1);
}
```

Q50. Topological sort: job dependencies (Kahn's algorithm).

Repeatedly take a job with no remaining prerequisites. O(V + E). If some jobs are never taken, there is a cycle, and it's the same algorithm that detects it.




```
List<Integer> topoSort(int n, int[][] deps) {           // deps[i] = {before, after}
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    int[] waitingOn = new int[n];
    for (int[] d : deps) { g.get(d[0]).add(d[1]); waitingOn[d[1]]++; }
    Deque<Integer> ready = new ArrayDeque<>();
    for (int i = 0; i < n; i++) if (waitingOn[i] == 0) ready.offer(i);
    List<Integer> order = new ArrayList<>();
    while (!ready.isEmpty()) {
        int u = ready.poll();
        order.add(u);
        for (int v : g.get(u)) if (--waitingOn[v] == 0) ready.offer(v);
    }
    if (order.size() != n) throw new IllegalArgumentException("Cycle in job dependencies");
    return order;
}
```

Q51. Detect a cycle in a directed graph with DFS.

Three colours: unvisited, in the current path, done. An edge to a node in the current path is a cycle. O(V + E). A plain visited set is not enough, because a node reached twice by different paths isn't necessarily a cycle.

```
boolean hasCycle(List<List<Integer>> g) {
    int[] state = new int[g.size()];                    // 0 new, 1 in path, 2 done
    for (int i = 0; i < g.size(); i++) if (state[i] == 0 && visit(g, i, state)) return true;
    return false;
}
boolean visit(List<List<Integer>> g, int u, int[] state) {
    state[u] = 1;
    for (int v : g.get(u)) {
        if (state[v] == 1) return true;
        if (state[v] == 0 && visit(g, v, state)) return true;
    }
    state[u] = 2;
    return false;
}
```

Q52. Dijkstra's shortest path.

Always expand the closest unfinished node. O((V + E) log V). Weights must not be negative; use Bellman-Ford if they are. Java's `PriorityQueue` has no decrease-key, so we add duplicates and skip stale entries.

```
long[] dijkstra(List<List<int[]>> g, int src) {          // g.get(u) holds {v, weight}
    long[] dist = new long[g.size()];
    Arrays.fill(dist, Long.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<long[]> pq = new PriorityQueue<>(Comparator.comparingLong(a -> a[1]));
    pq.offer(new long[] { src, 0 });
    while (!pq.isEmpty()) {
        long[] top = pq.poll();
        int u = (int) top[0];
        if (top[1] > dist[u]) continue;                  // stale entry
        for (int[] e : g.get(u)) {
            long d = dist[u] + e[1];
            if (d < dist[e[0]]) { dist[e[0]] = d; pq.offer(new long[] { e[0], d }); }
        }
    }
    return dist;
}
```

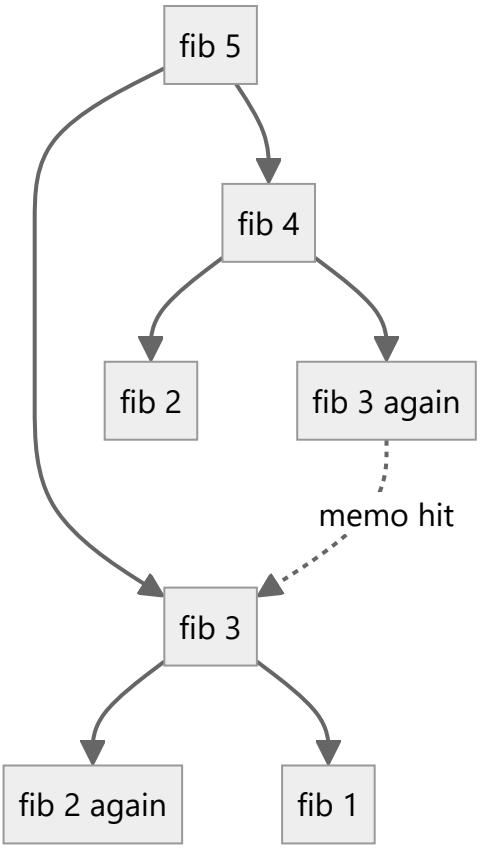
Q53. Union-Find: count connected components.

Path compression and union by size give nearly O(1) per operation.

```
class UnionFind {
    private final int[] parent, size;
    int components;
    UnionFind(int n) {
        parent = new int[n]; size = new int[n]; components = n;
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    boolean union(int a, int b) {
        int x = find(a), y = find(b);
        if (x == y) return false;
        if (size[x] < size[y]) { int t = x; x = y; y = t; }
        parent[y] = x; size[x] += size[y]; components--;
        return true;
    }
}
```

36.11 Dynamic programming

Recognise it by two properties: an optimal answer built from optimal sub-answers, and the same sub-problems recurring. The method is to define the state in words, write the recurrence, set base cases, then choose memoisation (top-down) or a table (bottom-up).



Q54. Fibonacci with memoisation.

Exponential without the cache, O(n) with it. A `Map` memo must not use `computeIfAbsent` recursively, because `HashMap` throws `ConcurrentModificationException`. Use an array or `get` then `put`.

```
long fib(int n, long[] memo) {
    if (n < 2) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
}
```

Q55. Climbing stairs (1 or 2 steps).

`ways(n) = ways(n-1) + ways(n-2)`: Fibonacci again. O(n), O(1) with two variables.

```
int climb(int n) {
    int a = 1, b = 1;
    for (int i = 2; i <= n; i++) { int c = a + b; a = b; b = c; }
    return b;
}
```

Q56. Coin change: fewest coins.

`dp[x] = min(dp[x - coin] + 1)`. O(amount × coins). Greedy fails: with coins {1, 3, 4} and amount 6, greedy gives 4+1+1 but the best is 3+3.

```java
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;
    for (int x = 1; x <= amount; x++)
        for (int c : coins)
            if (c <= x) dp[x] = Math.min(dp[x], dp[x - c] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Q57. Longest increasing subsequence.

`tails[i]` is the smallest tail of any increasing subsequence of length i+1. Binary search gives O(n log n); the plain DP is O(n^2). `tails` is not itself a valid subsequence, and interviewers like to ask about that.

```java
int lis(int[] a) {
    int[] tails = new int[a.length];
    int len = 0;
    for (int x : a) {
        int i = Arrays.binarySearch(tails, 0, len, x);
        if (i < 0) i = -(i + 1);
        tails[i] = x;
        if (i == len) len++;
    }
    return len;
}
```

Q58. Longest common subsequence.

`dp[i][j]` : LCS of the first i and j characters. O(n × m) time and space.

```java
int lcs(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];
    for (int i = 1; i <= a.length(); i++)
        for (int j = 1; j <= b.length(); j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1)
                ? dp[i - 1][j - 1] + 1
                : Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[a.length()][b.length()];
}
```

Q59. 0/1 knapsack.

A 1-D array, looping capacity **downwards** so each item is used at most once. Looping upwards would give the unbounded version. O(n × W).

```java
int knapsack(int[] weight, int[] value, int capacity) {
    int[] dp = new int[capacity + 1];
    for (int i = 0; i < weight.length; i++)
        for (int w = capacity; w >= weight[i]; w--)
            dp[w] = Math.max(dp[w], dp[w - weight[i]] + value[i]);
    return dp[capacity];
}
```

36.12 Heaps

Q60. Top K frequent elements.

A min-heap of size k, evicting the least frequent. O(n log k). A bucket sort by frequency gives O(n).

```
List<Integer> topK(int[] a, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int x : a) freq.merge(x, 1, Integer::sum);
    PriorityQueue<Map.Entry<Integer, Integer>> heap = new PriorityQueue<>
(Map.Entry.comparingByValue());
    for (var e : freq.entrySet()) {
        heap.offer(e);
        if (heap.size() > k) heap.poll();
    }
    return heap.stream().map(Map.Entry::getKey).toList();
}
```

Q61. K-th largest element.

A min-heap of size k: its top is the answer. O(n log k), which is better than sorting for small k on huge or streaming input. Quickselect is O(n) on average.

```
int kthLargest(int[] a, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int x : a) {
        heap.offer(x);
        if (heap.size() > k) heap.poll();
    }
    return heap.peek();
}
```

Q62. Merge K sorted lists.

Keep the head of each list in a heap. O(N log k). Comparing `a.val - b.val` could overflow, so use `comparingInt`.

```
ListNode mergeK(List<ListNode> lists) {
    PriorityQueue<ListNode> heap = new PriorityQueue<>(Comparator.comparingInt((ListNode n) ->
n.val));
    for (ListNode n : lists) if (n != null) heap.offer(n);
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!heap.isEmpty()) {
        ListNode n = heap.poll();
        tail = tail.next = n;
        if (n.next != null) heap.offer(n.next);
    }
    return dummy.next;
}
```

36.13 Concurrency coding

For a senior role, expect at least one of these. Interviewers watch for `while` around `wait` / `await`, `finally` around `unlock`, and correct handling of interrupts.

Q63. Thread-safe singleton: three ways.

The enum is the simplest, and handles serialisation and reflection. The holder idiom is lazy with no locks, because class initialisation is thread-safe. Double-checked locking needs `volatile`; without it another thread may see a partly constructed object through reordering.

```
enum ConfigEnum { INSTANCE }

class ConfigHolder {
    private ConfigHolder() {}
    private static class Holder { static final ConfigHolder I = new ConfigHolder(); }
    static ConfigHolder get() { return Holder.I; }
}

class ConfigDcl {
    private static volatile ConfigDcl instance;
    private ConfigDcl() {}
    static ConfigDcl get() {
        ConfigDcl local = instance;
        if (local == null) {
            synchronized (ConfigDcl.class) {
                local = instance;
                if (local == null) instance = local = new ConfigDcl();
            }
        }
        return local;
    }
}
```

Q64. Producer-consumer with `BlockingQueue`.

`put` blocks when full, which gives backpressure; `take` blocks when empty. A poison pill stops consumers, with one pill per consumer.

```
void produceConsume(int consumers) throws InterruptedException {
    final int POISON = -1;
    BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(100);
    try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int c = 0; c < consumers; c++) {
            pool.submit(() -> {
                for (int item; (item = queue.take()) != POISON; ) System.out.println(item);
                return null;
            });
        }
        for (int i = 0; i < 1000; i++) queue.put(i);
        for (int c = 0; c < consumers; c++) queue.put(POISON);
    }                                                   // Java 21: close() waits for tasks
}
```

Q65. Print 1 to N in order using 3 threads.

Each thread waits on its own semaphore and releases the next. A shared counter and `wait/notifyAll` also works, but wakes every thread each time.

```
void printInTurn(int n, int threads) throws InterruptedException {
    Semaphore[] turn = new Semaphore[threads];
    for (int i = 0; i < threads; i++) turn[i] = new Semaphore(i == 0 ? 1 : 0);
    List<Thread> all = new ArrayList<>();
    for (int t = 0; t < threads; t++) {
        final int me = t;
        Thread thread = new Thread(() -> {
            try {
                for (int v = me + 1; v <= n; v += threads) {
                    turn[me].acquire();
                    System.out.println(Thread.currentThread().getName() + ": " + v);
                    turn[(me + 1) % threads].release();
                }
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }, "T" + (t + 1));
        all.add(thread);
        thread.start();
    }
    for (Thread thread : all) thread.join();
}
```

Q66. Token-bucket rate limiter.

Refill lazily from elapsed time, with no background thread. Tokens accumulate up to capacity, which allows short bursts.

```
class TokenBucket {
    private final double capacity, tokensPerNano;
    private double tokens;
    private long last = System.nanoTime();
    TokenBucket(int perMinute) {
        capacity = perMinute;
        tokens = perMinute;
        tokensPerNano = perMinute / 60_000_000_000d;
    }
    synchronized boolean tryAcquire() {
        long now = System.nanoTime();
        tokens = Math.min(capacity, tokens + (now - last) * tokensPerNano);
        last = now;
        if (tokens < 1) return false;
        tokens--;
        return true;
    }
}
```

RefinePoint: the gateway's `RateLimitFilter` keeps buckets like this in a `ConcurrentHashMap`, per API key (300/min), per user (900/min) and per IP (1,800/min), and answers 429 with `Retry-After`. Use `System.nanoTime` rather than `currentTimeMillis`, because wall-clock time can jump backwards.

Q67. Bounded blocking queue with `Lock` and `Condition`.

Two conditions avoid waking threads of the wrong kind. Always `while`, not `if`, because of spurious wakeups and competing threads. `unlock` always goes in `finally`.

```
class BoundedQueue<T> {
    private final Object[] items;
    private int head, tail, count;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition(), notEmpty = lock.newCondition();
    BoundedQueue(int capacity) { items = new Object[capacity]; }

    void put(T item) throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (count == items.length) notFull.await();
            items[tail] = item;
            tail = (tail + 1) % items.length;
            count++;
            notEmpty.signal();
        } finally { lock.unlock(); }
    }
    @SuppressWarnings("unchecked")
    T take() throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (count == 0) notEmpty.await();
            T item = (T) items[head];
            items[head] = null;                          // let GC collect it
            head = (head + 1) % items.length;
            count--;
            notFull.signal();
            return item;
        } finally { lock.unlock(); }
    }
}
```

Q68. A simple thread pool.

Workers loop over a bounded queue. Shutdown lets queued work finish. Catching exceptions keeps one bad task from killing a worker.

```
class SimplePool implements AutoCloseable {
    private final BlockingQueue<Runnable> queue;
    private final List<Thread> workers = new ArrayList<>();
    private volatile boolean closed;

    SimplePool(int threads, int capacity) {
        queue = new ArrayBlockingQueue<>(capacity);
        for (int i = 0; i < threads; i++) {
            Thread t = new Thread(this::work, "pool-" + i);
            workers.add(t);
            t.start();
        }
    }
    void submit(Runnable task) throws InterruptedException {
        if (closed) throw new RejectedExecutionException("Pool is closed");
        queue.put(task);
    }
```

```
    private void work() {
        while (true) {
            try {
                Runnable task = queue.poll(100, TimeUnit.MILLISECONDS);
                if (task != null) task.run();
                else if (closed) return;
            } catch (InterruptedException e) {
                return;
            } catch (RuntimeException e) {
                System.err.println("Task failed: " + e);
            }
        }
    }
    @Override public void close() throws InterruptedException {
        closed = true;
        for (Thread t : workers) t.join();
    }
}
```

Follow-up: a `submit` racing with `close` can leave a task in the queue after workers exit. Fixing that needs a lock around the check and the insert, which is one reason to use `ThreadPoolExecutor` in real code. Say so.

Q69. Write a deadlock, then fix it.

Two threads take the same two locks in opposite orders. The fix is a global lock order, here by account id. `tryLock` with a timeout is the alternative.

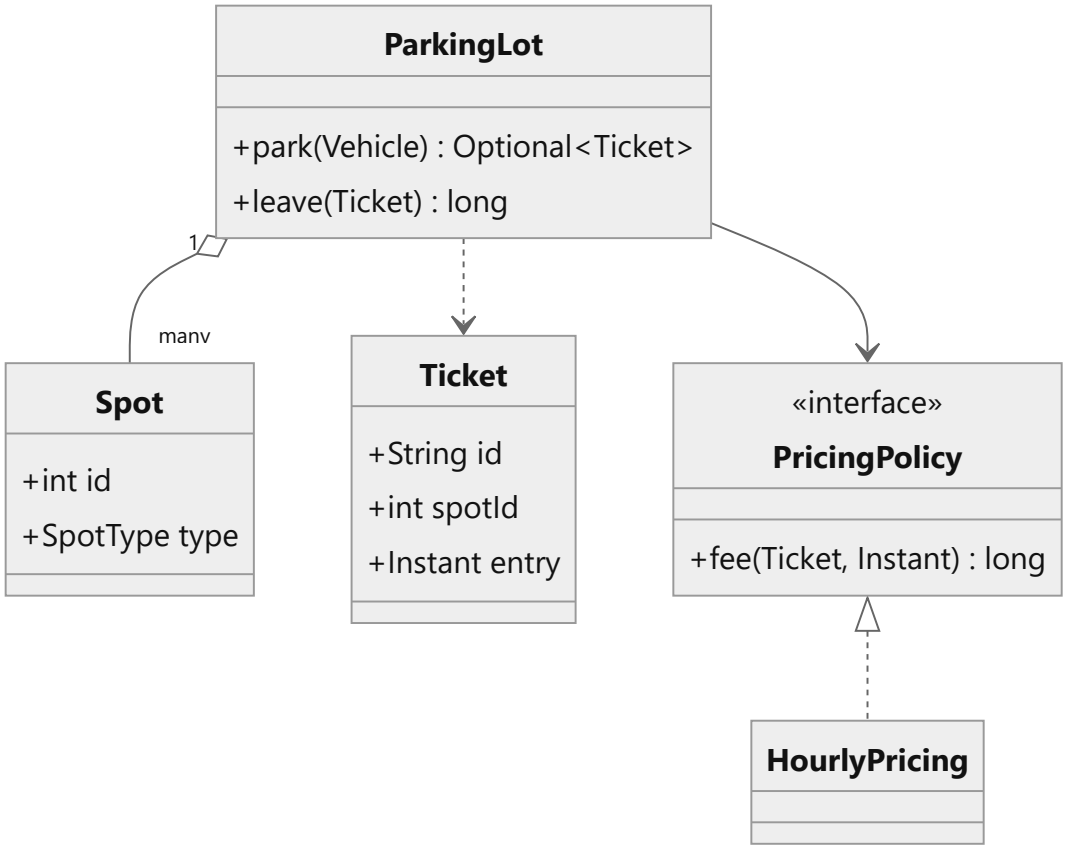
```
record Account(long id, AtomicLong balance) {}
void transfer(Account from, Account to, long amount) {
    Account first = from.id() < to.id() ? from : to;
    Account second = first == from ? to : from;
    synchronized (first) {
        synchronized (second) {
            from.balance().addAndGet(-amount);
            to.balance().addAndGet(amount);
        }
    }
}
```

36.14 Low-level design

Follow the same order each time: clarify requirements, list entities, assign responsibilities, draw classes, write the core flow, then discuss concurrency and extensions. Patterns should be a consequence of the design, not its starting point.

Q70. Design a parking lot.

Entities: lot, spot, vehicle, ticket, and a pricing policy. The concurrency risk is two cars taking one spot, so spots come from a concurrent queue per type. Pricing is a strategy, so it can change independently.




```
enum SpotType { BIKE, CAR, TRUCK }
record Vehicle(String plate, SpotType needs) {}
record Spot(int id, SpotType type) {}
record Ticket(String id, Spot spot, Instant entry) {}
interface PricingPolicy { long fee(Ticket ticket, Instant exit); }

class ParkingLot {
    private final Map<SpotType, Queue<Spot>> free = new EnumMap<>(SpotType.class);
    private final Map<String, Ticket> active = new ConcurrentHashMap<>();
    private final PricingPolicy pricing;
    private final Clock clock;

    ParkingLot(List<Spot> spots, PricingPolicy pricing, Clock clock) {
        this.pricing = pricing;
        this.clock = clock;
        for (SpotType t : SpotType.values()) free.put(t, new ConcurrentLinkedQueue<>());
        spots.forEach(s -> free.get(s.type()).add(s));
    }
    Optional<Ticket> park(Vehicle v) {
        Spot spot = free.get(v.needs()).poll();          // atomic: no two cars get one spot
        if (spot == null) return Optional.empty();
        Ticket t = new Ticket(UUID.randomUUID().toString(), spot, clock.instant());
        active.put(t.id(), t);
        return Optional.of(t);
    }
    long leave(String ticketId) {
        Ticket t = active.remove(ticketId);              // a second call finds nothing
        if (t == null) throw new IllegalArgumentException("Unknown or used ticket");
        free.get(t.spot().type()).add(t.spot());
        return pricing.fee(t, clock.instant());
    }
}
```

Extensions to mention: multiple floors (a spot-allocation strategy), larger vehicles using bigger spots, EV charging, persistence. Inject `Clock` so that pricing can be tested.

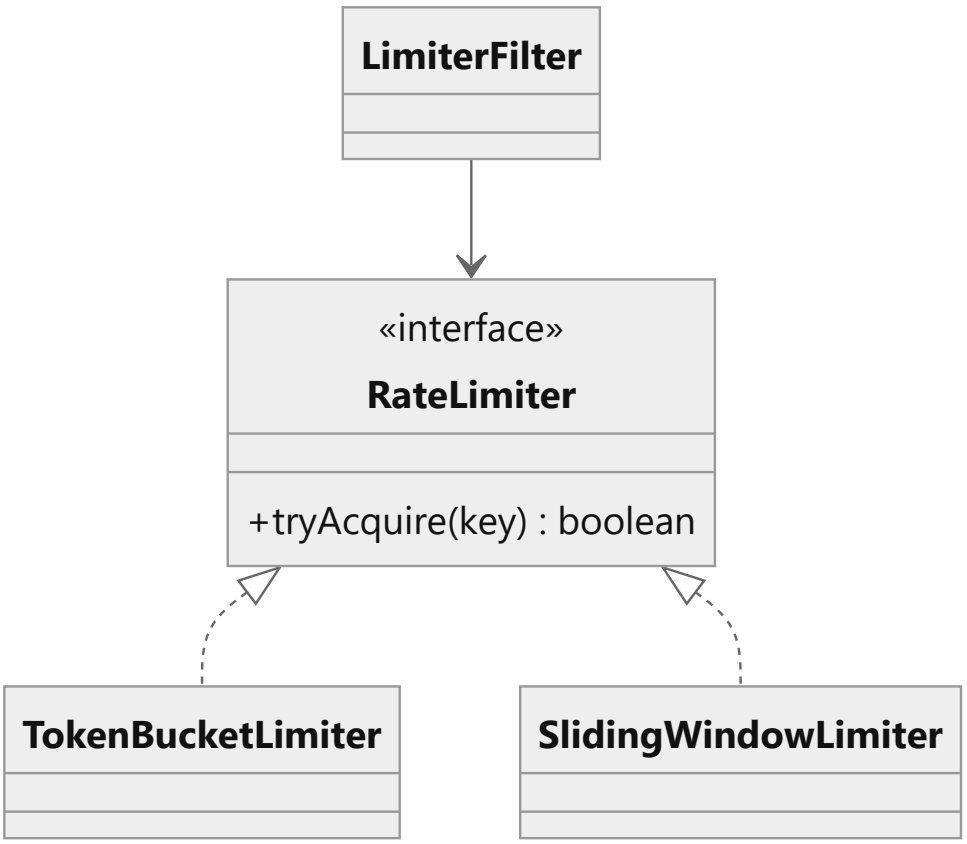
Q71. Design an LRU cache (LLD discussion).

Use Q37's structure behind a small `Cache<K,V>` interface. Discuss: the eviction policy as a strategy (LRU, LFU, TTL), thread safety (one lock vs segments), per-entry TTL, metrics such as hit rate, and loading on a miss with protection against a stampede. In production, say you'd use Caffeine, whose W-TinyLFU gives better hit rates than plain LRU.

Q72. Design a rate limiter.

Clarify: per what (user, key, IP)? Hard or soft limit? One node or many? Choose the algorithm:

Algorithm	Accuracy	Memory	Bursts	Notes
Fixed window	Low (double burst at the edges)	Tiny	Edge bursts	Simplest
Sliding log	Exact	One entry per request	Controlled	Memory-heavy
Sliding window counter	Good	Small	Smoothed	Good default
Token bucket	Good	Small	Allowed up to capacity	Most common for APIs
Leaky bucket	Good	Small	Smooths output	Good for protecting a downstream

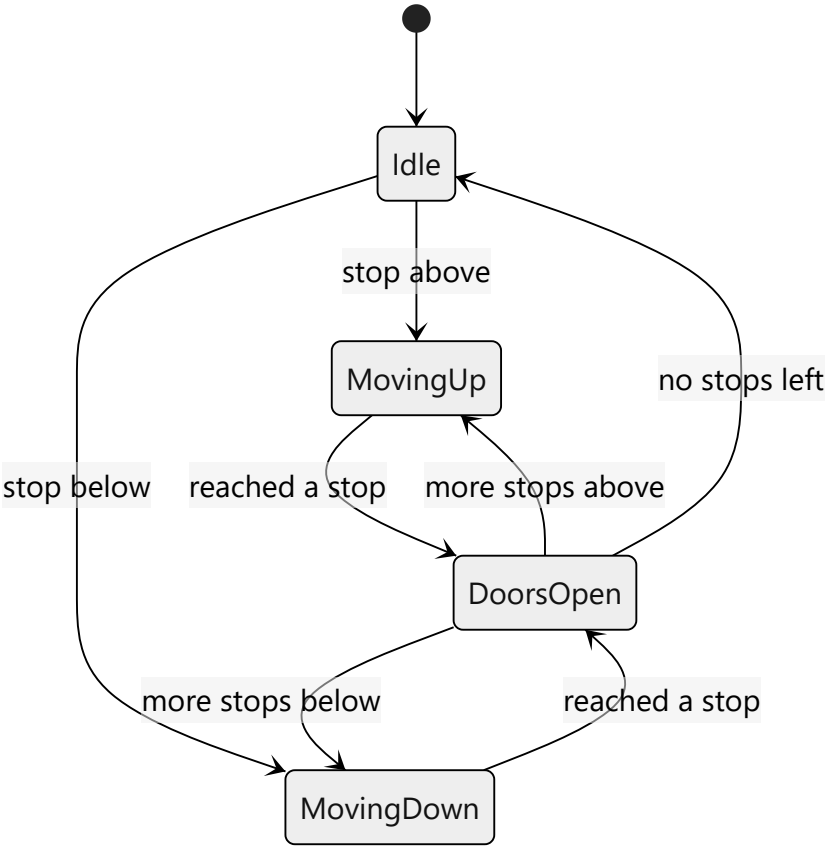


For several nodes, shared state moves to Redis with a Lua script, so that read-and-update is atomic. Decide whether to fail open or closed when Redis is down. Evict idle keys, or the map grows forever.

> **RefinePoint:** the gateway uses token buckets, the services use a `SlidingWindowLimiter` (a per-key `Deque<Long>` of timestamps). Both are in-memory and per instance, which the design accepts as a floor; a Redis-backed limiter is the known next step for several gateways.

Q73. Design an elevator (outline).

State the requirements (N elevators, M floors, hall and cabin calls), then separate **state** from **dispatch**. Each elevator uses the SCAN algorithm: continue in the current direction serving stops, then reverse. A `Dispatcher` strategy picks which elevator takes a hall call, for instance the nearest one already moving that way.




```
enum Direction { UP, DOWN, IDLE }
interface Dispatcher { int choose(List<ElevatorState> elevators, int floor, Direction direction);
}
record ElevatorState(int id, int floor, Direction direction, int stops) {}

class Elevator {
    private final TreeSet<Integer> up = new TreeSet<>(), down = new TreeSet<>();
    private int floor;
    private Direction direction = Direction.IDLE;

    synchronized void addStop(int target) {
        if (target >= floor) up.add(target); else down.add(target);
        if (direction == Direction.IDLE) direction = target >= floor ? Direction.UP :
Direction.DOWN;
    }
    synchronized Integer nextStop() {                    // SCAN: finish this direction, then
reverse
        if (direction == Direction.UP && up.isEmpty()) direction = down.isEmpty() ?
Direction.IDLE : Direction.DOWN;
        if (direction == Direction.DOWN && down.isEmpty()) direction = up.isEmpty() ?
Direction.IDLE : Direction.UP;
        return switch (direction) {
            case UP -> up.pollFirst();
            case DOWN -> down.pollLast();
            case IDLE -> null;
        };
    }
}
```

36.15 Java Streams coding questions

Assume these for all the questions below:

```
record Employee(String name, String dept, double salary, int age) {}
```

Q74. Group employees by department.

```
Map<String, List<Employee>> byDept(List<Employee> e) {
    return e.stream().collect(Collectors.groupingBy(Employee::dept));
}
```

Q75. Average salary per department.

```
Map<String, Double> avgByDept(List<Employee> e) {
    return e.stream().collect(Collectors.groupingBy(Employee::dept,
Collectors.averagingDouble(Employee::salary)));
}
```

Q76. Highest-paid employee in each department.

`toMap` with a merge function returns the employee directly. `groupingBy` + `maxBy` works too, but wraps each in an `Optional`.

```
Map<String, Employee> topByDept(List<Employee> e) {
    return e.stream().collect(Collectors.toMap(Employee::dept, Function.identity(),
        BinaryOperator.maxBy(Comparator.comparingDouble(Employee::salary))));
}
```

Q77. Second-highest distinct salary.

```
Optional<Double> secondHighest(List<Employee> e) {
    return e.stream().map(Employee::salary).distinct()
        .sorted(Comparator.reverseOrder()).skip(1).findFirst();
}
```

Q78. Word frequency, most common first.

```
List<Map.Entry<String, Long>> wordFreq(String text) {
    return Arrays.stream(text.toLowerCase().split("\\W+"))
        .filter(w -> !w.isBlank())
        .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
        .entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
        .toList();
}
```

Q79. First non-repeated character using streams.

`LinkedHashMap::new` keeps encounter order, which plain `groupingBy` doesn't.

```
Optional<Character> firstNonRepeated(String s) {
    return s.chars().mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new,
Collectors.counting()))
        .entrySet().stream()
        .filter(en -> en.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst();
}
```

Q80. Find the duplicates in a list.

`Set.add` returns false when the value is already there. This filter is stateful, so it isn't safe with `parallel()`. The grouping version is.

```
Set<Integer> duplicates(List<Integer> a) {
    Set<Integer> seen = new HashSet<>();
    return a.stream().filter(x -> !seen.add(x)).collect(Collectors.toSet());
}
```

Q81. Partition numbers into even and odd; sum the squares of the evens.

```
Map<Boolean, List<Integer>> evenOdd(List<Integer> a) {
    return a.stream().collect(Collectors.partitioningBy(x -> x % 2 == 0));
}
int sumEvenSquares(List<Integer> a) {
    return a.stream().filter(x -> x % 2 == 0).mapToInt(x -> x * x).sum();
}
```

Q82. Flatten a list of lists, keep distinct values, and sort.

```
List<Integer> flatten(List<List<Integer>> lists) {
    return lists.stream().flatMap(List::stream).distinct().sorted().toList();
}
```

Q83. Top 3 earners' names, as a comma-separated string.

```
String top3(List<Employee> e) {
    return e.stream().sorted(Comparator.comparingDouble(Employee::salary).reversed())
        .limit(3).map(Employee::name).collect(Collectors.joining(", "));
}
```

Q84. Count employees over 30 per department, and get salary statistics overall.

```
Map<String, Long> over30(List<Employee> e) {
    return e.stream().filter(x -> x.age() > 30)
        .collect(Collectors.groupingBy(Employee::dept, Collectors.counting()));
}
DoubleSummaryStatistics stats(List<Employee> e) {
    return e.stream().mapToDouble(Employee::salary).summaryStatistics();
}
```

Trap: `Collectors.toMap` throws `IllegalStateException` on a duplicate key unless you provide a merge function, and `NullPointerException` on a null value. `Stream.toList()` (Java 16+) returns an unmodifiable list; `Collectors.toList()` makes no such promise. `map(Employee::salary)` boxes every value, and `mapToDouble` avoids that. Don't use `parallel()` for small lists or anything with blocking I/O, because it shares the common fork-join pool.

37. Rapid-Fire Revision: 600 One-Line Answers

This is the night-before chapter. Each entry is a question in bold followed by the answer you would give in the first ten seconds. If an answer surprises you, go back to the chapter that covers the topic; do not try to learn new material from this list. Read it once from start to end, mark the ones you hesitate on, and read only those again in the morning.

Section	Topic	Entries
37.1–37.8	OOP, core Java, strings, collections, generics, Java 8–21, concurrency, JVM	1–204
37.9–37.18	Design patterns, Spring, data access, transactions, security, SQL	205–421
37.19–37.27	Databases, caching, microservices, Kafka, system design, DevOps, testing, AI, RAG and MCP	422–600

37.1 OOP and SOLID

1. **What are the four pillars of OOP?** Encapsulation, abstraction, inheritance and polymorphism.
2. **Encapsulation versus abstraction?** Encapsulation hides data to protect invariants; abstraction hides complexity behind a simpler contract.
3. **Is a class with getters and setters for every field encapsulated?** No; real encapsulation protects invariants through behaviour, not field access.
4. **Overloading versus overriding?** Overloading is resolved at compile time from static argument types; overriding is resolved at runtime from the object's actual class.
5. **How is runtime polymorphism implemented?** Through `invokevirtual` / `invokeinterface` and per-class method tables, usually optimised by the JIT with inline caches.
6. **Can you override a static method?** No; a same-signature static method hides the parent's, and the reference type decides which runs.
7. **Can you override a private or final method?** No; private methods are not inherited and final methods cannot be overridden.
8. **What is a covariant return type?** An override may return a subtype of the parent's return type, bridged by a compiler-generated method.
9. **Which overload does `print(null)` choose between `Object` and `String`?** The most specific applicable one, `String`.
10. **Overload resolution order?** Exact or widening first, then boxing and unboxing, then varargs.
11. **Abstract class versus interface?** Abstract classes hold state and constructors with single inheritance; interfaces define capabilities, allow multiple inheritance and no instance state.
12. **How does Java resolve the default-method diamond?** Class methods win, then the more specific interface, otherwise the class must override and may call `X.super.m()`.
13. **Why prefer composition over inheritance?** It couples to an interface rather than an implementation and avoids the fragile base class problem.
14. **What is the fragile base class problem?** A subclass breaks when the parent's undocumented internals change, as in `HashSet.addAll` calling `add`.
15. **What is the Liskov Substitution Principle?** A subtype must work wherever its parent is expected without stronger preconditions or weaker postconditions.

16. **Classic LSP violation?** `Square extends Rectangle`, where setting width and height independently breaks the square.
17. **Single Responsibility Principle?** A class should have one reason to change, meaning one actor who requests changes.
18. **Open/Closed Principle?** Add behaviour by adding code, such as a new strategy or plugin, rather than editing tested code.
19. **Interface Segregation Principle?** Clients should not depend on methods they do not use; split fat interfaces into role interfaces.
20. **Dependency Inversion versus Dependency Injection?** DIP is the principle that both layers depend on abstractions; DI is the technique of supplying dependencies from outside.
21. **What happens if a constructor calls an overridable method?** The override runs before the subclass's fields are initialised, often causing a `NullPointerException`.
22. **Order of initialisation in a hierarchy?** Parent then child static blocks once; then for each object, parent initialisers and constructor, then child's.
23. **How do you make a class immutable?** Make it final, fields private final, no setters, defensive copies in and out, and do not leak `this`.

37.2 Core Java language

24. **JDK, JRE and JVM?** The JVM runs bytecode, the JRE is the JVM plus libraries, and the JDK adds compiler and tools; since Java 11 there is no separate JRE download.
25. **Is Java pass-by-value or pass-by-reference?** Always pass-by-value; for objects, the value passed is a copy of the reference.
26. `==` **versus** `equals()`**?** `==` compares references for objects, while `equals()` compares logical equality as the class defines it.
27. **Why is** `Integer a = 127, b = 127; a == b` **true but false for 128?** `Integer.valueOf` caches -128 to 127, so only those share instances.
28. **The** `equals` **/** `hashCode` **contract?** Equal objects must have equal hash codes; breaking it makes hash-based collections lose entries.
29. **Size of** `int`**,** `long`**,** `char`**?** 32-bit, 64-bit and 16-bit unsigned UTF-16 code unit respectively.
30. **What does** `0.1 + 0.2 == 0.3` **give?** False, because binary floating point cannot represent 0.1 exactly; use `BigDecimal` for money.
31. `new BigDecimal(0.1)` **versus** `BigDecimal.valueOf(0.1)`**?** The constructor captures the inexact double; `valueOf` uses its canonical string form, `0.1`.
32. **Why does** `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` **return false?** `equals` compares scale too; use `compareTo` for numeric equality.
33. **Checked versus unchecked exceptions?** Checked exceptions extend `Exception` and must be declared or caught; unchecked ones extend `RuntimeException` or `Error`.
34. `final`**,** `finally`**,** `finalize`**?** A non-changeable declaration, a block that always runs after `try`, and a deprecated GC callback to avoid.
35. **When does** `finally` **not run?** On `System.exit`, a JVM crash or kill, or when the try block never finishes.
36. **What if both** `try` **and** `finally` **return?** The `finally` return wins and silently discards the try's value or exception.

37. **What is try-with-resources?** It closes `AutoCloseable` resources in reverse order and attaches close failures as suppressed exceptions.
38. **Static versus instance initialiser blocks?** Static blocks run once at class initialisation; instance blocks run before each constructor body.
39. **What does `transient` do?** It excludes a field from Java serialisation.
40. **What does `volatile` guarantee?** Visibility and ordering of reads and writes across threads, but not atomicity of compound actions like `i++`.
41. **What is `serialVersionUID`?** A version stamp checked on deserialisation; declare it, or harmless class changes break compatibility.
42. **Why is Java deserialisation dangerous?** Crafted byte streams can run gadget chains; never deserialise untrusted data, or use filters (`ObjectInputFilter`).
43. **What is autoboxing's main trap?** Unboxing a null `Integer` throws `NullPointerException`, and boxing in loops creates garbage.
44. **Static nested versus inner class?** An inner class holds a hidden reference to its outer instance, which can cause leaks; a static nested class does not.
45. **What is effectively final?** A local variable never reassigned after initialisation, which lambdas and inner classes may capture.
46. **What does `instanceof` with a pattern do?** `if (o instanceof String s)` tests and binds in one step (Java 16).

37.3 Strings

47. **Why is `String` immutable?** For pool sharing, security of validated values, thread safety and a cacheable hash code.
48. **What is the string pool?** A JVM table of interned strings on the heap, where literals are shared.
49. **`String s = "a"` versus `new String("a")`?** The literal reuses the pooled instance; `new` always creates another object.
50. **What does `intern()` do?** Returns the pooled instance equal to this string, adding it if absent.
51. **`String`, `StringBuilder`, `StringBuffer`?** Immutable; mutable and unsynchronised; mutable and synchronised, now rarely needed.
52. **Is `+` in a loop slow?** Yes, each iteration creates new objects; use a `StringBuilder` in loops.
53. **How is `a + b` compiled since Java 9?** Through `invokedynamic` and `StringConcatFactory`, not a hard-coded `StringBuilder`.
54. **What are compact strings?** Since Java 9, strings store Latin-1 text as one byte per character, using UTF-16 only when needed.
55. **Where did the pool move in Java 7?** From PermGen to the main heap.
56. **`"a" + "b" == "ab"`?** True, because the compiler folds constant expressions into one literal.
57. **`s1 + s2 == "ab"` with variables?** False, because runtime concatenation creates a new object.
58. **`trim()` versus `strip()`?** `trim()` removes characters up to U+0020; `strip()` (Java 11) removes all Unicode whitespace.
59. **`isEmpty()` versus `isBlank()`?** Empty means length zero; blank (Java 11) means empty or only whitespace.
60. **How do you compare strings ignoring case?** `equalsIgnoreCase`, or `compareToIgnoreCase` for ordering.

61. **Why is a `char[]` preferred for passwords?** It can be wiped after use, while a `String` stays in memory until collected.
62. **What does `String.split` take?** A regular expression, so `split(".")` needs `split("\\.")`.
63. **What are text blocks?** Multi-line `"""` literals with incidental indentation removed (Java 15).
64. **Is `String.hashCode()` cached?** Yes, computed once and stored in the object.
65. **What is a surrogate pair?** Two UTF-16 code units representing one code point beyond U+FFFF, so `length()` can exceed the number of characters.
66. **Why should `toUpperCase()` specify a locale?** Locale rules differ, such as Turkish dotless i; use `Locale.ROOT` for identifiers.
67. **What does `String.format` cost?** It parses the format each time, so avoid it in hot loops.
68. **What does `repeat(n)` do?** Returns the string repeated n times (Java 11).

37.4 Collections

69. **`ArrayList` versus `LinkedList`?** `ArrayList` has O(1) index access and cache locality; `LinkedList` is rarely faster in practice.
70. **How does `ArrayList` grow?** By about 1.5 times (old + old >> 1) when full, copying into a new array.
71. **How does `HashMap.put` work?** It hashes the key, spreads the high bits, picks a bucket with `(n-1) & hash`, and compares keys with `equals` there.
72. **When does a `HashMap` bucket become a tree?** At 8 entries with a table of at least 64 buckets; otherwise the table resizes instead.
73. **Default capacity and load factor of `HashMap`?** 16 buckets and 0.75, resizing when the size exceeds 12.
74. **Why must the `HashMap` capacity be a power of two?** So `(n-1) & hash` replaces a modulo and a resize splits buckets cleanly.
75. **Does `HashMap` allow null keys?** One null key, stored in bucket 0, and any number of null values.
76. **Does `ConcurrentHashMap` allow nulls?** No, because null would make `get` ambiguous between absent and present-with-null.
77. **How does Java 8 `ConcurrentHashMap` lock?** CAS for empty bins and `synchronized` on the first node of a bin, with no segments.
78. **What happens if you mutate a key after putting it in a `HashMap`?** Its hash changes, so lookups go to the wrong bucket and the entry is effectively lost.
79. **Fail-fast versus fail-safe iterators?** Fail-fast iterators throw `ConcurrentModificationException` on change; concurrent collections iterate a snapshot or weakly consistent view.
80. **How do you remove items while iterating?** Use `Iterator.remove()` or `removeIf`, never `list.remove` inside a for-each.
81. **`HashMap`, `LinkedHashMap`, `TreeMap` ordering?** None guaranteed, insertion or access order, and sorted by key.
82. **What does `TreeMap` use internally?** A red-black tree with O(log n) operations.
83. **What does `compareTo` consistent with `equals` mean?** If `compareTo` returns zero only when `equals` is true; otherwise a `TreeSet` may drop elements a `HashSet` keeps.
84. **`Comparable` versus `Comparator`?** `Comparable` is the natural order inside the class; `Comparator` is an external order you pass in.
85. **Which structure for a LIFO stack?** `ArrayDeque`; `Stack` is synchronised legacy code built on `Vector`.

86. **Is `PriorityQueue` sorted when iterated?** No, only the head is guaranteed smallest; poll repeatedly to get sorted order.
87. **`List.of` versus `Arrays.asList`?** `List.of` is fully immutable and rejects nulls; `Arrays.asList` is a fixed-size view that allows `set`.
88. **What is `Collections.unmodifiableList`?** A read-only view; changes to the underlying list still show through.
89. **`CopyOnWriteArrayList` is good for?** Rarely written, often read lists such as listeners, since every write copies the array.
90. **What does `WeakHashMap` do?** Removes entries whose keys are no longer strongly referenced elsewhere.
91. **What is `IdentityHashMap`?** A map comparing keys with `==` rather than `equals`.
92. **What is `EnumMap`?** A compact array-backed map for enum keys, faster than `HashMap`.
93. **What are sequenced collections?** Java 21 interfaces adding `getFirst`, `getLast` and `reversed` to lists, deques and linked sets and maps.
94. **How do you find a `ConcurrentHashMap`'s size accurately?** `mappingCount()`, which is still only an estimate under concurrent updates.

37.5 Generics

95. **What is type erasure?** Generic types are checked at compile time and replaced by their bounds in bytecode, so `List<String>` is `List` at runtime.
96. **Why can't you write `new T()`?** The type is erased, so pass a `Class<T>` or a `Supplier<T>`.
97. **Why can't you create a generic array `new T[10]`?** Arrays check element types at runtime, which erased generics cannot support safely.
98. **Is `List<Integer>` a subtype of `List<Number>`?** No, generics are invariant, though arrays are covariant.
99. **What does `? extends T` mean?** A producer you can read `T` from but cannot add to, except null.
00. **What does `? super T` mean?** A consumer you can add `T` to, reading only `Object`.
01. **What is PECS?** Producer Extends, Consumer Super, as in `Collections.copy(List<? super T> dst, List<? extends T> src)`.
02. **What is a bridge method?** A synthetic method the compiler generates so overriding still works after erasure.
03. **Can you overload `m(List<String>)` and `m(List<Integer>)`?** No, they have the same erasure.
04. **What is a raw type?** A generic type used without arguments, losing type safety and producing unchecked warnings.
05. **What is heap pollution?** A variable of a parameterised type referring to an object of another parameterisation, often from varargs or raw types.
06. **What does `@SafeVarargs` do?** Asserts that a generic varargs method does not pollute the heap, silencing the warning.
07. **What is a bounded type parameter?** `<T extends Comparable<T>>` restricts T to types with that capability.
08. **What is a recursive generic bound?** `<T extends Comparable<? super T>>`, which allows comparison inherited from a supertype.
09. **Can a static field use the class's type parameter?** No, statics belong to the class shared by all parameterisations.

10. **How do frameworks read generic types at runtime?** From signatures kept in class metadata, such as a super type token `new TypeReference<List<X>>() {}`.
11. **Does `instanceof List<String>` compile?** No; you can only test `List<?>` or a reifiable type.
12. **What is a wildcard capture?** The compiler naming an unknown `?` type internally, often fixed with a private generic helper method.
13. **Generics versus `Object` with casts?** Generics move type errors from runtime `ClassCastException` to compile time.
14. **Why does `List<?>` differ from `List<Object>`?** `List<?>` accepts any list but you cannot add to it; `List<Object>` accepts only lists of `Object`.
15. **Can enums or records be generic?** Records yes, enums no.
16. **What is a generic method?** A method declaring its own type parameters, `<T> T first(List<T> list)`.

37.6 Java 8 to 21 features

17. **What is a functional interface?** An interface with exactly one abstract method, usable as a lambda target.
18. **How are lambdas compiled?** To a private method plus an `invokedynamic` call bootstrapped by `LambdaMetafactory`, not an anonymous class file.
19. **Four core functional interfaces?** `Function`, `Consumer`, `Supplier` and `Predicate`.
20. **Intermediate versus terminal stream operations?** Intermediate ones are lazy and return a stream; a terminal one triggers processing.
21. **Can you reuse a stream?** No; a second terminal operation throws `IllegalStateException`.
22. **`map` versus `flatMap`?** `map` transforms each element one-to-one; `flatMap` flattens each result stream into one.
23. **`findFirst` versus `findAny`?** `findAny` may return any element and is cheaper in parallel streams.
24. **When is `parallel()` a bad idea?** For small data, blocking I/O, ordered or stateful work, since it uses the shared fork-join pool.
25. **What is `Optional` for?** A return type that signals "may be absent"; not for fields, parameters or collections.
26. **`orElse` versus `orElseGet`?** `orElse` always evaluates its argument; `orElseGet` calls the supplier only when empty.
27. **What does `Collectors.groupingBy` return?** A `Map` of key to list by default, with downstream collectors for counts, sums or other aggregation.
28. **What does `Stream.toList()` return?** An unmodifiable list (Java 16).
29. **What changed in Java 8 date-time?** `java.time` gave immutable, thread-safe `LocalDate`, `Instant`, `ZonedDateTime` and `Duration`.
30. **Which `java.time` type stores a moment on the timeline?** `Instant` (UTC); `LocalDateTime` has no zone and is not a moment.
31. **What does `var` do?** Local type inference (Java 10); the static type is still fixed at compile time.
32. **What is a record?** A transparent, immutable data carrier with generated constructor, accessors, `equals`, `hashCode` and `toString` (Java 16).
33. **Can a record have a compact constructor?** Yes, to validate or normalise components, such as defensive copies.

34. **What are sealed classes?** Types that list their permitted subclasses, enabling exhaustive pattern matching (Java 17).
35. **What do switch expressions add?** Arrow labels, no fall-through, `yield` and a value result (Java 14).
36. **What is pattern matching for switch?** `case Circle c ->` with type and record patterns plus `when` guards, final in Java 21.
37. **What are record patterns?** Deconstruction in patterns, such as `case Point(int x, int y) ->` (Java 21).
38. **What are virtual threads?** Lightweight JVM-managed threads mounted on carrier threads, final in Java 21.
39. **What is pinning?** A virtual thread blocking inside `synchronized` or native code cannot unmount its carrier (fixed for `synchronized` in Java 24).
40. **Should you pool virtual threads?** No; create one per task and limit concurrency with semaphores instead.
41. **What did the Java 11 `HttpClient` add?** A standard HTTP/1.1 and HTTP/2 client with synchronous and asynchronous APIs.
42. **What is the module system?** Java 9 JPMS: `module-info.java` declares what a module requires and exports, giving strong encapsulation.
43. **What are helpful NullPointerExceptions?** Messages naming exactly which expression was null (Java 14, default since 15).
44. **What does `List.copyOf` do?** Returns an unmodifiable copy, or the same list if it is already unmodifiable (Java 10).
45. **What is structured concurrency?** A preview API treating a group of subtasks as one unit that succeeds, fails or is cancelled together.
46. **What are scoped values?** A preview, immutable, bounded alternative to `ThreadLocal` suited to virtual threads.
47. **What did Java 21 add to collections?** Sequenced collections.

37.7 Concurrency

48. **Process versus thread?** A process has its own memory; threads share the process's heap but each has its own stack.
49. **Thread states?** NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING and TERMINATED.
50. **`start()` versus `run()`?** `start()` creates a new thread that calls `run()`; calling `run()` executes it on the current thread.
51. **What is a race condition?** A result that depends on the timing of unsynchronised access to shared state.
52. **What does `synchronized` provide?** Mutual exclusion plus happens-before visibility on the same monitor.
53. **What is happens-before?** A JMM guarantee that one action's effects are visible to another, such as unlock before a later lock of the same monitor.
54. **`sleep()` versus `wait()`?** `sleep` keeps locks and is a static thread method; `wait` releases the monitor and needs `notify`.
55. **Why call `wait()` in a loop?** Spurious wakeups and competing threads mean the condition must be checked again.
56. **`notify` versus `notifyAll`?** `notify` wakes one waiter, which risks the wrong one; `notifyAll` wakes all to recheck.
57. **What is a deadlock?** Threads each holding a lock while waiting for another's, in a cycle.
58. **How do you prevent deadlock?** A global lock order, `tryLock` with timeouts, or fewer and smaller locks.

59. **Livelock versus starvation?** Livelock: threads keep reacting without progress; starvation: a thread never gets the resource.
60. `ReentrantLock` **over** `synchronized` **gives?** `tryLock`, timeouts, interruptible locking, fairness and multiple conditions.
61. **What is a** `ReadWriteLock`**?** Many readers or one writer, useful when reads dominate.
62. **What is** `StampedLock`**?** A lock with optimistic reads validated afterwards; not reentrant.
63. **How do atomic classes work?** With compare-and-swap CPU instructions, without locks.
64. **What is the ABA problem?** A CAS succeeds although the value changed and changed back; fix with a stamp (`AtomicStampedReference`).
65. `AtomicLong` **versus** `LongAdder`**?** `LongAdder` spreads contention over cells and is faster for frequent increments with rare reads.
66. **Why use** `ExecutorService` **rather than** `new Thread`**?** It reuses threads, bounds concurrency, queues work and manages shutdown.
67. **What are** `ThreadPoolExecutor`**'s key parameters?** Core size, maximum size, keep-alive, work queue, thread factory and rejection policy.
68. **When does the pool grow past core size?** Only when the queue is full, so with an unbounded queue it never grows.
69. **Why is** `Executors.newFixedThreadPool` **risky?** Its unbounded queue can grow until the heap runs out.
70. **What does** `CallerRunsPolicy` **do?** Runs a rejected task on the submitting thread, slowing the producer as backpressure.
71. `Future` **versus** `CompletableFuture`**?** `Future` only blocks with `get`; `CompletableFuture` composes stages, combines results and handles errors.
72. **Which pool does** `CompletableFuture.supplyAsync` **use by default?** The common fork-join pool, unsuitable for blocking I/O; pass your own executor.
73. `thenApply` **versus** `thenCompose`**?** `thenApply` maps a value; `thenCompose` chains a function that returns another future.
74. `CountDownLatch` **versus** `CyclicBarrier`**?** A latch is a one-time countdown; a barrier lets a fixed group meet repeatedly.
75. **What is a** `Semaphore` **for?** Limiting how many threads use a resource at once.
76. **What does** `ThreadLocal` **risk in pools?** Values leak between tasks and cause memory leaks unless removed in `finally`.
77. **How do you stop a thread correctly?** Interrupt it and have it check `isInterrupted` or handle `InterruptedException`.
78. **What should you do when catching** `InterruptedException`**?** Restore the flag with `Thread.currentThread().interrupt()` or rethrow.
79. **What is false sharing?** Different threads updating variables on the same cache line, slowing both; fixed by padding (`@Contended`).
80. **What does a** `ForkJoinPool` **use?** Work stealing: idle threads take tasks from the tails of others' deques.

37.8 JVM and garbage collection

81. **Main JVM memory areas?** Heap, metaspace, per-thread stacks, code cache and direct (native) memory.
82. **What replaced PermGen?** Metaspace (Java 8), in native memory and sized by `MaxMetaspaceSize`.

83. **Class loading phases?** Loading, linking (verify, prepare, resolve) and initialisation.
84. **What is parent delegation?** A class loader asks its parent first, so core classes cannot be replaced by application jars.
85. **Which class loaders exist?** Bootstrap, platform (formerly extension) and application, plus any custom ones.
86. **What is the generational hypothesis?** Most objects die young, so collecting the young generation often is cheap.
87. **What is a TLAB?** A thread-local allocation buffer in Eden making allocation a pointer bump.
88. **Minor versus major versus full GC?** Minor collects the young generation, major the old, and full collects everything, usually stopping the world longer.
89. **Default GC since Java 9?** G1.
90. **How does G1 work?** It divides the heap into regions and collects those with the most garbage first, aiming at a pause target.
91. **When choose ZGC?** For large heaps needing pauses under a millisecond; a generational mode arrived in Java 21 and became the default in Java 23.
92. **What is a stop-the-world pause?** A point where all application threads halt so the collector can work safely.
93. **What are GC roots?** Thread stacks, static fields, JNI references and other live entry points from which reachability is traced.
94. **Can Java leak memory with GC?** Yes, through references kept unintentionally: static caches, listeners, `ThreadLocal`s, unbounded maps.
95. **Strong, soft, weak, phantom references?** Never collected while reachable; collected under memory pressure; collected at next GC; used for cleanup after collection.
96. **How do you diagnose a memory leak?** Take heap dumps over time and compare the dominator trees in Eclipse MAT.
97. **What causes `OutOfMemoryError: Metaspace`?** Too many loaded classes, often a class loader leak during redeploys.
98. **What causes `StackOverflowError`?** Too-deep recursion; raise `-Xss` only after fixing the recursion.
99. **Why set `-Xms` equal to `-Xmx` in servers?** To avoid heap resizing pauses and to claim memory up front.
00. **How does the JVM size the heap in containers?** From the container's memory limit, by default a quarter of it; tune with `MaxRAMPercentage`.
01. **What is JIT compilation?** Hot bytecode is compiled to native code by tiered C1 and C2 compilers, using runtime profiles.
02. **What is escape analysis?** The JIT proving an object never leaves a method, so it can avoid heap allocation or locking.
03. **Which tools capture a thread dump?** `jstack`, `jcmd <pid> Thread.print` or `kill -3`.
04. **What is JFR?** Java Flight Recorder, a low-overhead built-in profiler for production use.

37.9 Design patterns

05. **Three GoF pattern families?** Creational, structural and behavioural.
06. **Singleton's best Java implementation?** An enum, which handles serialisation and reflection; in Spring, simply a singleton-scoped bean.

07. **Why is a hand-written singleton often an anti-pattern?** It is global mutable state that hides dependencies and is hard to test.
08. **Factory method versus abstract factory?** A factory method creates one product via subclass or method; an abstract factory creates families of related products.
09. **When use a builder?** For objects with many optional parameters, giving readable, immutable construction.
10. **What is the strategy pattern?** Interchangeable algorithms behind one interface, chosen at runtime, such as match strategies.
11. **How does Spring make strategy easy?** Inject `Map<String, Strategy>` or `List<Strategy>` and pick by key.
12. **Decorator versus proxy?** A decorator adds behaviour the client asked for; a proxy controls access, such as lazy loading, security or transactions.
13. **Adapter pattern?** Converts one interface into another a client expects, such as wrapping a vendor SDK behind `LlmClient`.
14. **Facade pattern?** A simple entry point over a complex subsystem.
15. **Observer pattern in Spring?** `ApplicationEventPublisher` with `@EventListener` or `@TransactionalEventListener`.
16. **Template method pattern?** A base class fixes an algorithm's steps and subclasses fill some in, as in `JdbcTemplate` callbacks.
17. **Chain of responsibility?** Handlers pass a request along a chain, as in servlet filters and Spring Security.
18. **Command pattern?** Encapsulates a request as an object so it can be queued, logged or undone.
19. **State pattern?** An object changes behaviour with its state by delegating to state objects.
20. **Composite pattern?** Treats individual objects and groups uniformly, as in tree structures.
21. **Prototype pattern?** Creates objects by copying an existing instance.
22. **Flyweight pattern?** Shares immutable fine-grained objects, as with the `Integer` cache.
23. **Visitor pattern?** Adds operations over a type hierarchy without changing it; sealed types with switch patterns often replace it in Java 21.
24. **Circuit breaker pattern?** Stops calling a failing dependency for a while, then tests with trial calls before closing again.
25. **Which patterns does Spring itself use most?** Factory (`BeanFactory`), singleton, proxy (AOP), template, observer and strategy.
26. **What is dependency injection as a pattern?** Objects receive collaborators from outside instead of creating or looking them up.

37.10 Spring core

27. **What is IoC?** The framework creates and wires objects and calls your code, rather than your code controlling everything.
28. **`BeanFactory` versus `ApplicationContext`?** `ApplicationContext` adds eager singleton creation, events, internationalisation, AOP integration and environment handling.
29. **Constructor versus field injection?** Constructor injection allows `final` fields and makes dependencies explicit and testable; prefer it.
30. **Is `@Autowired` needed on a single constructor?** No, not since Spring 4.3.
31. **Default bean scope?** Singleton, one instance per application context.

32. **Other scopes?** Prototype, request, session, application and websocket.
33. **What happens when a singleton injects a prototype?** It gets one prototype instance forever; use `ObjectProvider` or lookup methods for a fresh one.
34. **How does Spring resolve two beans of the same type?** By `@Primary`, then `@Qualifier`, then parameter name, otherwise `NoUniqueBeanDefinitionException`.
35. **Bean lifecycle order?** Instantiate, populate properties, aware callbacks, `postProcessBeforeInitialization`, `@PostConstruct`, `afterPropertiesSet`, init-method, `postProcessAfterInitialization`, then destroy callbacks.
36. **Where are AOP proxies created?** In `BeanPostProcessor.postProcessAfterInitialization`.
37. **`BeanPostProcessor` versus `BeanFactoryPostProcessor`?** The first changes bean instances; the second changes bean definitions before any bean is created.
38. **How does Spring handle circular dependencies?** Setter or field cycles of singletons via early references; Spring Boot 2.6+ forbids cycles by default, and constructor cycles always fail.
39. **`@Component`, `@Service`, `@Repository`, `@Controller`?** All register beans; `@Repository` also translates persistence exceptions and `@Controller` marks web handlers.
40. **`@Configuration` versus `@Component` with `@Bean` methods?** `@Configuration` classes are CGLIB-proxied, so calling one `@Bean` method from another returns the same singleton.
41. **What does `proxyBeanMethods = false` do?** Skips that proxy for faster startup, accepting that inter-bean calls create new objects.
42. **What is `@Lazy`?** Creates the bean on first use, or injects a lazy proxy.
43. **What is `@Profile`?** Registers beans only when that profile is active.
44. **What is the `Environment`?** Spring's abstraction over profiles and ordered property sources.
45. **`@Value` versus `@ConfigurationProperties`?** `@Value` injects one value; `@ConfigurationProperties` binds a typed, validated group of properties.
46. **What are Spring events?** In-process publish and subscribe via `ApplicationEventPublisher`, synchronous by default.
47. **What does `@TransactionalEventListener` do?** Runs the listener at a transaction phase, by default after commit.
48. **What does `@DependsOn` do?** Forces another bean to initialise first.

37.11 Spring AOP and proxies

49. **What is AOP?** Separating cross-cutting concerns such as transactions, security and logging into aspects.
50. **Aspect, advice, pointcut, join point?** The module, the action, the matching expression, and the point in execution, which in Spring is always a method call.
51. **Advice types?** `@Before`, `@AfterReturning`, `@AfterThrowing`, `@After` and `@Around`.
52. **How does Spring AOP work?** With runtime proxies wrapping the bean, not bytecode weaving.
53. **JDK dynamic proxy versus CGLIB?** JDK proxies implement interfaces; CGLIB subclasses the class; Boot uses CGLIB by default.
54. **Why does self-invocation skip `@Transactional`?** `this.method()` calls the target directly, bypassing the proxy.
55. **How do you fix self-invocation?** Move the method into another bean, or inject the bean's own proxy.
56. **Can AOP advise private or final methods?** No; proxies only intercept public, overridable methods.

57. **Can CGLIB proxy a final class?** No, because it must subclass it.
58. **Spring AOP versus AspectJ?** Spring AOP is proxy-based and method-only; AspectJ weaves bytecode and can advise fields, constructors and self-calls.
59. **What must an `@Around` advice do?** Call `proceed()` and return its result, or the target never runs.
60. **How do you order aspects?** With `@Order` or `Ordered`; the lowest value is outermost.
61. **Which runs first, `@PreAuthorize` or `@Transactional`?** With `@EnableMethodSecurity`, the authorisation interceptors are ordered ahead of the transaction interceptor, so access is checked before a transaction starts.
62. **Pointcut for all methods of a package?** `execution(* com.acme.service..*(..))`.
63. **Pointcut for an annotation?** `@annotation(com.acme.Audited)`.
64. **What is `@EnableAspectJAutoProxy`?** Turns on detection of `@Aspect` beans; Boot enables it automatically.
65. **Is the proxy's class equal to your class?** No; `getClass()` returns the proxy, so use `AopUtils.getTargetClass`.
66. **How do you see whether a bean is proxied?** `AopUtils.isAopProxy(bean)`.
67. **Which annotations work through proxies?** `@Transactional`, `@Cacheable`, `@Async`, `@PreAuthorize`, `@Retryable` and `@Validated`.
68. **Why does `@Async` on a method called internally run synchronously?** The same self-invocation bypass of the proxy.
69. **Cost of AOP?** A small per-call overhead and confusing stack traces, so keep aspects few and cheap.

37.12 Spring Boot

70. **What does `@SpringBootApplication` combine?** `@SpringBootConfiguration`, `@EnableAutoConfiguration` and `@ComponentScan`.
71. **How does auto-configuration find classes?** Through `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` since Boot 2.7.
72. **Key conditional annotations?** `@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`, `@ConditionalOnBean` and `@ConditionalOnWebApplication`.
73. **Why `@ConditionalOnMissingBean`?** So your own bean replaces the default.
74. **How do you see which auto-configurations applied?** Run with `--debug` for the conditions report, or `/actuator/conditions`.
75. **How do you exclude an auto-configuration?** `@SpringBootApplication(exclude = X.class)` or `spring.autoconfigure.exclude`.
76. **What is a starter?** A dependency bundle that brings libraries and, through them, their auto-configuration.
77. **Property source precedence, highest first?** Command-line arguments, system properties, environment variables, then profile-specific and plain `application` files.
78. **How do environment variables map to properties?** Relaxed binding: `SPRING_DATASOURCE_URL` maps to `spring.datasource.url`.
79. **What is an `EnvironmentPostProcessor`?** A hook that edits the environment before the context starts, such as adding default properties.
80. **What is a fat jar?** An executable jar with nested dependency jars loaded by Boot's own launcher.
81. **What does Actuator provide?** Health, info, metrics, Prometheus, environment, thread dump and similar endpoints.

82. **Liveness versus readiness probes?** Liveness says restart me if broken; readiness says send me traffic.
83. **How do you shut down gracefully?** `server.shutdown=graceful` with a timeout per phase, so in-flight requests finish.
84. **What is `CommandLineRunner`?** Code run once after the context starts.
85. **Default embedded server?** Tomcat for MVC, Netty for WebFlux.
86. **What is the Boot 3 baseline?** Java 17 and Jakarta EE 9+, so `javax.*` became `jakarta.*`.
87. **What replaced Sleuth in Boot 3?** Micrometer Tracing with the Observation API.
88. **How does Boot enable virtual threads?** `spring.threads.virtual.enabled=true` (Boot 3.2+ on Java 21).
89. **What is `spring.jpa.open-in-view`?** On by default, keeping a session open through the view; turn it off to avoid hidden queries.
90. **What does `ddl-auto=validate` do?** Checks the entity mapping against the schema at startup without changing it.
91. **What is a `FailureAnalyzer`?** It turns startup exceptions into readable explanations.

37.13 Spring MVC and REST

92. **What does the `DispatcherServlet` do?** It finds a handler, runs interceptors, invokes the controller, and resolves the response.
93. **`@Controller` versus `@RestController`?** `@RestController` adds `@ResponseBody`, writing return values to the body.
94. **How is JSON written?** By an `HttpMessageConverter`, usually Jackson's.
95. **Filter versus interceptor?** Filters are servlet-level and see every request; interceptors are Spring MVC-level and know the handler.
96. **How do you handle exceptions globally?** `@RestControllerAdvice` with `@ExceptionHandler` methods.
97. **What is `ProblemDetail`?** RFC 7807 (now 9457) error bodies, supported natively since Spring 6.
98. **`@PathVariable`, `@RequestParam`, `@RequestBody`?** URL segment, query or form parameter, and deserialised body.
99. **How do you validate input?** `@Valid` on the parameter with Bean Validation annotations; failures throw `MethodArgumentNotValidException`.
00. **Which HTTP methods are idempotent?** GET, PUT, DELETE, HEAD and OPTIONS; POST is not, and PATCH not necessarily.
01. **PUT versus PATCH?** PUT replaces the whole resource; PATCH applies a partial change.
02. **201, 202, 204?** Created with a `Location`, accepted for later processing, and success with no body.
03. **401 versus 403?** Not authenticated versus authenticated but not allowed.
04. **409, 422, 429?** Conflict with current state, unprocessable valid syntax, and too many requests.
05. **How do you version an API?** URL path (`/v1`), header or media type; path versioning is simplest to operate.
06. **Offset versus cursor pagination?** Offset is simple but slow and unstable for deep pages; cursor (keyset) is fast and stable.
07. **How do you make POST safe to retry?** An `Idempotency-Key` header stored with the result.
08. **What is HATEOAS?** Responses include links to related actions; rarely used in practice.
09. **How does CORS work?** The browser sends a preflight `OPTIONS`; the server answers with allowed origins, methods and headers.

10. **What is ETag and `If-None-Match`?** A version token that lets the server answer 304 Not Modified.
11. **`RestTemplate`, `WebClient`, `RestClient`?** `RestTemplate` is in maintenance; `RestClient` (Spring 6.1) is the modern synchronous client; `WebClient` is reactive.
12. **How do you stream a large download?** Return `StreamingResponseBody` or a `Resource` instead of building it in memory.
13. **What is content negotiation?** Choosing the response format from the `Accept` header.

37.14 WebFlux and reactive

14. **`Mono` versus `Flux`?** Zero or one element versus zero to many.
15. **What is backpressure?** A subscriber signalling how much it can take, so producers do not overwhelm it.
16. **When is WebFlux worth it?** Many concurrent slow I/O calls with a fully non-blocking stack; virtual threads reduce the need.
17. **What happens if you block in a reactive pipeline?** You stall an event-loop thread and the throughput of every request on it.
18. **How do you call blocking code from Reactor?** `subscribeOn(Schedulers.boundedElastic())`.
19. **`map` versus `flatMap` in Reactor?** `map` is synchronous transformation; `flatMap` subscribes to an inner publisher, possibly concurrently.
20. **`flatMap` versus `concatMap`?** `flatMap` interleaves results; `concatMap` keeps order by processing one at a time.
21. **What is cold versus hot?** Cold publishers start afresh per subscriber; hot ones share one ongoing stream.
22. **Does nothing happen until subscribe?** Correct; assembling a pipeline does no work until a subscription.
23. **`publishOn` versus `subscribeOn`?** `publishOn` moves downstream operators to a scheduler; `subscribeOn` decides where the source runs.
24. **How do you handle errors?** `onErrorResume`, `onErrorReturn`, `retryWhen` with backoff, and `timeout`.
25. **Why doesn't `ThreadLocal` work in reactive code?** Execution hops threads, so use the Reactor `Context`.
26. **What server does WebFlux use?** Netty by default, with a small number of event-loop threads.
27. **Is JDBC usable in WebFlux?** Only by offloading it; use R2DBC for a non-blocking database driver.
28. **How do you test reactive code?** `StepVerifier` and `WebTestClient`.
29. **What is Spring Cloud Gateway built on?** WebFlux and Netty, so gateway filters must not block.
30. **What does `zip` do?** Combines the first elements of several publishers, then the second ones and so on, into tuples; it completes when the shortest source does.
31. **What is `Sinks`?** Reactor's API for pushing values into a `Flux` or `Mono` programmatically.
32. **Reactive versus virtual threads?** Both handle high I/O concurrency; virtual threads keep simple blocking code, reactive adds streaming and backpressure.
33. **What does `block()` do inside WebFlux handlers?** Throws `IllegalStateException` on Netty threads, since blocking is not allowed.

37.15 JPA and Hibernate

34. **JPA versus Hibernate?** JPA is the specification; Hibernate is the most common implementation.
35. **Entity states?** Transient, managed, detached and removed.

36. **What is the persistence context?** The first-level cache of managed entities for one session, guaranteeing one object per row.
37. **What is dirty checking?** At flush, Hibernate compares managed entities with snapshots and issues updates.
38. `persist` **versus** `merge`**?** `persist` makes a new instance managed; `merge` copies a detached state onto a managed copy and returns it.
39. **When does Hibernate flush?** Before commit, before queries that touch affected tables (AUTO mode), or on `flush()`.
40. **Default fetch types?** `@ManyToOne` and `@OneToOne` are eager; `@OneToMany` and `@ManyToMany` are lazy.
41. **Why make all associations lazy?** Eager fetching loads unexpected graphs and causes N+1 queries.
42. **What is the N+1 problem?** One query for parents and one more per parent for children.
43. **How do you fix N+1?** `JOIN FETCH`, `@EntityGraph`, batch fetching (`@BatchSize`) or a DTO projection.
44. **What causes** `LazyInitializationException`**?** Touching a lazy association after the session closed.
45. **Why not fetch-join two collections?** It creates a Cartesian product and `MultipleBagFetchException` for lists.
46. `GenerationType.IDENTITY` **drawback?** It disables JDBC insert batching, since the id is known only after each insert.
47. **Which id generation batches best?** `SEQUENCE` with an allocation size, or application-assigned UUIDs.
48. **What is optimistic locking?** A `@Version` column checked on update, throwing on a concurrent change.
49. **What is pessimistic locking?** `SELECT ... FOR UPDATE` via `LockModeType.PESSIMISTIC_WRITE`.
50. **What is the second-level cache?** A shared cache across sessions, for read-mostly entities.
51. **JPQL versus native query?** JPQL works on entities and is portable; native SQL accesses database-specific features.
52. **What is a projection?** Selecting only needed fields into an interface or a record DTO, avoiding managed entities.
53. **What does** `orphanRemoval = true` **do?** Deletes a child removed from the parent's collection.
54. `CascadeType.ALL` **on** `@ManyToOne`**?** Usually a bug: deleting a child would delete the shared parent.
55. **Why can bulk updates surprise you?** `@Modifying` queries bypass the persistence context, leaving managed entities stale.

37.16 Transactions

56. **What is ACID?** Atomicity, consistency, isolation and durability.
57. **Default propagation of** `@Transactional`**?** `REQUIRED`: join an existing transaction or start one.
58. `REQUIRES_NEW` **does what?** Suspends the current transaction and runs in a new, independent one.
59. `NESTED` **does what?** Uses a savepoint within the same transaction, rolling back only to it.
60. **Which exceptions roll back by default?** Unchecked exceptions and errors; checked exceptions commit unless `rollbackFor` says otherwise.
61. **What is "rollback-only"?** An inner `REQUIRED` method failed and marked the shared transaction, so the outer commit throws `UnexpectedRollbackException`.
62. **Does** `@Transactional` **work on private methods?** No, proxies only intercept public methods (protected also works with CGLIB since Spring 6).
63. **What does** `readOnly = true` **do?** Hints the driver and lets Hibernate skip dirty checking and flushing.

64. **Isolation levels, weakest first?** Read uncommitted, read committed, repeatable read, serializable.
65. **Dirty, non-repeatable and phantom reads?** Reading uncommitted data, a row changing between reads, and new rows appearing in a range.
66. **PostgreSQL's default isolation?** Read committed.
67. **What is a lost update?** Two transactions read and write the same row and one overwrites the other; prevent with versions or locks.
68. **What is write skew?** Two transactions read overlapping data and write different rows, breaking an invariant; serializable prevents it.
69. **Why avoid remote calls inside a transaction?** They hold connections and locks for the call's duration and cannot be rolled back.
70. **What is the dual-write problem?** Writing to a database and a broker separately risks one succeeding without the other.
71. **How does the outbox solve it?** The event is inserted in the same transaction and sent afterwards by a relay.
72. **What is a saga?** A sequence of local transactions with compensating actions instead of a distributed transaction.
73. **Choreography versus orchestration sagas?** Services react to events, versus a coordinator telling each step what to do.
74. **Why avoid two-phase commit in microservices?** It blocks on the coordinator, couples availability and is poorly supported by brokers.
75. **What does `TransactionTemplate` give you?** Programmatic transactions with precise boundaries, such as one transaction per batch.
76. **How do you run code after commit?** `@TransactionalEventListener(phase = AFTER_COMMIT)` or `TransactionSynchronization`.
77. **Does `@Transactional` on a test roll back?** Yes, Spring test transactions roll back by default.

37.17 Spring Security, OAuth2 and JWT

78. **How does Spring Security plug into the servlet stack?** `DelegatingFilterProxy` points to a `FilterChainProxy` holding one or more `SecurityFilterChain`s.
79. **How are several chains chosen?** By `securityMatcher` in `@Order`; the first match handles the request.
80. **Authentication versus authorisation?** Proving who you are versus deciding what you may do.
81. **What stores the current user?** `SecurityContextHolder`, thread-local by default.
82. **What replaced `WebSecurityConfigurerAdapter`?** Declaring `SecurityFilterChain` beans (Spring Security 5.7+, removed in 6).
83. **`hasRole("ADMIN")` versus `hasAuthority("ADMIN")`?** `hasRole` adds the `ROLE_` prefix.
84. **How do you enable `@PreAuthorize`?** `@EnableMethodSecurity`.
85. **When disable CSRF?** For stateless APIs using bearer tokens, not cookie sessions.
86. **What is a JWT?** A signed token of header, payload and signature, base64url-encoded; it is readable, not encrypted.
87. **How is a JWT validated?** Signature against the issuer's public key (JWKS), then expiry, issuer and audience.
88. **Why are JWTs hard to revoke?** They are valid until expiry; use short lifetimes, refresh tokens or a deny list.

89. **What is OAuth2?** A delegation framework for obtaining access tokens, not an authentication protocol.
90. **What does OIDC add?** An ID token and user info, making OAuth2 usable for sign-in.
91. **Which grant should a browser app use?** Authorisation code with PKCE.
92. **What is PKCE?** A code verifier and challenge that stop an intercepted authorisation code being exchanged.
93. **Which grant for service-to-service?** Client credentials.
94. **Resource server versus client?** A resource server validates tokens on APIs; a client obtains tokens.
95. **Where do you store passwords?** Hashed with bcrypt, scrypt or Argon2 through a `PasswordEncoder`, never encrypted or plain.
96. **How do you secure API keys?** Store only their hash, scope them, allow revocation and rotate them.
97. **What is SSRF?** Making the server fetch an attacker-chosen URL, such as cloud metadata; validate outbound URLs.
98. **How do you compare secrets safely?** In constant time, such as `MessageDigest.isEqual`.
99. **What is the OWASP Top 10's first item?** Broken access control.

37.18 SQL

00. **Execution order of a SELECT?** FROM and JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT.
01. **WHERE versus HAVING?** WHERE filters rows before grouping; HAVING filters groups after aggregation.
02. **INNER, LEFT, RIGHT, FULL join?** Matches only; all left rows; all right rows; all rows from both sides.
03. **Why does a WHERE on the right table turn a LEFT JOIN into an inner join?** It removes the null rows; put that condition in the ON clause instead.
04. `COUNT(*)` **versus** `COUNT(col)`**?** `COUNT(*)` counts rows; `COUNT(col)` skips nulls.
05. **Does** `NULL = NULL` **return true?** No, it returns unknown; use `IS NULL` or `IS NOT DISTINCT FROM`.
06. **Why is** `NOT IN` **with a null in the subquery dangerous?** It returns no rows; use `NOT EXISTS`.
07. `UNION` **versus** `UNION ALL`**?** `UNION` removes duplicates with a sort or hash; `UNION ALL` keeps them and is faster.
08. `ROW_NUMBER`**,** `RANK`**,** `DENSE_RANK`**?** Unique numbers; ties share a rank with gaps; ties share a rank without gaps.
09. **How do you find the second-highest salary?** `SELECT DISTINCT salary ... ORDER BY salary DESC OFFSET 1 LIMIT 1`, or `DENSE_RANK() = 2`.
10. **How do you find duplicates?** `GROUP BY` the columns with `HAVING COUNT(*) > 1`.
11. **How do you delete duplicates keeping one?** Delete rows whose `ROW_NUMBER()` over the duplicate key is greater than 1.
12. **Top N per group?** `ROW_NUMBER() OVER (PARTITION BY group ORDER BY value DESC)`, then filter `<= N`.
13. **What is a running total?** `SUM(x) OVER (ORDER BY date)`.
14. **What is a CTE?** A named subquery with `WITH`; `WITH RECURSIVE` walks hierarchies.
15. `EXISTS` **versus** `IN`**?** Both are semi-joins; `EXISTS` stops at the first match and handles nulls predictably.
16. **What does** `LAG` **do?** Reads a previous row's value in the window, such as yesterday's balance.
17. **How do you do an upsert in PostgreSQL?** `INSERT ... ON CONFLICT (key) DO UPDATE` or `DO NOTHING`.
18. **What does** `FOR UPDATE SKIP LOCKED` **do?** Locks chosen rows and skips rows other transactions have locked, the basis of a database queue.

19. **Why is `SELECT *` discouraged?** It reads unneeded columns, prevents index-only scans and breaks when columns change.
20. **Why is a function on an indexed column slow?** `WHERE lower(email) = ?` cannot use a plain index; add an expression index.
21. **What does `EXPLAIN ANALYZE` show?** The actual plan with real row counts and times, since it executes the query.

37.19 Database internals and PostgreSQL

22. **What is a B-tree index good for?** Equality, ranges, sorting and prefix matches, in O(log n).
23. **What is a composite index's leftmost prefix rule?** An index on (a, b) helps queries on a, or a and b, but not on b alone.
24. **Which column goes first in a composite index?** Equality columns before range columns, then consider selectivity.
25. **What is a covering index?** One holding every column a query needs, allowing an index-only scan (`INCLUDE` in PostgreSQL).
26. **Clustered versus non-clustered index?** A clustered index stores the table rows in key order (InnoDB primary key); PostgreSQL tables are heaps with separate indexes.
27. **When does the planner ignore an index?** When many rows match, statistics are stale, types mismatch, or a function wraps the column.
28. **Seq scan, index scan, bitmap scan?** Read everything, follow the index row by row, or collect matching pages first and read them in order.
29. **Nested loop, hash join, merge join?** Small outer with an indexed inner; large unsorted sets; inputs already sorted on the key.
30. **What is MVCC?** Each write creates a new row version, so readers see a snapshot and do not block writers.
31. **Why does PostgreSQL need VACUUM?** Old row versions remain as dead tuples until vacuum reclaims them and prevents transaction-id wraparound.
32. **What is table bloat?** Space held by dead tuples or index pages, slowing scans; autovacuum tuning or `pg_repack` fixes it.
33. **What is the WAL?** The write-ahead log: changes are logged durably before data pages, enabling crash recovery and replication.
34. **Row lock versus advisory lock?** Row locks protect rows; advisory locks are application-named locks, such as `pg_advisory_xact_lock`.
35. **How do you find a blocking query?** `pg_stat_activity` with `pg_blocking_pids()`.
36. **How does PostgreSQL resolve deadlocks?** It detects the cycle after `deadlock_timeout` and aborts one transaction.
37. **Why use a connection pool?** Each PostgreSQL connection is a process with real memory, so many connections hurt; pool them.
38. **How large should a Hikari pool be?** Small, roughly cores times two plus disks as a start, then measure; bigger is usually slower.
39. **Read replica trade-off?** Scales reads but lags the primary, so read-your-writes needs routing to the primary.
40. **Partitioning versus sharding?** Partitioning splits a table inside one database; sharding spreads data across several databases.

41. **What does `CREATE INDEX CONCURRENTLY` avoid?** Blocking writes while the index is built, at the cost of a slower build that can fail.

37.20 Caching and Redis

42. **Cache-aside pattern?** Read the cache, on a miss load from the database and populate the cache; the most common pattern.
43. **Write-through versus write-behind?** Write to cache and database together, versus write the cache now and the database later.
44. **What is a cache stampede?** Many requests miss the same key at once and all hit the database; use locking, early refresh or request coalescing.
45. **How do you invalidate on update?** Delete the key after the database commit rather than updating it, to avoid races.
46. **What is cache penetration?** Repeated lookups for keys that do not exist; cache the negative result or use a Bloom filter.
47. **Why add jitter to TTLs?** So many keys do not expire at the same moment.
48. **What does `@Cacheable` do?** Returns a cached value by key or calls the method and stores the result, through a proxy.
49. **`@CachePut` versus `@CacheEvict`?** Always runs and updates the cache, versus removes entries.
50. **Local versus distributed cache?** Local (Caffeine) is fastest but per instance; distributed (Redis) is shared but adds a network hop.
51. **Why is Redis fast?** In-memory data, efficient structures and a single-threaded command loop without locks.
52. **Redis data types?** Strings, hashes, lists, sets, sorted sets, streams, bitmaps and HyperLogLog.
53. **What is a sorted set used for?** Leaderboards, priority queues and sliding-window rate limits.
54. **RDB versus AOF persistence?** Periodic snapshots versus an append-only log of writes; often both are used.
55. **What is `SETNX` used for?** A simple lock: set if absent with an expiry, `SET key value NX PX 30000`.
56. **Is a Redis lock safe for correctness?** Not fully, because of expiry and failover; use fencing tokens or a database for critical sections.
57. **How do you make several Redis operations atomic?** A Lua script, or `MULTI` / `EXEC` transactions.
58. **Eviction policies?** Such as `allkeys-lru`, `allkeys-lfu`, `volatile-ttl` and `noeviction`.
59. **Redis Cluster key distribution?** 16,384 hash slots spread across primaries; hash tags `{user1}` keep related keys together.
60. **Sentinel versus Cluster?** Sentinel gives failover for one primary; Cluster shards data and fails over per shard.
61. **What is Caffeine's eviction policy?** W-TinyLFU, which usually beats plain LRU in hit rate.

37.21 Microservices

62. **Main benefits of microservices?** Independent deployment and scaling, fault isolation and team autonomy.
63. **Main costs?** Network failures, distributed data, operational complexity and harder debugging.

64. **How do you draw service boundaries?** Around business capabilities and data ownership (bounded contexts), not technical layers.
65. **What is a distributed monolith?** Services that must deploy together or share a database, getting the costs without the benefits.
66. **Database per service, why?** Each service owns its schema so it can change it independently.
67. **Synchronous versus asynchronous communication?** HTTP or gRPC is simple and immediate but couples availability; messaging decouples but is eventually consistent.
68. **What does an API gateway do?** Routing, authentication or rate limiting, TLS termination and aggregation at the edge.
69. **What is service discovery?** Finding instance addresses at runtime, client-side (Eureka) or server-side (Kubernetes services).
70. **Circuit breaker states?** Closed, open and half-open.
71. **Retry best practices?** Only idempotent operations, exponential backoff with jitter, and a cap.
72. **Why do timeouts matter?** Without them, a slow dependency exhausts threads and the failure spreads.
73. **What is a bulkhead?** Separate resource pools per dependency, so one cannot exhaust all threads.
74. **What is the strangler fig pattern?** Gradually routing features from a legacy system to new services until the old one can be removed.
75. **What is CQRS?** Separate models for writes and reads, often with read models built from events.
76. **What is event sourcing?** Storing state as an append-only series of events and rebuilding state by replaying them.
77. **Orchestration versus choreography?** A coordinator directs the steps, versus services reacting to each other's events.
78. **How do you handle consistency across services?** Sagas, the outbox pattern, idempotent consumers and compensating actions.
79. **What is the backend-for-frontend pattern?** A separate API layer tailored to one client type, such as mobile.
80. **What is contract testing?** Verifying that provider and consumer agree on an API, as with Pact or Spring Cloud Contract.
81. **When should you not use microservices?** Small teams, unclear domains or early products; start with a modular monolith.

37.22 Kafka and messaging

82. **Topic, partition, offset?** A named log, an ordered shard of it, and a message's position within the shard.
83. **Where is order guaranteed?** Only within a partition, so key related messages to the same partition.
84. **How does the producer choose a partition?** By hashing the key, or spreading keyless messages with the sticky partitioner.
85. **What is a consumer group?** Consumers sharing a topic; each partition goes to exactly one consumer in the group.
86. **More consumers than partitions?** The extra consumers sit idle; partitions cap parallelism.
87. **What is a rebalance?** Reassigning partitions when consumers join, leave or time out; cooperative rebalancing reduces the pause.
88. **What does `acks=all` mean?** The leader waits for all in-sync replicas before acknowledging.

89. **What does `min.insync.replicas` do?** With `acks=all`, rejects writes when fewer replicas than this are in sync.
90. **What does an idempotent producer prevent?** Duplicates from producer retries, using producer ids and sequence numbers.
91. **At-most-once, at-least-once, exactly-once?** Commit before processing; commit after processing; transactions with read-process-write inside Kafka.
92. **Does Kafka exactly-once cover your database?** No; only Kafka-to-Kafka, so use the outbox and idempotent consumers for databases.
93. **When should a consumer commit offsets?** After processing succeeds, accepting redeliveries, which idempotency absorbs.
94. **What is a dead-letter topic?** Where failing messages go after retries, so they do not block the partition.
95. **What is log compaction?** Keeping only the latest message per key, useful for changelog and state topics.
96. **What is consumer lag?** How far the committed offset trails the latest offset; the key health metric.
97. **What replaced ZooKeeper?** KRaft, Kafka's built-in Raft metadata quorum, the only mode in Kafka 4.0.
98. **Kafka versus RabbitMQ?** Kafka is a replayable, partitioned log for streams; RabbitMQ is a broker with routing and per-message acknowledgement for work queues.
99. **What is the poison pill problem?** A message that always fails and blocks its partition unless skipped or sent to a dead-letter topic.
00. **What does `auto.offset.reset=earliest` do?** A group with no committed offset starts from the oldest message.
01. **How do you increase throughput?** More partitions and consumers, batching (`linger.ms`, `batch.size`), compression and async sends.

37.23 System design

02. **First steps in a design interview?** Clarify functional and non-functional requirements, estimate scale, then sketch the high-level design.
03. **What is the CAP theorem?** During a network partition you must choose consistency or availability.
04. **What does PACELC add?** Even without a partition you trade latency against consistency.
05. **Vertical versus horizontal scaling?** A bigger machine versus more machines.
06. **What makes a service horizontally scalable?** Statelessness, with state in databases, caches or object storage.
07. **Load balancing algorithms?** Round robin, least connections, weighted, and consistent hashing for affinity.
08. **What is consistent hashing?** Keys and nodes on a ring, so adding a node moves only a fraction of keys.
09. **Strong versus eventual consistency?** Every read sees the latest write, versus replicas converging over time.
10. **What does 99.9% availability allow?** About 43 minutes of downtime a month, or 8.8 hours a year.
11. **How do you design a URL shortener's key?** Base62 of a unique id or a random code, with collision checks, cached reads and a redirect.
12. **How do you design a rate limiter?** A token bucket or sliding window per key, shared state in Redis for many nodes, answering 429.
13. **How do you design a notification system?** A queue per channel, workers with retries and idempotency keys, user preferences and rate limits.

14. **How do you design a news feed?** Fan-out on write for most users, fan-out on read for celebrities, cached timelines.
15. **How do you handle large file uploads?** Pre-signed URLs to object storage, multipart upload, then asynchronous processing.
16. **What is back-of-the-envelope estimation?** Rough QPS, storage and bandwidth from users and usage, to size components.
17. **Push versus pull for workers?** Push gives low latency but needs backpressure; pull lets workers take what they can handle.
18. **How do you make a scheduler run each job once across instances?** Claim rows with `FOR UPDATE SKIP LOCKED`, a lease, or a leader election.
19. **How do you design for multi-tenancy?** Choose shared schema, schema per tenant or database per tenant, then enforce isolation and per-tenant limits.
20. **How do you avoid a single point of failure?** Redundant instances, replicated data, health checks and automatic failover.

37.24 DevOps, cloud and observability

21. **Docker image versus container?** An image is a read-only layered template; a container is a running instance of it.
22. **Why use multi-stage builds?** To build with the JDK and ship only a small runtime image.
23. **What do layered Spring Boot jars give Docker?** Separate layers for dependencies and application code, so rebuilds are faster.
24. **Pod, Deployment, Service in Kubernetes?** The smallest running unit, a managed replica set with rollouts, and a stable network endpoint.
25. **ConfigMap versus Secret?** Plain configuration versus sensitive values (base64-encoded, not encrypted unless configured).
26. **Liveness, readiness and startup probes?** Restart when broken, route traffic when ready, and give slow starts time.
27. **What does a horizontal pod autoscaler do?** Scales replicas on CPU, memory or custom metrics.
28. **Requests versus limits?** Requests are guaranteed for scheduling; limits are the ceiling, and exceeding the memory limit kills the container.
29. **Rolling, blue-green and canary deployment?** Gradual replacement; switch between two environments; a small share of traffic first.
30. **What are the three pillars of observability?** Metrics, logs and traces.
31. **RED and USE methods?** Rate, errors and duration for services; utilisation, saturation and errors for resources.
32. **Why percentiles rather than averages?** Averages hide the slow tail that users feel; watch p95 and p99.
33. **What is distributed tracing?** Following one request across services with a shared trace id, propagated by W3C `traceparent`.
34. **Why structured logs?** JSON fields can be searched and aggregated; plain text cannot reliably.
35. **SLI, SLO and SLA?** A measured indicator, an internal target, and a contractual promise with penalties.
36. **What is an error budget?** The failure an SLO allows; when spent, slow down releases and fix reliability.
37. **CI versus CD?** Continuous integration builds and tests every change; continuous delivery keeps it always deployable or deploys it.

38. **What is infrastructure as code?** Declaring infrastructure in versioned files, such as Terraform or Helm.
39. **How should secrets reach an application?** From a secret manager or injected environment, never committed to Git.
40. **What is Prometheus's model?** It scrapes metrics over HTTP from targets and stores time series queried with PromQL.

37.25 Testing

41. **What is the test pyramid?** Many fast unit tests, fewer integration tests, and few end-to-end tests.
42. **Mock versus stub versus spy?** A mock verifies interactions; a stub returns canned answers; a spy wraps a real object.
43. **`@Mock` versus `@MockBean`?** `@Mock` is plain Mockito; `@MockBean` replaces a bean in the Spring context, deprecated in Boot 3.4 for `@MockitoBean`.
44. **`@SpringBootTest` versus slice tests?** The full context, versus `@WebMvcTest` or `@DataJpaTest` loading only one layer.
45. **Why are many `@MockBean`s slow?** Each different set creates a new application context instead of reusing the cached one.
46. **What is Testcontainers?** Real databases and brokers in Docker for integration tests.
47. **Why test against a real database rather than H2?** H2 differs in SQL, locking and features, so tests can pass while production fails.
48. **What does `@Transactional` on tests do?** Rolls back each test, but hides commit-time behaviour such as constraint checks at flush.
49. **How do you test time-dependent code?** Inject a `Clock` and use a fixed one in tests.
50. **How do you test asynchronous code?** Wait for a condition with a timeout (Awaitility), not `Thread.sleep`.
51. **What is mutation testing?** Changing code automatically to check whether tests fail, as PIT does.
52. **What is code coverage's limit?** It shows lines executed, not whether behaviour is checked.
53. **What is a flaky test?** One that passes and fails without code changes, from timing, order or shared state.
54. **What is contract testing for?** Catching breaking API changes between services before deployment.
55. **What is a load test versus a stress test?** Expected load to check targets, versus beyond capacity to find the breaking point.
56. **What is TDD's cycle?** Red, green, refactor.
57. **How do you test a REST controller?** `MockMvc` or `WebTestClient` with JSON assertions.
58. **What are parameterised tests?** One test run with many inputs via `@ParameterizedTest` and sources such as `@CsvSource`.
59. **Given-when-then?** Arrange the state, perform the action, and assert the outcome.
60. **What makes a unit test good?** Fast, isolated, deterministic, testing behaviour rather than implementation, with a clear name.

37.26 AI and LLM integration

61. **What is a token?** A piece of text, roughly three-quarters of an English word, that models read and are priced by.

62. **What is a context window?** The maximum tokens of input plus output a model can handle in one request.
63. **What does temperature control?** Randomness of sampling; low values make answers more deterministic.
64. **What is a system prompt?** Instructions that set the model's role, rules and output format for the conversation.
65. **What is a hallucination?** A fluent but false answer; reduce it with grounding, citations and validation.
66. **How do you get structured output?** JSON mode or schema-constrained output, a precise prompt, then parse and validate.
67. **What is tool calling?** The model returns a structured request to call a function; your code runs it and returns the result.
68. **What is an agent?** A loop where the model chooses tools step by step until the task is done, with limits on steps.
69. **What is prompt injection?** Untrusted text that tells the model to ignore its instructions; treat model output and tool input as untrusted.
70. **How do you control LLM cost?** Smaller models for simple tasks, prompt caching, shorter context, batching and limits per workspace.
71. **What is prompt caching?** Reusing the processed prefix of repeated prompts to cut cost and latency.
72. **Why stream responses?** Users see the first tokens quickly instead of waiting for the whole answer.
73. **How do you make the provider swappable?** A small interface such as `LlmClient`, with one adapter per provider chosen by configuration.
74. **What is Spring AI's `ChatClient`?** A fluent Spring abstraction over chat models, with advisors, tools and structured output.
75. **How do you evaluate LLM features?** A fixed test set with expected answers, automated checks, and human review of samples.
76. **Why not use an LLM for arithmetic or statistics?** Code is exact, cheap and explainable; use the model for language.
77. **How do you handle sensitive data?** Send the minimum, mask fields, use a local model where needed, and keep data sharing off by default.
78. **What is fine-tuning versus prompting?** Training on examples to change behaviour, versus instructing at request time; try prompting and retrieval first.
79. **How do you handle model timeouts and rate limits?** Timeouts, retries with backoff on 429 and 5xx, and a clear error to the user.
80. **What is a guardrail?** A check before or after the model, such as input filtering, output validation or human approval.

37.27 RAG, vectors and MCP

81. **What is RAG?** Retrieval-augmented generation: find relevant documents and put them in the prompt so the answer is grounded.
82. **What is an embedding?** A vector of numbers representing meaning, so similar texts are close together.
83. **Which similarity measure is common?** Cosine similarity, or dot product on normalised vectors.
84. **What is a vector database?** A store indexing embeddings for nearest-neighbour search, such as pgvector, Qdrant or Pinecone.
85. **What is HNSW?** A graph-based approximate nearest-neighbour index, fast with high recall.

86. **Why chunk documents?** Embeddings work best on focused passages and context windows are limited.
87. **What chunk size is typical?** A few hundred tokens with some overlap, tuned on your own data.
88. **What is hybrid search?** Combining keyword (BM25) and vector search, then merging the rankings.
89. **What is re-ranking?** A second model reordering the retrieved candidates by relevance to the question.
90. **Why does RAG return wrong answers?** Poor chunking, the wrong documents retrieved, or missing data; measure retrieval separately.
91. **How do you keep tenants apart in RAG?** Filter by tenant in the vector query itself, never only in the prompt.
92. **How do you cite sources?** Return chunk ids with the answer and show links to them.
93. **What is MCP?** The Model Context Protocol, an open standard for exposing tools, resources and prompts to AI assistants.
94. **MCP's three primitives?** Tools the model calls, resources it reads, and prompts as reusable templates.
95. **MCP transports?** stdio for local servers and streamable HTTP for remote ones.
96. **How should an MCP server handle authorisation?** Use the caller's own credentials against the real API, so it can do only what they can.
97. **Why cap MCP tool output?** Large results flood the assistant's context and raise cost.
98. **Which actions should an AI tool not perform?** Irreversible or high-risk ones such as approvals or payments, without a human confirming.
99. **RAG versus fine-tuning for company knowledge?** RAG for changing facts with citations; fine-tuning for style or format.
00. **How do you evaluate RAG?** Retrieval recall and precision on a labelled set, plus answer faithfulness to the retrieved text.